TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ HỌA MÁY TÍNH

BÀI TẬP LỚP NHÀ

Giảng viên: Thầy Lý Quốc Ngọc

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên	Lớp
Tô Thành Đạt	24127153	24TGMT
Lương Hưng Phát	24127298	24TGMT
Nguyễn Nhựt Huy	24127398	24TGMT

TP. Hồ Chí Minh, tháng 10/2026

BÀI TẬP LỚP NHÀ: NHÓM GITREMOVEALL

Tô Thành Đạt – 24127153
Khoa Công nghệ Thông tin
Trường ĐH Khoa học Tự nhiên
24127153@student.hcmus.edu.vn

Lương Hưng Phát – 24127298
Khoa Công nghệ Thông tin
Trường ĐH Khoa học Tự nhiên
24127298@student.hcmus.edu.vn

Nguyễn Nhựt Huy – 24127398
Khoa Công nghệ Thông tin
Trường ĐH Khoa học Tự nhiên
24127398@student.hcmus.edu.vn

ABSTRACT

Báo cáo này tổng hợp toàn bộ nội dung bài học, câu hỏi lý thuyết và bài tập môn Đồ họa Máy tính (Computer Graphics) theo lộ trình 12 tuần học.

MỤC LỤC

1 Tuần 1: Tổng quan Khoa học Dữ liệu Thị giác & Computer Graphics **4**

1.1 Đối tượng nghiên cứu của Khoa học Dữ liệu Thị giác và Dữ liệu Đa phương thức . 4
1.2 Mục tiêu và bản chất của cuộc Cách mạng Công nghiệp 4.0 4
1.3 Tiến trình phát triển kỹ thuật qua 4 cuộc Cách mạng Công nghiệp 4
1.4 Vai trò của dữ liệu thị giác trong sản xuất và quản lý xã hội 5
1.5 Tác động của dữ liệu thị giác trên các quy trình sản xuất ngành nghề 5
1.6 Nguồn gốc dữ liệu thị giác và Sự khác biệt giữa mắt người và thiết bị thu nhận hình ảnh 6
1.7 Hệ thống các loại đặc trưng trong dữ liệu thị giác 6
1.8 Khái niệm đối tượng trong dữ liệu thị giác: Phân biệt cấp độ lớp và cấp độ thực thể 6
1.9 Bản chất và thách thức của khoảng cách ngữ nghĩa (Semantic Gap) 7
1.10 Tiến trình phát triển trong việc thu hẹp khoảng cách ngữ nghĩa 7
1.11 Động lực mở rộng từ dữ liệu hình ảnh sang dữ liệu đa phương thức 7
1.12 Phân định ranh giới giữa Đồ họa máy tính, Xử lý ảnh và Thị giác máy tính 8
1.13 Khái quát các bài toán cốt lõi trong Thị giác máy tính 8
1.14 So sánh bài toán Nhận dạng (Recognition) và Truy vấn ảnh (Image Retrieval) . . . 11
1.15 So sánh bài toán Truy vấn ảnh (Retrieval) và Theo dõi đối tượng (Tracking) 11
1.16 So sánh bài toán Đếm đối tượng (Counting) và Phát hiện đối tượng (Object Detection) 11
1.17 Mô hình Tạo sinh (Generative) và tương quan với các tác vụ Phân tích (Discriminative) 11

2 Tuần 2: Trí tuệ Nhân tạo, Học máy & Học sâu trong Khoa học Dữ liệu Thị giác **13**

2.1 Bản chất của Trí tuệ Tự nhiên, Trí tuệ Nhân tạo và Khái niệm Thông minh 13
2.2 Tác động của Trí tuệ Nhân tạo đối với sự phát triển của Trí tuệ Tự nhiên 14

2.3 Phân định bản chất giữa Thông minh (Intelligence) và Thông tuệ (Wisdom) 16
2.4 Giải thích Luận điểm: "Thông minh là Khả năng Thích ứng với sự Thay đổi" . . . 16
2.5 Bản chất của Học máy: Phân tích tư tưởng Arthur Samuel (1959) 17
2.6 Định nghĩa Hình thức về Học máy theo Tom Mitchell (1997) 17
2.7 Tiến trình Chuyển tiếp từ Học máy sang Học sâu: Giới hạn và Động lực Đột phá . . 17
2.8 Nền tảng của Học sâu: Vì sao Nguyên lý Đơn giản tạo nên Khả năng Biểu diễn Vượt trội? 18
2.9 Mô hình 5 Cấp độ Nhận thức và Dữ liệu: Ứng dụng Giám sát An ninh Trường học . 19
2.10 Mô hình Giải pháp 3 Tầng trong Khoa học Dữ liệu Thị giác 19
2.11 Khái quát các Nhóm Ứng dụng Cơ bản của Thị giác Máy tính 20
2.12 Phân tích Chuyên sâu: Ứng dụng Trí tuệ Nhân tạo trong Ngành Điện ảnh 21
2.13 Ý nghĩa Thực tiễn của Bài toán Đếm Đám đông ở Tầng Ứng dụng Nghiệp vụ . . . 21
2.14 Phân tích Hệ thống Tái tạo Khuôn mặt 3D theo Mô hình Giải pháp 3 Tầng 21

3 Tuần 4: Hệ tọa độ, Thuật toán Vẽ Đường thẳng, Đường tròn và Đường Elip 22

3.1 Phân biệt ba hệ tọa độ WCS, UCS và DCS 22
3.2 Vì sao mô phỏng được thế giới thực bằng các vector có kích thước nhỏ 23
3.3 Hiển thị thế giới thực rất lớn trên màn hình 2D hữu hạn 24
3.4 Giải thuật vẽ đoạn thẳng theo phương pháp trên bảng (tổng quát cho mọi hệ số góc) 25
3.5 Vì sao giới hạn bài toán trong giả thiết $0 < m < 1$, $x_a < x_b$ và cách suy diễn các trường hợp còn lại 26
3.6 So sánh Bresenham với Cách 2: vì sao Bresenham khử được phép cộng số thực . . 28
3.7 Tính giá trị khởi tạo p_0 28
3.8 Vì sao phương pháp Bresenham loại được phép nhân và số thực 29
3.9 Thuật toán vẽ đường tròn cho kết quả "nham nhở" thì có được áp dụng hay không . 29
3.10 Chọn pixel để đảm bảo liên tục trong lân cận 8 khi vẽ đường tròn 30
3.11 Vị trí midpoint quyết định pixel nào gần đường tròn hơn 30
3.12 Khử phân số $5/4$ trong p_0 của thuật toán vẽ đường tròn 31
3.13 Tọa độ 7 điểm đối xứng còn lại trong thuật toán vẽ đường tròn 32
3.14 Mô tả quá trình Text-to-Vector (theo bài đăng text2vector) 33
3.15 Ý nghĩa của góc α trong hệ tọa độ cực khi vẽ đường elip 34

4 Tuần 5: Vẽ Cung tròn – Ellipse, Đường cong Bézier và Tô màu Đa giác theo Dòng quét 37

4.1 Phương pháp vẽ cung tròn và cung ellipse trên lưới tọa độ nguyên 37
4.2 Khó khăn khi biểu diễn đường cong bằng quan hệ tường minh $y = f(x)$ 38
4.3 Vẽ đường cong Bézier trên lưới tọa độ nguyên 38
4.4 Đường cong Bézier bậc 1 nối hai đầu mút làm tiền đề cho điều gì 39
4.5 Duy trì bậc thấp nhưng số điểm điều khiển cao 40
4.6 Ý nghĩa các biểu thức trong slide trang 5 (Bézier) 40
4.7 Ý nghĩa của $\mathbf{P}'(0) = n(\mathbf{p}_1 - \mathbf{p}_0)$ 41

4.8 Thực hiện các phép biến đổi hình học trên đường cong Bézier đã vẽ 41
4.9 Vế trái, vế phải trong slide trang 7 (tính bất biến affine) và kết luận 42
4.10 Phương trình đường cong Bézier bậc 3 và cách vẽ 42
4.11 Kiểm tra đạo hàm bậc 2 của đường cong Bézier bậc 3 43
4.12 Phương trình đoạn thẳng theo tham số 44
4.13 Chứng minh điểm De Casteljau $\mathbf{p}_0^2(t)$ trùng với đường cong Bézier 44
4.14 Thiết kế giao diện người dùng để vẽ đường cong Bézier bậc 3 45
4.15 Nhận xét đường spline Hermite so với đường cong Bézier bậc 3 và so sánh tính chất của hai phương trình 45
4.16 Xác định một điểm nằm trong hay ngoài đa giác (góc độ máy tính) 47
4.17 Định lý Jordan và cách chứng minh (trường hợp đa giác) 48
4.18 Vì sao lấp đầy các cặp giao điểm lẻ–chẵn ở bước L4 thì lấp được miền trong 49
4.19 Trường hợp dòng quét $i = 7$ trong ví dụ của slide 49
4.20 Rút ngắn cạnh tại đỉnh không cực trị 50
4.21 Vì sao chỉ xử lý đỉnh không cực trị mà không xử lý đỉnh cực trị 51
4.22 Ý nghĩa từng trường trong cấu trúc AEL 52

5 Tuần 6: Cải tiến Tô màu theo Dòng quét và Các phép Biến đổi Hình học 2D, 3D 54
5.1 Tô màu theo dòng quét nghiêng một góc α, khoảng cách giữa hai dòng là a điểm ảnh 54
5.2 Tô màu theo dòng quét với biên là đường tròn, ellipse 54
5.3 Vai trò của phép biến đổi hình học trong đồ họa máy tính 55
5.4 Phép tịnh tiến 2D: ý nghĩa và ứng dụng 55
5.5 Phép quay quanh một tâm bất kỳ 56
5.6 Ma trận co giãn (tỉ lệ) với tâm C 56
5.7 Phép đối xứng qua một trục bất kỳ 57
5.8 Vai trò của phép biến đổi trong slide trang 15 57
5.9 Sự cần thiết của việc đồng bộ viewport-to-window và window-to-viewport 58
5.10 Toán tử zoom (đầu vào, đầu ra) và toán tử pan 59
5.11 Vai trò của v2w, w2v, zoom, pan trong quy trình hoạt động của phần mềm đồ họa . 59
5.12 Vai trò của các phép biến đổi 3D 60
5.13 Màn hình là 2 chiều thì biến đổi 3D như thế nào 60
5.14 Phép quay quanh một trục bất kỳ trong không gian 3D 61
5.15 Phép biến đổi quay vector $\mathbf{v}$ trùng với trục Oz 62
5.16 Chứng minh hai góc α trong hình trang 28 bằng nhau 62
5.17 Vì sao tích vô hướng và tích có hướng là công cụ hiệu quả trong đồ họa máy tính . 62
5.18 Nguồn gốc dòng 2 của tích có hướng trong slide trang 29 63

1 Tuần 1: Tổng quan Khoa học Dữ liệu Thị giác & Computer Graphics

1.1 Đối tượng nghiên cứu của Khoa học Dữ liệu Thị giác và Dữ liệu Đa phương thức

Khoa học Dữ liệu Thị giác (Visual Data Science) là lĩnh vực liên ngành nghiên cứu các nguyên lý toán học, cấu trúc dữ liệu và mô hình tính toán nhằm thu thập, biểu diễn, trích xuất đặc trưng và suy luận tri thức từ dữ liệu trực quan. Miền dữ liệu này bao gồm hình ảnh số 2D, chuỗi khung hình video có tính tương quan thời gian, đám mây điểm 3D (Point Clouds) từ thiết bị quét laser, dữ liệu thể tích voxel trong chẩn đoán y khoa (chụp cắt lớp vi tính CT, cộng hưởng từ MRI), cho đến ảnh viễn thám đa phổ từ vệ tinh.

Nhằm vượt qua giới hạn biểu diễn của từng kênh cảm biến đơn lẻ, ngành khoa học này đã mở rộng phạm vi sang dữ liệu đa phương thức (Multimodal Data). Việc thiết lập liên kết biểu diễn giữa dữ liệu thị giác với các phương thức khác như văn bản tự nhiên, tín hiệu âm thanh hay chuỗi đo lường quán tính (IMU) cho phép giải quyết các bài toán nhận thức không gian và ngữ cảnh phức tạp. Điển hình cho hướng tiếp cận này là bài toán phân đoạn đối tượng trong video dựa trên mô tả hành động bằng văn bản hoặc âm thanh dẫn đường (Referring Motion Expression Video Segmentation) (Ding et al., 2025). Thay vì chỉ định mục tiêu bằng các chỉ số lớp cố định, mô hình tiếp nhận câu mô tả tự nhiên hoặc âm thanh đặc trưng để khoanh vùng chính xác mặt nạ pixel (segmentation mask) của đối tượng chuyển động xuyên suốt các khung hình.

1.2 Mục tiêu và bản chất của cuộc Cách mạng Công nghiệp 4.0

Trọng tâm của cuộc Cách mạng Công nghiệp 4.0 là quá trình thông minh hóa (Smartification) toàn diện các quy trình sản xuất công nghiệp và hệ thống quản trị xã hội trên nền tảng chuyển đổi số.

Quá trình thông minh hóa được hiện thực hóa thông qua việc tích hợp các mô hình học máy và mạng lưới cảm biến vạn vật (IoT) vào thiết bị phần cứng. Nhờ đó, hệ thống kỹ thuật đạt được năng lực tự thu thập dữ liệu môi trường, tự hiệu chỉnh thông số vận hành và ra quyết định tối ưu theo thời gian thực mà **không cần sự can thiệp thủ công liên tục từ con người**.

Trong lĩnh vực sản xuất công nghiệp, quá trình này giúp nâng cao năng suất lao động, giảm thiểu chi phí vận hành, loại bỏ sai sót chủ quan và hiện thực hóa mô hình cá nhân hóa sản phẩm hàng loạt (Mass Customization). Trong quản trị xã hội, việc triển khai các hệ thống điều khiển giao thông thích ứng, lưới điện thông minh và nền tảng y tế số góp phần tối ưu hóa dịch vụ công và phân bổ nguồn lực đô thị minh bạch, hiệu quả.

Tiến trình chuyển đổi công nghệ trong kỷ nguyên mới diễn ra qua ba cấp độ kế thừa:

1. **Số hóa dữ liệu (Digitization):** Quá trình chuyển đổi dữ liệu từ dạng vật lý hoặc tương tự (tài liệu giấy, hình ảnh chụp, băng từ analog) sang định dạng nhị phân số.
2. **Số hóa quy trình (Digitalization):** Việc ứng dụng công nghệ kỹ thuật số và dữ liệu đã số hóa vào việc tối ưu hóa hoặc tự động hóa các quy trình nghiệp vụ sẵn có mà chưa làm thay đổi bản chất mô hình vận hành tổng thể.
3. **Chuyển đổi số (Digital Transformation):** Sự tái cấu trúc toàn diện mô hình hoạt động, quy trình nghiệp vụ, phương pháp ra quyết định và văn hóa tổ chức dựa trên việc khai thác dữ liệu và trí tuệ nhân tạo.

Trong hệ sinh thái công nghệ 4.0, các ngành khoa học cơ bản như Toán học, Vật lý, Hóa học và Sinh học đóng vai trò nền tảng trong việc xây dựng cơ sở lý thuyết và chế tạo linh kiện bán dẫn, cảm biến quang học. Tuy nhiên, Trí tuệ nhân tạo (AI) đóng vai trò cốt lõi bởi đây là công cụ tính toán duy nhất có khả năng trích xuất tri thức từ các tập dữ liệu phi cấu trúc khổng lồ và đưa ra quyết định tự hành trong môi trường thay đổi liên tục.

1.3 Tiến trình phát triển kỹ thuật qua 4 cuộc Cách mạng Công nghiệp

Lịch sử phát triển sản xuất của văn minh nhân loại được đánh dấu qua 4 bước chuyển dịch công nghệ nền tảng:

Cách mạng Công nghiệp 1.0 (Cuối thế kỷ 18 đến đầu thế kỷ 19): Khởi đầu từ việc thương mại hóa động cơ hơi nước và ứng dụng cơ giới hóa vào ngành dệt. Bước tiến này đã thay thế sức kéo cơ bắp của con người và động vật bằng năng lượng cơ khí, đặt nền móng cho các đô thị công nghiệp đầu tiên.

Cách mạng Công nghiệp 2.0 (Cuối thế kỷ 19 đến đầu thế kỷ 20): Được thúc đẩy bởi việc phát minh ra năng lượng điện, động cơ đốt trong và dây chuyền lắp ráp liên tục (Assembly Line). Đột phá này đưa phương thức sản xuất hàng loạt với chi phí thấp trở thành chuẩn mực công nghiệp, đồng thời thúc đẩy sự bùng nổ của mạng lưới giao thông vận tải và viễn thông hiện đại.

Cách mạng Công nghiệp 3.0 (Thập niên 1960 đến cuối thế kỷ 20): Chuyển trọng tâm sang công nghệ bán dẫn, máy vi tính cá nhân và mạng Internet. Sự ra đời của các bộ điều khiển logic lập trình được (PLC) trong tự động hóa nhà máy đã thay đổi căn bản cách thức vận hành công nghiệp và mở ra kỷ nguyên số hóa thông tin.

Cách mạng Công nghiệp 4.0 (Từ đầu thế kỷ 21 đến nay): Hướng tới xây dựng các hệ thống không gian mạng - vật lý (Cyber-Physical Systems - CPS), nơi thế giới thực và mô hình số đồng bộ hai chiều thông qua AI, IoT, dữ liệu lớn và điện toán đám mây, tiến tới mô hình nhà máy tự hành và đô thị thông minh.

1.4 Vai trò của dữ liệu thị giác trong sản xuất và quản lý xã hội

Xét về mặt nhận thức sinh học, thị giác cung cấp hơn 80% tổng lượng thông tin mà con người tiếp nhận từ thế giới xung quanh. Dưới góc độ kỹ thuật, dữ liệu thị giác đại diện cho phương thức thu thập thông tin không tiếp xúc (non-contact) và không phá hủy mẫu vật (non-destructive), mang lại mật độ thông tin không gian - thời gian vượt trội so với các loại cảm biến đo điểm truyền thống.

Trong sản xuất công nghiệp, hệ thống camera thị giác máy tính quét bề mặt sản phẩm với tần số cao để đo đạc kích thước hình học, kiểm tra độ phẳng và phát hiện khuyết tật vật liệu theo thời gian thực. Trong quản lý xã hội, luồng dữ liệu hình ảnh và video từ camera giám sát đô thị cùng ảnh vệ tinh viễn thám cung cấp thông tin không gian chi tiết, phục vụ công tác điều phối giao thông, phát hiện ùn tắc, giám sát an ninh trật tự và ứng phó thiên tai kịp thời.

1.5 Tác động của dữ liệu thị giác trên các quy trình sản xuất ngành nghề

Ứng dụng của dữ liệu thị giác hiện diện sâu rộng trong nhiều lĩnh vực sản xuất và quản lý tài nguyên:

Trong công nghiệp sản xuất và chế tạo cơ khí: Dữ liệu thị giác đóng vai trò cốt lõi trong hệ thống kiểm tra quang học tự động (Automated Optical Inspection - AOI) nhằm phát hiện các vết nứt vi mô, lỗi hàn vi mạch trên bo mạch điện tử với độ chính xác micromet. Đồng thời, thị giác robot (Vision-Guided Robotics) cung cấp tọa độ không gian 3D để cánh tay robot tự động gắp thả linh kiện, hàn điểm và lắp ráp phụ tùng trên dây chuyền tốc độ cao. Ngoài ra, hệ thống camera thông minh còn giám sát việc chấp hành trang bị bảo hộ cá nhân (PPE) và cảnh báo công nhân khi bước vào vùng nguy hiểm quanh máy móc nặng.

Trong nông nghiệp công nghệ cao: Hình ảnh kỹ thuật số hỗ trợ phân loại nông sản tự động sau thu hoạch dựa trên kích thước, màu sắc, độ chín và phát hiện dị tật bề mặt. Trên cánh đồng, camera gắn trên máy bay không người lái (drone) kết hợp thuật toán thị giác giúp phát hiện sớm sâu bệnh và nấm lá, cho phép phun thuốc bảo vệ thực vật chính xác theo từng cụm cây nhiễm bệnh. Ảnh đa phổ từ drone còn được sử dụng để tính toán chỉ số sinh khối (NDVI), hỗ trợ dự báo năng suất mùa màng chính xác.

Trong lâm nghiệp và quản lý môi trường: Dữ liệu ảnh vệ tinh đa thời gian được phân tích để theo dõi diễn biến độ che phủ rừng, phát hiện kịp thời các hoạt động phá rừng hoặc lấn chiếm đất lâm nghiệp trái phép. Bên cạnh đó, hệ thống camera nhiệt và camera quang học tầm xa đặt tại các tháp quan sát hỗ trợ phát hiện sớm cột khói và các điểm cháy rừng bất thường từ khoảng cách hàng chục kilômét.

1.6 Nguồn gốc dữ liệu thị giác và Sự khác biệt giữa mắt người và thiết bị thu nhận hình ảnh

Dữ liệu thị giác được thu thập từ đa dạng các loại cảm biến quang điện tử: camera quang học dải khả kiến (cảm biến CCD/CMOS), camera nhiệt hồng ngoại, thiết bị quét laser LiDAR, cảm biến thời gian bay (Time-of-Flight - ToF), thiết bị chụp cắt lớp y tế (X-ray, CT, MRI, siêu âm) cũng như dữ liệu tổng hợp từ các engine đồ họa 3D.

Xét về mặt cơ chế thu nhận và xử lý, giữa hệ thống thị giác sinh học của con người và thiết bị thu nhận ảnh kỹ thuật số tồn tại những khác biệt nền tảng:

Cơ chế cảm nhận và xử lý tín hiệu: Võng mạc mắt người chứa hàng triệu tế bào thụ cảm quang phân bố không đồng đều, gồm tế bào que cảm nhận độ sáng trong điều kiện thiếu sáng và tế bào nón cảm nhận màu sắc tập trung với mật độ cao nhất tại vùng hoàng điểm (fovea). Tín hiệu thị giác được tiền xử lý cục bộ ngay tại các tầng tế bào thần kinh võng mạc trước khi truyền qua dây thần kinh thị giác lên vỏ não. Ngược lại, cảm biến máy ảnh kỹ thuật số sử dụng mảng lưới điểm ảnh đồng nhất được phủ bộ lọc màu Bayer (RGB), thực hiện biến đổi tuyến tính photon ánh sáng thành điện tích để tạo ra ma trận số thô.

Dải động học (Dynamic Range): Mắt người sở hữu khả năng thích ứng động học linh hoạt vượt quá 20 stop ($> 100,000 : 1$), cho phép nhận biết đồng thời các chi tiết nằm trong vùng bóng râm sâu và vùng nguồn sáng chói. Cảm biến kỹ thuật số thương mại thông thường chỉ đạt dải động khoảng 10–14 stop, dễ dẫn đến hiện tượng cháy sáng ở vùng quá sáng hoặc mất chi tiết do nhiễu ở vùng tối.

Dải phổ điện từ (Spectral Range): Thị giác con người bị giới hạn nghiêm ngặt trong dải ánh sáng khả kiến với bước sóng từ 380 nm đến 740 nm. Thiết bị thu nhận nhân tạo có thể mở rộng độ nhạy quang phổ sang tia X, tia tử ngoại, dải hồng ngoại gần, hồng ngoại nhiệt và sóng radar vô tuyến.

Độ chính xác định lượng và tính ổn định: Thiết bị quang học đo lường cường độ bức xạ ánh sáng bằng các giá trị số học tuyệt đối và hoạt động liên tục với tính ổn định cao. Ngược lại, nhận thức thị giác của con người mang tính định tính, chịu sự chi phối của tâm lý học nhận thức và dễ bị suy giảm độ chính xác do ảo ảnh thị giác hoặc mệt mỏi thể chất.

1.7 Hệ thống các loại đặc trưng trong dữ liệu thị giác

Bên cạnh các đặc trưng thị giác mức thấp (màu sắc, kết cấu texture, hình dạng đường biên) và đặc trưng ngữ nghĩa mức cao (nhãn phân loại lớp đối tượng), việc biểu diễn dữ liệu thị giác hiện đại còn khai thác nhiều nhóm đặc trưng chuyên sâu:

Đặc trưng không gian (Spatial Features): Bao gồm độ sâu từng pixel, tọa độ không gian 3D thực tế, vector pháp tuyến bề mặt, góc xoay và tịnh tiến 6 bậc tự do (6-DoF Pose), cấu trúc đám mây điểm và lưới đa giác 3D (Polygon Mesh).

Đặc trưng thời gian (Temporal Features): Bao gồm luồng quang học (Optical Flow) biểu diễn vector chuyển động của từng pixel giữa hai khung hình liên tiếp (u, v), quỹ đạo di chuyển của đối tượng qua thời gian, cùng vận tốc và gia tốc chuyển động.

Đặc trưng cấu trúc và quan hệ (Relational Features): Được mô hình hóa dưới dạng Đồ thị quan hệ cảnh (Scene Graphs), trong đó các nút đại diện cho các thực thể và các cạnh mô tả quan hệ không gian hoặc tương tác người - vật (Human-Object Interaction - HOI), giúp hệ thống nắm bắt ngữ cảnh toàn diện thay vì các đối tượng đơn lẻ.

Đặc trưng thống kê và tần số (Statistical Features): Bao gồm phổ tần số không gian thông qua biến đổi Fourier, biểu diễn đa tỷ lệ qua biến đổi Wavelet, ma trận đồng xuất mức xám (GLCM) và ma trận hiệp phương sai đặc trưng.

1.8 Khái niệm đối tượng trong dữ liệu thị giác: Phân biệt cấp độ lớp và cấp độ thực thể

Trong thị giác máy tính, một đối tượng thị giác (Visual Object) là một vùng không gian hoặc tập hợp các pixel liên thông tương ứng với một thực thể vật lý xác định trong thế giới thực, có ranh giới phân định rõ ràng với môi trường nền xung quanh.

Việc định danh đối tượng được phân chia thành hai cấp độ nhận thức:

- **Đối tượng chung (General Object / Category-level):** Nhận diện một thực thể thuộc về một lớp khái niệm chung mà không phân biệt danh tính cá thể cụ thể (ví dụ: xác định một vùng ảnh chứa "nam giới", "xe ô tô" hoặc "con mèo").
- **Đối tượng cụ thể (Specific Object / Instance-level):** Định danh chính xác một thực thể duy nhất trong cơ sở dữ liệu mẫu (ví dụ: nhận diện một khuôn mặt cụ thể là "bạn An", chiếc ô tô mang biển số riêng "30F-999.99" hoặc tòa nhà cụ thể "Landmark 81").

1.9 Bản chất và thách thức của khoảng cách ngữ nghĩa (Semantic Gap)

Khoảng cách ngữ nghĩa (Semantic Gap) là sự chênh lệch căn bản giữa **cách biểu diễn dữ liệu mức thấp** của máy tính và **cách hiểu ngữ nghĩa mức cao** của con người.

Dưới góc nhìn của máy tính, hình ảnh số chỉ là một mảng dữ liệu đa chiều chứa các giá trị cường độ điểm ảnh số thô. Khi điều kiện chiếu sáng, góc chụp, khoảng cách hoặc tư thế vật thể thay đổi, toàn bộ các giá trị số trong ma trận điểm ảnh bị biến đổi hoàn toàn dù thực thể vật lý trong ảnh không thay đổi.

Ngược lại, hệ thống thị giác con người tự động trích xuất các cấu trúc bất biến, kết hợp với tri thức nền tảng về thế giới để nhận thức ngay lập tức danh tính thực thể, ngữ cảnh không gian, cảm xúc và các mối quan hệ logic phức tạp. Việc thu hẹp khoảng cách giữa tín hiệu điểm ảnh số thô và khái niệm ngữ nghĩa trừu tượng chính là bài toán trung tâm của thị giác máy tính.

1.10 Tiến trình phát triển trong việc thu hẹp khoảng cách ngữ nghĩa

Quá trình giải quyết khoảng cách ngữ nghĩa trong thị giác máy tính đã trải qua ba giai đoạn phát triển chính:

Giai đoạn trích xuất đặc trưng thủ công (Trước năm 2012): Dựa trên các thuật toán hình học và xử lý tín hiệu xác định như SIFT (Lowe, 2004), SURF (Bay et al., 2008), HOG (Dalal & Triggs, 2005) kết hợp với các bộ phân loại nông như Support Vector Machines (SVM) (Cortes & Vapnik, 1995) hoặc Random Forest (Breiman, 2001). Phương pháp này chỉ mô tả được các mẫu cục bộ đơn giản và phụ thuộc nặng nề vào kỹ nghệ thiết kế đặc trưng bằng tay, khiến khoảng cách ngữ nghĩa vẫn còn rất lớn.

Kỷ nguyên Học sâu phân cấp (Giai đoạn 2012–2020): Sự ra đời của các mạng nơ-ron tích chập sâu như AlexNet (Krizhevsky et al., 2012), VGG (Simonyan & Zisserman, 2015) và ResNet (He et al., 2016) đã mở ra cơ chế tự động học biểu diễn đặc trưng phân cấp trực tiếp từ dữ liệu thô. Các tầng nông học các đặc trưng biên nét cục bộ, trong khi các tầng sâu trừu tượng hóa thành các bộ phận và ngữ nghĩa hoàn chỉnh của đối tượng, giúp thu hẹp đáng kể khoảng cách ngữ nghĩa trong các bài toán đóng.

Kỷ nguyên Mô hình Nền tảng và Thị giác - Ngôn ngữ (Từ 2020 đến nay): Kiến trúc Vision Transformer (ViT) (Dosovitskiy et al., 2020) kết hợp với các mô hình học biểu diễn liên kết đa phương thức như CLIP (Radford et al., 2021), BLIP (Li et al., 2022), LLaVA (Liu et al., 2023) và GPT-4V (OpenAI, 2023) đã ánh xạ trực tiếp không gian ảnh và không gian ngôn ngữ tự nhiên vào cùng một không gian nhúng chung. Nhờ đó, mô hình có khả năng suy luận ngữ cảnh mở, sinh mô tả tự nhiên (Image Captioning) và trả lời câu hỏi thị giác (Visual Question Answering) một cách linh hoạt (Dai et al., 2023).

Mặc dù đạt được những tiến bộ vượt bậc, các mô hình học sâu và mô hình nền tảng hiện nay vẫn bộc lộ điểm yếu khi đối mặt với nhiễu đối kháng (adversarial perturbations) (Li et al., 2024), dễ sinh ảo giác (hallucination) trong điều kiện thiếu sáng hoặc bị che khuất nghiêm trọng (Hendrycks & Dietterich, 2019; Li et al., 2023; Guan et al., 2024), đồng thời còn hạn chế đáng kể trong khả năng suy luận quan hệ nhân quả (causal reasoning) (Zang et al., 2023).

1.11 Động lực mở rộng từ dữ liệu hình ảnh sang dữ liệu đa phương thức

Việc giới hạn mô hình trong dữ liệu hình ảnh hoặc video đơn lẻ bộc lộ nhiều điểm nghẽn nhận thức. Về mặt hình học, hình ảnh 2D là phép chiếu thu nhỏ từ thế giới 3D, làm suy giảm thông tin chiều

sâu và gây ra tính nhập nhằng về kích thước thực tế của vật thể. Hơn nữa, nhiều hiện tượng thực tế đòi hỏi sự kết hợp của các tín hiệu bổ trợ như âm thanh (tiếng va chạm cơ học, tiếng rò rỉ khí), văn bản ngữ cảnh (biển báo giao thông, tài liệu hướng dẫn) hay dữ liệu đo lường từ cảm biến gia tốc và nhiệt độ.

Dữ liệu đa phương thức mang lại cơ chế bổ trợ ngữ nghĩa hai chiều: ngôn ngữ tự nhiên cung cấp cấu trúc ngữ nghĩa trừu tượng và tri thức logic, trong khi dữ liệu thị giác cung cấp tọa độ hình học và chi tiết không gian cụ thể. Sự kết hợp này là điều kiện tiên quyết để xây dựng các hệ thống tự hành đáng tin cậy như xe tự lái (kết hợp đồng thời Camera RGB, Radar, LiDAR và bản đồ số HD) hay robot tương tác thông minh.

1.12 Phân định ranh giới giữa Đồ họa máy tính, Xử lý ảnh và Thị giác máy tính

Ba lĩnh vực Đồ họa máy tính (Computer Graphics - CG), Xử lý ảnh (Image Processing - IP) và Thị giác máy tính (Computer Vision - CV) cùng thao tác trên dữ liệu thị giác nhưng khác biệt căn bản về mục tiêu, phạm vi bài toán, phương pháp tiếp cận và bản chất chuyển đổi dữ liệu. Bảng 1 so sánh toàn diện giữa ba lĩnh vực trên 5 tiêu chí chuẩn hóa: Mục tiêu (Goal), Phạm vi (Scope), Phương pháp tiếp cận (Approach), Dữ liệu đầu vào (Input) và Dữ liệu đầu ra (Output).

Bảng 1: So sánh toàn diện giữa Computer Graphics, Image Processing và Computer Vision theo 5 tiêu chí chuẩn hóa

Tiêu chí	**Computer Graphics (CG)**	**Image Processing (IP)**	**Computer Vision (CV)**
1. Mục tiêu (Goal)	Mô phỏng thế giới thực	Nâng cao chất lượng, khôi phục ảnh suy thoái, lọc nhiễu, nén hoặc biến đổi biểu diễn dữ liệu ảnh.	Hiểu, phân tích, trích xuất cấu trúc ngữ nghĩa, tri thức và suy luận ra quyết định từ dữ liệu thị giác.
2. Phạm vi (Scope)	Không gian mô hình hình học 2D/3D trừu tượng, thực hiện chiếu phối cảnh, đổ bóng và mô phỏng vật liệu quang học.	Không gian tín hiệu pixel và tần số mức thấp, thao tác không can thiệp hay thay đổi ngữ nghĩa mức cao của đối tượng.	Không gian ngữ nghĩa mức cao, ánh xạ từ tín hiệu pixel sang biểu diễn nhận thức và quan hệ logic.
3. Phương pháp (Approach)	Mô hình hóa toán học (Polygon Mesh, Spline, NeRF, 3D Gaussian Splatting), đường ống đồ họa (Graphics Pipeline, Shaders, Ray Tracing).	Xử lý tín hiệu số 2D (lọc không gian, biến đổi Fourier/Wavelet, cân bằng lược đồ mức xám, toán tử hình thái học).	Học máy và học sâu (CNNs, Vision Transformers, Foundation Models) kết hợp hình học thị giác đa góc nhìn.
4. Đầu vào (Input)	Mô hình hình học 2D/3D (tập điểm, mesh, vertices), thuộc tính vật liệu, vị trí camera và thông số nguồn sáng.	Dữ liệu ảnh số thô, ảnh bị suy thoái do nhiễu vật lý, mờ chuyển động, thiếu sáng hoặc nén suy hao.	Hình ảnh số 2D $I \in \mathbb{R}^{H\times W\times C}$, chuỗi video $\{I_t\}$, ảnh đa phổ, đám mây điểm 3D từ môi trường thực tế.
5. Đầu ra (Output)	Hình ảnh hoặc chuỗi video được kết xuất trực quan chân thực lên màn hình hiển thị.	Hình ảnh đã qua xử lý, tăng cường hoặc khôi phục trong cùng không gian điểm ảnh.	Dữ liệu ngữ nghĩa có cấu trúc (nhãn phân loại, hộp bao, mặt nạ phân đoạn, pose 3D, quyết định hành động).
Bản chất chuyển đổi	**Dữ liệu trừu tượng → Hình ảnh**	**Hình ảnh → Hình ảnh**	**Hình ảnh → Dữ liệu ngữ nghĩa**

1.13 Khái quát các bài toán cốt lõi trong Thị giác máy tính

Các hệ thống thị giác máy tính hiện đại được cấu thành từ 7 bài toán nền tảng. Nhằm chuẩn hóa việc đánh giá và xây dựng mô hình, mỗi bài toán được phân tích có hệ thống theo 5 tiêu chí: **Mục tiêu**

(Goal), **Phạm vi (Scope)**, **Phương pháp tiếp cận (Approach)**, **Dữ liệu đầu vào (Input)** và **Dữ liệu đầu ra (Output)**.

1. Classification (Phân loại ảnh):

- **Mục tiêu (Goal):** Xác định và gán nhãn ngữ nghĩa ở mức toàn cảnh (Image-level) cho bức ảnh từ một tập danh mục lớp định trước.
- **Phạm vi (Scope):** Cấp độ toàn ảnh, tập danh mục đóng cố định ($|\mathcal{C}|$ lớp), không định vị tọa độ hay phân tách từng đối tượng riêng lẻ.
- **Phương pháp tiếp cận (Approach):** Mạng trích xuất đặc trưng sâu (CNN / ViT) kết hợp lớp Fully-Connected + Softmax, tối ưu hóa bằng hàm mất mát Cross-Entropy Loss hoặc Focal Loss.
- **Đầu vào (Input):** Hình ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$ (với chiều cao H, chiều rộng W và C kênh màu).
- **Đầu ra (Output):** Nhãn danh mục $c \in \mathcal{C}$ hoặc vector phân phối xác suất dự đoán $\mathbf{p} = P(Y \mid I) \in \Delta^{|\mathcal{C}|-1}$.

2. Detection (Phát hiện đối tượng):

- **Mục tiêu (Goal):** Đồng thời xác định vị trí không gian (Localization) và phân loại ngữ nghĩa (Classification) cho tất cả các thực thể quan tâm xuất hiện trong ảnh.
- **Phạm vi (Scope):** Cấp độ đối tượng cụ thể (Instance-level), phát hiện đa đối tượng và đa lớp trong cùng một khung ảnh với kích thước và tỷ lệ khung hình biến thiên.
- **Phương pháp tiếp cận (Approach):** Kiến trúc Two-stage như Faster R-CNN (Ren et al., 2015), One-stage như YOLO (Redmon et al., 2016), RetinaNet (Lin et al., 2017), SSD (Liu et al., 2016) hoặc kiến trúc dựa trên Transformer như DETR (Carion et al., 2020); tối ưu đồng thời hàm mất mát phân loại (Cross-Entropy/Focal Loss) và hồi quy tọa độ hộp bao (GIoU (Rezatofighi et al., 2019), DIoU Loss (Zheng et al., 2020)) kết hợp thuật toán lọc Non-Maximum Suppression (NMS).
- **Đầu vào (Input):** Hình ảnh tĩnh hoặc từng khung hình video $I \in \mathbb{R}^{H \times W \times C}$.
- **Đầu ra (Output):** Tập hợp các hộp bao đối tượng $\mathcal{B} = \{(x_i, y_i, w_i, h_i, c_i, s_i)\}_{i=1}^{N}$, trong đó (x_i, y_i, w_i, h_i) là tọa độ tâm và kích thước khung chữ nhật bao bọc (Bounding Box), $c_i \in \mathcal{C}$ là nhãn lớp và $s_i \in [0, 1]$ là điểm số tin cậy.

3. Recognition (Nhận dạng):

- **Mục tiêu (Goal):** Định danh cụ thể một cá thể riêng biệt (Instance-level identity) trong một tập danh tính hoặc cơ sở dữ liệu mẫu đã biết (Closed-set) sau khi vùng đối tượng đã được cô lập.
- **Phạm vi (Scope):** Cấp độ cá thể riêng biệt (Fine-grained), vùng ảnh đối tượng đã được crop hoặc căn chỉnh (ROI), không gian danh tính hữu hạn định trước ($\mathcal{Y}_{\text{known}}$).
- **Phương pháp tiếp cận (Approach):** Trích xuất đặc trưng biểu diễn mức sâu kết hợp Metric Learning với góc margin lớn (CosFace, ArcFace) trên tập nhãn cố định, thực hiện so khớp 1:N với cơ sở dữ liệu mẫu (Gallery).
- **Đầu vào (Input):** Vùng ảnh chứa đối tượng đã cô lập $I_{\text{crop}} \in \mathbb{R}^{h \times w \times C}$ và cơ sở dữ liệu mẫu đã đăng ký $\mathcal{D}_{\text{gallery}}$.
- **Đầu ra (Output):** Mã định danh cá thể cụ thể (Identity ID) $y \in \mathcal{Y}_{\text{known}}$ (hoặc nhãn Unknown) kèm điểm số so khớp tương đồng.

4. Tracking (Theo dõi đối tượng):

- **Mục tiêu (Goal):** Duy trì mã định danh duy nhất (Track ID) và ước lượng quỹ đạo chuyển động không gian theo thời gian của từng đối tượng qua chuỗi khung hình video liên tiếp. Bản chất của bài toán theo dõi là sự kết hợp liên hoàn giữa Phát hiện đối tượng và So khớp liên kết dữ liệu theo thời gian (Tracking = Detection + Matching).

- **Phạm vi (Scope):** Miền không - thời gian (Spatio-temporal), liên tục trên chuỗi video thời gian thực, phạm vi tìm kiếm cục bộ quanh vị trí khung hình liền kề ($t \to t+1$).
- **Phương pháp tiếp cận (Approach):** Mô hình Tracking-by-Detection (TBD) hoặc Joint Detection & Embedding (JDE), kết hợp mô hình chuyển động động học (Kalman Filter, Optical Flow) với giải thuật tối ưu liên kết dữ liệu như thuật toán Hungarian (Kuhn, 1955) hay ByteTrack (Zhang et al., 2022) dựa trên chỉ số IoU và độ tương đồng Cosine đặc trưng.
- **Đầu vào (Input):** Chuỗi khung hình video liên tục $\{I_t\}_{t=1}^{T}$ với $I_t \in \mathbb{R}^{H \times W \times C}$.
- **Đầu ra (Output):** Tập các vết quỹ đạo không gian - thời gian $\mathcal{T} = \{(\text{ID}_i, t, x_{i,t}, y_{i,t}, w_{i,t}, h_{i,t})\}$, đảm bảo tính nhất quán định danh của từng cá thể qua trục thời gian t.

5. Retrieval (Truy vấn hình ảnh):

- **Mục tiêu (Goal):** Tìm kiếm và xếp hạng các hình ảnh có độ tương đồng nội dung cao nhất với ảnh truy vấn từ một kho cơ sở dữ liệu lớn ở môi trường mở (Open-set).
- **Phạm vi (Scope):** Cấp độ toàn cục trên kho dữ liệu quy mô lớn $\mathcal{D}$, không gian mở đối với các thực thể chưa từng thấy khi huấn luyện, các mẫu dữ liệu ảnh tĩnh độc lập phi thời gian.
- **Phương pháp tiếp cận (Approach):** Học không gian nhúng biểu diễn sâu (Deep Metric Learning / Contrastive Learning) với Triplet Loss hoặc InfoNCE Loss; ánh xạ ảnh thành vector nhúng $\mathbf{v} = f_\theta(I) \in \mathbb{R}^d$, lập chỉ mục vector và tìm kiếm lân cận gần nhất xấp xỉ (Approximate Nearest Neighbor - ANN như FAISS, HNSW).
- **Đầu vào (Input):** Ảnh truy vấn (Query Image) $I_q \in \mathbb{R}^{H \times W \times C}$ và kho cơ sở dữ liệu ảnh mẫu $\mathcal{D} = \{I_1, I_2, \ldots, I_M\}$.
- **Đầu ra (Output):** Danh sách xếp hạng thứ tự top-K ảnh $\{I_{\pi(1)}, I_{\pi(2)}, \ldots, I_{\pi(K)}\}$ có khoảng cách biểu diễn nhỏ nhất trong không gian nhúng kèm điểm số tương đồng Cosine.

6. Counting (Đếm đối tượng):

- **Mục tiêu (Goal):** Ước lượng tổng số lượng thực thể xuất hiện trong khung cảnh, đặc biệt hiệu quả trong các tình huống có mật độ cực cao hoặc che khuất nghiêm trọng.
- **Phạm vi (Scope):** Cấp độ khung cảnh mật độ cao (đám đông, vi sinh vật, đàn gia súc), chấp nhận hiện tượng che khuất chồng chéo sâu (Severe Occlusion).
- **Phương pháp tiếp cận (Approach):** Hồi quy trường mật độ liên tục (Density Map Estimation) qua mạng nơ-ron tích chập hoàn toàn (FCNs) hoặc Transformer như CSRNet (Li et al., 2018) hay DM-Count (Wang et al., 2020) với hàm mất mát MSE hoặc Bayesian Loss dựa trên nhãn điểm tâm (Dot annotation) được lọc Gaussian; tránh hoàn toàn bước phân tách hộp bao rời rạc và lọc NMS.
- **Đầu vào (Input):** Hình ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$ (kèm nhãn tọa độ điểm chấm (x_p, y_p) trong pha huấn luyện).
- **Đầu ra (Output):** Bản đồ mật độ liên tục $D \in \mathbb{R}^{H \times W}$, trong đó tổng tích phân $\hat{N} = \sum_{x,y} D(x,y)$ xấp xỉ tổng số đối tượng (hoặc giá trị số thực ước tính $\hat{N} \in \mathbb{R}^+$).

7. Generating (Sinh dữ liệu thị giác):

- **Mục tiêu (Goal):** Khởi tạo hình ảnh hoặc chuỗi video mới hoàn chỉnh, mô phỏng đúng phân phối dữ liệu thị giác thực tế $P(X)$ hoặc sinh có điều kiện theo yêu cầu người dùng.
- **Phạm vi (Scope):** Toàn bộ không gian phân phối xác suất dữ liệu cao chiều, từ sinh vô điều kiện đến sinh có điều kiện đa phương thức (Text-to-Image, Image-to-Image, Inpainting, ControlNet).
- **Phương pháp tiếp cận (Approach):** Mô hình xác suất tạo sinh sâu (GANs với Minimax Adversarial Loss, VAEs với ELBO, Diffusion Models / Flow Matching với Denoising Score Matching và U-Net/DiT).

- **Đầu vào (Input):** Vector nhiễu ngẫu nhiên $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ trong không gian tiềm ẩn, kèm theo điều kiện dẫn đường tùy chọn (mô tả văn bản $\mathbf{c}_{\text{text}}$, ảnh điều kiện I_{cond}, bản đồ bố cục Canny/Pose).
- **Đầu ra (Output):** Hình ảnh hoặc chuỗi video tổng hợp mới $\hat{I} \in \mathbb{R}^{H \times W \times C}$ có độ chân thực cao và thỏa mãn các ràng buộc ngữ nghĩa đầu vào.

Bảng 2 tóm tắt ma trận so sánh 7 bài toán cốt lõi của Thị giác máy tính theo đúng 5 tiêu chí chuẩn hóa.

Bảng 2: Ma trận tổng hợp so sánh 7 bài toán cốt lõi trong Thị giác máy tính theo 5 tiêu chí

Bài toán	**Mục tiêu (Goal)**	**Phạm vi (Scope)**	**Phương pháp (Approach)**	**Đầu vào (Input)**	**Đầu ra (Output)**
Classification	Gán nhãn ngữ nghĩa toàn cảnh	Toàn ảnh, tập đóng $\mathcal{C}$ lớp	CNN / ViT + Softmax (Cross-Entropy Loss)	Ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$	Nhãn $c \in \mathcal{C}$, phân phối $\mathbf{p}$
Detection	Định vị & phân loại đối tượng	Đối tượng (Instance), đa lớp	Two/One-stage, DETR, Box Loss + NMS	Ảnh tĩnh / Khung video I	Hộp bao $\mathcal{B} = \{(x, y, w, h, c, s)\}$
Recognition	Định danh cá thể cụ thể	Cá thể (Instance), tập đóng	Deep Embedding + ArcFace / Softmax	Ảnh crop $I_{\text{crop}} + \mathcal{D}_{\text{gallery}}$	Identity ID $y \in \mathcal{Y}_{\text{known}}$
Tracking	Duy trì ID & quỹ đạo theo thời gian	Không - thời gian, video liên tục	Tracking-by-Detection + Kalman + ByteTrack	Chuỗi video $\{I_t\}_{t=1}^{T}$	Quỹ đạo $\mathcal{T} = \{(\text{ID}_i, t, \mathbf{b}_{i,t})\}$
Retrieval	Tìm ảnh tương đồng ngữ nghĩa	Toàn cục kho ảnh, tập mở (Open-set)	Metric Learning (InfoNCE, Triplet) + ANN	Ảnh truy vấn $I_q + \mathcal{D}$	Ranked List $\{I_{\pi(k)}\}$
Counting	Ước lượng số lượng thực thể	Mật độ cao, chịu che khuất nặng	Hồi quy bản đồ mật độ (CSRNet) + Density Loss	Ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$	Bản đồ mật độ $D \to \hat{N} = \sum D$
Generating	Khởi tạo nội dung thị giác mới	Không gian phân phối dữ liệu $P(X)$	Diffusion Models, GANs, VAEs, Flow Matching	Nhiễu $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}) +$ Condition	Ảnh/video mới $\hat{I}$

1.14 So sánh bài toán Nhận dạng (Recognition) và Truy vấn ảnh (Image Retrieval)

Mặc dù đều phục vụ mục đích xác định sự tương đồng của đối tượng thị giác, Nhận dạng (Recognition) và Truy vấn ảnh (Image Retrieval) khác biệt căn bản về mục tiêu, không gian bài toán, cơ chế tối ưu hóa hàm mất mát, dữ liệu đầu vào và dạng thức đầu ra (Bảng 3).

1.15 So sánh bài toán Truy vấn ảnh (Retrieval) và Theo dõi đối tượng (Tracking)

Sự khác biệt cốt lõi giữa Truy vấn ảnh (Retrieval) và Theo dõi đối tượng (Tracking) bắt nguồn từ yếu tố phụ thuộc thời gian, miền không gian tìm kiếm, phương pháp kết hợp mô hình động học và dạng thức đầu ra (Bảng 4):

1.16 So sánh bài toán Đếm đối tượng (Counting) và Phát hiện đối tượng (Object Detection)

Trong các khung cảnh thông thường với mật độ thưa, bài toán đếm có thể giải quyết gián tiếp thông qua việc phát hiện từng đối tượng riêng lẻ. Tuy nhiên, khi mật độ tăng cao và xuất hiện hiện tượng che khuất nghiêm trọng, hai phương pháp này thể hiện sự khác biệt rõ rệt trên cả 5 tiêu chí (Bảng 5):

1.17 Mô hình Tạo sinh (Generative) và tương quan với các tác vụ Phân tích (Discriminative)

Các bài toán trong học máy và thị giác máy tính được phân chia thành hai trường phái nền tảng dựa trên mục tiêu mô hình hóa phân phối xác suất, phạm vi biến đổi không gian dữ liệu và cách thức tối ưu hóa (Bảng 6):

Bảng 3: So sánh toàn diện giữa bài toán Recognition và Image Retrieval theo 5 tiêu chí

Tiêu chí	Nhận dạng (Recognition)	Truy vấn hình ảnh (Image Retrieval)
1. Mục tiêu (Goal)	Định danh chính xác một cá thể cụ thể (Instance Identity) thuộc về một danh tính đã đăng ký trước trong hệ thống.	Tìm kiếm và xếp hạng các hình ảnh có độ tương đồng ngữ nghĩa/nội dung cao nhất với ảnh truy vấn từ một kho dữ liệu lớn.
2. Phạm vi (Scope)	Tập đóng (Closed-set): số lượng lớp và danh tính được cố định trước trong quá trình huấn luyện; đối tượng thường là vùng ảnh đã được cô lập (Cropped/Aligned ROI).	Tập mở (Open-set): tìm kiếm các thực thể và mẫu chưa từng xuất hiện trong tập huấn luyện; phạm vi tìm kiếm toàn cục trên cơ sở dữ liệu quy mô lớn $\mathcal{D}$.
3. Phương pháp (Approach)	Học các siêu phẳng phân tách ranh giới giữa các lớp xác định bằng Softmax hoặc Margin Loss (ArcFace, CosFace); suy luận trực tiếp bằng $\arg\max$ xác suất hoặc so khớp ngưỡng với Gallery mẫu cố định.	Học không gian nhúng biểu diễn (Deep Metric Learning / Contrastive Learning) với Triplet Loss, InfoNCE Loss để tối ưu khoảng cách biểu diễn; suy luận bằng trích xuất embedding và thuật toán tìm kiếm lân cận gần nhất (k-NN, ANN, FAISS).
4. Đầu vào (Input)	Vùng ảnh chứa đối tượng đã cô lập $I_{\text{crop}} \in \mathbb{R}^{h \times w \times C}$ và cơ sở dữ liệu mẫu đã đăng ký $\mathcal{D}_{\text{gallery}}$.	Ảnh truy vấn (Query Image) $I_q \in \mathbb{R}^{H \times W \times C}$ và kho cơ sở dữ liệu ảnh mẫu quy mô lớn $\mathcal{D} = \{I_1, I_2, \ldots, I_M\}$.
5. Đầu ra (Output)	Mã định danh cá thể cụ thể (Identity ID) $y \in \mathcal{Y}_{\text{known}}$ (hoặc nhãn Unknown) kèm xác suất hoặc điểm số tin cậy.	Danh sách có thứ tự xếp hạng (Ranked List) top-K hình ảnh tương đồng nhất $\{I_{\pi(1)}, \ldots, I_{\pi(K)}\}$ kèm điểm số khoảng cách.

Bảng 4: So sánh toàn diện giữa bài toán Image Retrieval và Object Tracking theo 5 tiêu chí

Tiêu chí	Truy vấn hình ảnh (Retrieval)	Theo dõi đối tượng (Tracking)
1. Mục tiêu (Goal)	Tìm kiếm và xếp hạng các hình ảnh tương đồng nhất với ảnh mẫu trong kho dữ liệu tĩnh.	Duy trì định danh duy nhất (Track ID) và ước lượng quỹ đạo chuyển động không gian theo thời gian của đối tượng qua chuỗi video.
2. Phạm vi (Scope)	Không gian toàn cục tĩnh: các mẫu dữ liệu độc lập, phi thời gian, không có liên kết vật lý thời gian.	Không - thời gian cục bộ: chuỗi video thời gian thực liên tục; tìm kiếm trong lân cận không gian dự đoán từ khung hình liền kề ($t \to t+1$).
3. Phương pháp (Approach)	Sử dụng thuần túy mô hình đặc trưng ngoại hình tĩnh kết hợp cấu trúc dữ liệu chỉ mục vector như IVF-PQ (Jégou et al., 2011) hay HNSW.	Kết hợp mô hình ngoại hình và mô hình chuyển động động học (Kalman Filter, Optical Flow) cùng giải thuật tối ưu phân phối liên kết dữ liệu (Hungarian Matching (Kuhn, 1955), ByteTrack (Zhang et al., 2022)).
4. Đầu vào (Input)	Ảnh truy vấn I_q và kho cơ sở dữ liệu ảnh mẫu độc lập $\mathcal{D} = \{I_1, \ldots, I_M\}$.	Chuỗi khung hình video liên tục $\{I_t\}_{t=1}^{T}$ có tính tương quan thời gian cao theo tốc độ khung hình ($I_t \in \mathbb{R}^{H \times W \times C}$).
5. Đầu ra (Output)	Danh sách các hình ảnh xếp hạng theo độ tương đồng $\{I_{\pi(1)}, \ldots, I_{\pi(K)}\}$.	Tập hợp các vết quỹ đạo không gian - thời gian $\mathcal{T} = \{(\text{Track ID}_i, t, x_{i,t}, y_{i,t}, w_{i,t}, h_{i,t})\}$.

Bảng 5: So sánh toàn diện giữa bài toán Object Detection và Crowd Counting theo 5 tiêu chí

Tiêu chí	Phát hiện đối tượng (Detection)	Đếm đối tượng (Counting)
1. Mục tiêu (Goal)	Đồng thời định vị ranh giới không gian (Bounding Box) và phân loại ngữ nghĩa cho từng cá thể độc lập.	Ước lượng tổng số lượng thực thể xuất hiện trong khung cảnh, tập trung vào mật độ tổng thể thay vì ranh giới từng cá thể.
2. Phạm vi (Scope)	Thích hợp cho khung cảnh thưa đến trung bình, đối tượng có ranh giới rõ ràng; suy giảm độ chính xác nghiêm trọng khi mật độ cực cao và che khuất chồng chéo sâu.	Chuyên biệt hóa cho khung cảnh mật độ từ trung bình đến cực dày đặc (đám đông, vi sinh vật, xe cộ), bền vững trước hiện tượng che khuất nghiêm trọng.
3. Phương pháp (Approach)	Hồi quy hộp bao rời rạc và phân loại nhãn (Two-stage/One-stage/DETR) với Cross-Entropy + GIoU Loss, lọc Non-Maximum Suppression (NMS); yêu cầu nhãn hộp bao chi tiết (x, y, w, h).	Hồi quy trường mật độ liên tục (Density Map Estimation) qua FCN/Transformer với hàm mất mát MSE hoặc Bayesian Loss; chỉ yêu cầu nhãn điểm tâm (Dot annotation) Gaussian hóa; không dùng NMS.
4. Đầu vào (Input)	Hình ảnh tĩnh hoặc khung hình video $I \in \mathbb{R}^{H\times W\times C}$ (kèm nhãn hộp bao chi tiết trong pha huấn luyện).	Hình ảnh tĩnh hoặc chuỗi khung hình $I \in \mathbb{R}^{H\times W\times C}$ (kèm nhãn tọa độ điểm chấm (x_p, y_p) trong pha huấn luyện).
5. Đầu ra (Output)	Tập hợp các hộp bao rời rạc $\mathcal{B} = \{(x_i, y_i, w_i, h_i, c_i, s_i)\}_{i=1}^{N} \Longrightarrow \hat{N} = \lvert\mathcal{B}\rvert$.	Bản đồ phân phối mật độ liên tục $D \in \mathbb{R}^{H\times W} \Longrightarrow \hat{N} = \sum_{x,y} D(x, y)$ (hoặc trực tiếp giá trị vô hướng $\hat{N}$).

2 Tuần 2: Trí tuệ Nhân tạo, Học máy & Học sâu trong Khoa học Dữ liệu Thị giác

2.1 Bản chất của Trí tuệ Tự nhiên, Trí tuệ Nhân tạo và Khái niệm Thông minh

Trong khoa học nhận thức và khoa học máy tính, việc phân định ranh giới bản thể luận giữa các dạng thức trí tuệ là tiền đề lý thuyết cốt lõi nhằm định hình phương pháp thiết kế các hệ thống tính toán thông minh.

Trí tuệ Tự nhiên (Natural Intelligence, NI): Đại diện cho năng lực sinh học bẩm sinh của con người và các sinh vật bậc cao, được tôi luyện qua hàng triệu năm tiến hóa chọn lọc tự nhiên. Trí tuệ tự nhiên tích hợp hữu cơ nhiều phương thức nhận thức phức tạp: khả năng thụ cảm đa giác quan, ý thức tự thân (Self-awareness), tư duy trừu tượng hóa, suy luận quan hệ nhân quả (Causal Reasoning), năng lực sáng tạo nghệ thuật và sự thích nghi linh hoạt trước những biến đổi bất định của sinh cảnh. Toàn bộ tiến trình nhận thức này luôn gắn liền với đời sống cảm xúc nội tâm, các quy chuẩn đạo đức và bối cảnh văn hóa xã hội.

Trí tuệ Nhân tạo (Artificial Intelligence, AI): Là một phân ngành mũi nhọn của khoa học máy tính, tập trung nghiên cứu các nền tảng toán học, cấu trúc dữ liệu và giải thuật nhằm kiến tạo các hệ thống tính toán có khả năng mô phỏng các khía cạnh hành vi thông minh của con người. Phạm vi năng lực của trí tuệ nhân tạo bao gồm việc học biểu diễn từ dữ liệu (Learning), suy luận logic hình thức (Reasoning), nhận thức thị giác máy tính (Visual Perception), hiểu ngôn ngữ tự nhiên (NLP) và hoạch định chiến lược ra quyết định (Decision Making).

Nhằm làm sáng tỏ sự khác biệt cơ bản giữa hai hệ thống này, Bảng 7 tổng hợp các tiêu chí đối chiếu bản chất giữa Trí tuệ Tự nhiên và Trí tuệ Nhân tạo.

Về mặt động học vận hành, mọi thực thể thông minh (dù là sinh học hay nhân tạo) đều hiện thực hóa một vòng lặp nhận thức khép kín bao gồm ba giai đoạn kế tiếp nhau:

1. **Giai đoạn 1: Nhận biết (Perception):** Tiếp nhận các kích thích và tín hiệu vật lý từ môi trường bên ngoài. Ở sinh vật, tiến trình này diễn ra thông qua hệ giác quan sinh học với cơ chế chú ý tự nhiên nhằm chắt lọc các kích thích có ý nghĩa sinh tồn. Trong hệ thống thị giác máy tính, nhận thức khởi đầu từ việc các cảm biến quang học, máy ảnh số hoặc

Bảng 6: So sánh toàn diện giữa mô hình Tạo sinh (Generative) và các tác vụ Phân tích (Discriminative) theo 5 tiêu chí

Tiêu chí	**Tác vụ Phân tích (Discriminative Tasks)**	**Tác vụ Tạo sinh (Generative Tasks)**
1. Mục tiêu (Goal)	Trích xuất thông tin ngữ nghĩa, định vị, phân loại hoặc đưa ra quyết định từ dữ liệu thị giác đã có (nhận thức thế giới quan).	Học phân phối dữ liệu thị giác thực tế $P(X)$ để tổng hợp, khởi tạo hoặc tái tạo nội dung hình ảnh/video mới chân thực (kiến tạo nội dung).
2. Phạm vi (Scope)	Không gian phân loại hoặc hồi quy có giám sát; ánh xạ thu hẹp từ không gian ảnh cao chiều về không gian nhãn/tọa độ/vector tóm lược ($f : \mathcal{X} \to \mathcal{Y}$).	Toàn bộ không gian phân phối dữ liệu thị giác cao chiều $P(X)$ hoặc $P(X \mid Y)$; bao gồm sinh vô điều kiện, có điều kiện (Text-to-Image, ControlNet) và mô hình hóa không gian tiềm ẩn liên tục.
3. Phương pháp (Approach)	Học xác suất hậu nghiệm $P(Y \mid X)$ (ResNet (He et al., 2016), ViT (Dosovitskiy et al., 2020), YOLO (Redmon et al., 2016), DETR (Carion et al., 2020)); tối ưu hóa ranh giới phân tách lớp bằng Cross-Entropy, Focal Loss (Lin et al., 2017), GIoU Loss (Rezatofighi et al., 2019).	Học phân phối thực $P(X)$ hoặc đồng thời $P(X, Y)$ (GANs (Goodfellow et al., 2014), VAEs (Kingma & Welling, 2014), Diffusion Models (Ho et al., 2020), Flow Matching (Lipman et al., 2023), DiT (Peebles & Xie, 2023)); tối ưu hóa phân phối bằng Minimax Adversarial Loss, ELBO, Denoising Score Matching MSE.
4. Đầu vào (Input)	Hình ảnh hoặc video thô $I \in \mathbb{R}^{H \times W \times C}$ thu nhận từ môi trường thực tế.	Vector nhiễu ngẫu nhiên $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ từ không gian tiềm ẩn kèm tín hiệu dẫn đường điều kiện (văn bản $\mathbf{c}_{\text{text}}$, ảnh bố cục, pose, cạnh nét).
5. Đầu ra (Output)	Dữ liệu ngữ nghĩa có cấu trúc (nhãn phân loại, hộp bao, mặt nạ phân đoạn, vector nhúng, số đếm, vết quỹ đạo).	Hình ảnh hoặc chuỗi video tổng hợp mới hoàn chỉnh $\hat{I} \in \mathbb{R}^{H \times W \times C}$ có độ phân giải cao và chân thực.

cảm biến LiDAR thu nhận sóng ánh sáng và chuyển hóa thành mảng ma trận điểm ảnh thô $I \in \mathbb{R}^{H \times W \times C}$.

2. **Giai đoạn 2: Hiểu (Understanding / Recognition):** Chuyển hóa dữ liệu cảm giác thô thành biểu diễn ngữ nghĩa và cấu trúc tri thức. Con người hiểu sâu sắc về công năng, bối cảnh vật lý và mối liên hệ nhân quả của các thực thể trong tầm nhìn. Đối với hệ thống nhân tạo, mô hình thực hiện trích xuất đặc trưng phân cấp, ánh xạ không gian tiềm ẩn để ước lượng phân phối xác suất lớp $P(Y \mid X)$, định vị không gian và xây dựng đồ thị quan hệ cảnh (Scene Graph).
3. **Giai đoạn 3: Thích nghi và Hành động (Action / Adaptation):** Hoạch định kế hoạch và ban hành quyết định hành vi nhằm tương tác với môi trường hoặc cực đại hóa mục tiêu tối ưu. Con người hành động dựa trên ý chí, trực giác và khả năng ứng biến linh hoạt. Hệ thống AI tương tác thông qua các cơ cấu chấp hành (như góc lái của xe tự hành, tín hiệu điều khiển rơ-le an ninh) hoặc đề xuất giải pháp nghiệp vụ, đồng thời thích ứng bằng cách cập nhật tập tham số trọng số θ qua quá trình học máy.

Từ tiến trình nhận thức ba bước trên, bản chất của khái niệm **Thông minh (Intelligence)** được xác lập dựa trên hai trụ cột cốt lõi: năng lực nhận biết, mô hình hóa nhanh chóng các quy luật tiềm ẩn từ luồng dữ liệu nhiễu loạn cao chiều; và năng lực thích ứng linh hoạt (Adaptability) nhằm điều chỉnh hành vi, cập nhật trọng số tri thức để duy trì hiệu năng tối ưu khi môi trường hoặc phân phối dữ liệu đầu vào có sự biến đổi.

2.2 Tác động của Trí tuệ Nhân tạo đối với sự phát triển của Trí tuệ Tự nhiên

Mối quan hệ tương tác giữa Trí tuệ Nhân tạo và Trí tuệ Tự nhiên mang tính chất biện chứng hai chiều sâu sắc, bao gồm cả những cơ hội khuếch đại nhận thức lẫn các thách thức suy giảm tư duy tiềm ẩn:

Bảng 7: Đối chiếu nền tảng giữa Trí tuệ Tự nhiên (NI) và Trí tuệ Nhân tạo (AI)

Tiêu chí đối chiếu	Trí tuệ Tự nhiên (NI)	Trí tuệ Nhân tạo (AI)
1. Cơ chất vật lý	Mạng nơ-ron sinh học hữu cơ với khoảng 86 tỷ tế bào thần kinh và 10^{14} khớp thần kinh synapse, vận hành qua tín hiệu điện hóa sinh học.	Nền tảng bán dẫn vô cơ trên các vi xử lý chuyên dụng (Silicon, GPU, TPU), thực thi tính toán dựa trên dòng chuyển dịch bit nhị phân.
2. Nguồn gốc & Mục đích	Sản phẩm của tiến hóa sinh học qua chọn lọc tự nhiên; mục đích cốt lõi là duy trì sự sống, thích nghi môi trường và bảo tồn nòi giống.	Do con người thiết kế và lập trình; đóng vai trò là công cụ tự động hóa tính toán nhằm giải quyết các bài toán kinh tế, kỹ thuật và xã hội.
3. Cơ chế học tập	Học hiệu quả từ số ít mẫu (Few-shot Learning), đúc kết tri thức qua trải nghiệm thực chứng và duy trì khả năng học liên tục suốt đời mà không bị xóa bỏ ký ức nền tảng.	Phụ thuộc vào quá trình tối ưu hóa thống kê trên tập dữ liệu quy mô lớn; dễ gặp hiện tượng quên tri thức cũ (Catastrophic Forgetting) khi chuyển sang miền dữ liệu mới.
4. Bản chất tư duy	Tư duy trừu tượng, suy luận quan hệ nhân quả và thấu hiểu bản chất nguồn gốc của sự vật hiện tượng nhằm trả lời câu hỏi vì sao (**"Tại sao"**).	So khớp mẫu quy mô lớn, tính toán tương quan xác suất thống kê $P(Y \mid X)$ và xấp xỉ hàm số phức tạp để trả lời câu hỏi dự đoán biến cố (**"Cái gì"**).
5. Ý thức & Cảm xúc	Sở hữu ý thức tự thân, trải nghiệm cảm xúc thực thể, gắn liền với lòng trắc ẩn và tinh thần trách nhiệm đạo đức trước cộng đồng.	Vận hành dựa trên các phép biến đổi đại số tuyến tính; phản hồi ngôn ngữ hoặc biểu cảm thị giác chỉ là kết quả mô phỏng thống kê trên bề mặt dữ liệu.
6. Hiệu suất năng lượng	Tối ưu hóa chuyển hóa năng lượng vượt bậc: toàn bộ não bộ con người chỉ tiêu thụ công suất sinh học ước tính khoảng 20 W.	Tiêu hao năng lượng lớn: các cụm máy chủ huấn luyện mô hình nền tảng đòi hỏi công suất từ hàng trăm kilowatt đến hàng megawatt điện.
7. Phạm vi năng lực	Trí tuệ tổng quát (General Intelligence): con người có thể tiếp thu đồng thời nhiều kỹ năng dị biệt và chuyển giao tri thức linh hoạt giữa các lĩnh vực.	Trí tuệ chuyên biệt (Narrow AI): thực thi vượt trội trong một tác vụ giới hạn được huấn luyện cụ thể nhưng gặp nhiều rào cản khi tổng quát hóa sang miền nghiệp vụ mới.

Xét về mặt tích cực, AI đóng vai trò như một đòn bẩy khuếch đại nhận thức (Cognitive Augmentation). Bằng cách tự động hóa các tác vụ tính toán lặp lại có tính quy chuẩn cao như thống kê dữ liệu quy mô lớn, rà soát văn bản pháp lý hay phân loại hàng loạt hình ảnh, AI giải phóng nguồn lực nhận thức của con người để tập trung vào các hoạt động sáng tạo chiến lược, tư duy phản biện và hoạch định tầm nhìn. Hơn thế nữa, AI mở rộng ranh giới khám phá khoa học khi giải quyết những bài toán đa chiều vượt ngoài năng lực trực giác sinh học. Minh chứng tiêu biểu là việc hệ thống AlphaFold (Jumper et al., 2021; Abramson et al., 2024) dự đoán chính xác cấu trúc 3D của hơn 200 triệu protein và các phức hợp phân tử sinh học, tạo bước ngoặt đột phá cho y học và sinh học phân tử.

Ngược lại, việc ứng dụng thiếu kiểm soát có thể dẫn đến nguy cơ suy giảm nhận thức do phụ thuộc công nghệ (Cognitive Offloading). Xu hướng ỷ lại vào các thuật toán tự động (Automation Bias) làm suy yếu phản xạ tư duy độc lập, suy giảm khả năng phân tích logic nền tảng và làm thui chột dung lượng trí nhớ làm việc (Working Memory). Bên cạnh đó, các thuật toán đề xuất thông tin cá nhân hóa dễ tạo ra hiện tượng bong bóng thông tin (Echo Chambers), cô lập người dùng trong những góc nhìn định kiến và làm suy giảm khả năng thấu cảm xã hội.

Như vậy, trí tuệ nhân tạo không tất yếu dẫn đến sự phát triển hay thoái hóa của trí tuệ con người. Kết quả thực tiễn hoàn toàn phụ thuộc vào phương thức con người định vị công nghệ: xem AI là công cụ đồng hành khuếch đại năng lực tư duy (Co-pilot) hay một sự thay thế thụ động toàn diện.

2.3 Phân định bản chất giữa Thông minh (Intelligence) và Thông tuệ (Wisdom)

Trong khoa học nhận thức và triết học, "Thông minh" và "Thông tuệ" (hay Trí huệ) đại diện cho hai nấc thang nhận thức hoàn toàn khác biệt về mặt mục đích và giá trị, được phân tích chi tiết trong Bảng 8.

Bảng 8: Đối chiếu toàn diện giữa khái niệm Thông minh (Intelligence) và Thông tuệ (Wisdom)

Tiêu chí	Thông minh (Intelligence)	Thông tuệ / Trí huệ (Wisdom)
Định nghĩa cốt lõi	Năng lực giải quyết bài toán và tối ưu hóa quy trình: thực thi tác vụ một cách chính xác, nhanh chóng và hiệu quả nhất (*Doing things right*).	Năng lực định hướng giá trị và mục đích: lựa chọn đúng mục tiêu có ý nghĩa nhân văn và đạo đức để cống hiến (*Doing the right things*).
Đối tượng xử lý	Dữ liệu số, cấu trúc thông tin, quy tắc logic và các không gian trạng thái toán học.	Ý nghĩa xã hội, chuẩn mực đạo đức, hạnh phúc con người và tính bền vững lâu dài của hệ sinh thái.
Động lực vận hành	Tối ưu hóa hàm mục tiêu kỹ thuật (cực tiểu hóa hàm tổn thất hoặc cực đại hóa phần thưởng kỳ vọng) dựa trên dữ liệu thống kê quá khứ.	Cân bằng hài hòa giữa lợi ích cá nhân và cộng đồng, gắn liền với lòng trắc ẩn (Compassion) và sự thấu cảm chiều sâu.
Khả năng hiện thực	Hệ thống máy tính và mô hình AI có thể đạt tới trình độ vượt bậc (Super-human) trong các tác vụ tính toán chuyên biệt.	Là đặc tính độc bản của nhận thức sinh học và trải nghiệm sống nội tâm; AI hiện đại chưa thể đạt được năng lực thông tuệ tự thân.

2.4 Giải thích Luận điểm: "Thông minh là Khả năng Thích ứng với sự Thay đổi"

Nhà vật lý lý thuyết Stephen Hawking từng đúc kết bản chất của trí tuệ thông qua phát biểu:

> *"Intelligence is the ability to adapt to change."* (*Thông minh là khả năng thích ứng với sự thay đổi*.)

Luận điểm này phản ánh nguyên lý vận hành cốt lõi của tính thông minh khi xét dưới cả hai lăng kính: tâm lý học nhận thức sinh học và khoa học máy tính hiện đại:

Dưới góc độ tâm lý học nhận thức, nhà nghiên cứu Jean Piaget (Piaget, 1950) khẳng định rằng sự phát triển trí tuệ thực chất là tiến trình cân bằng nhận thức động (Equilibration) thông qua hai cơ chế tương hỗ: **Đồng hóa (Assimilation)**, tức quá trình tiếp nhận thông tin mới từ môi trường vào các lược đồ tư duy (Schemas) sẵn có; và **Điều ứng (Accommodation)**, tức việc biến đổi và tái cấu trúc các lược đồ nhận thức nội tại để tương thích với những dữ kiện thực tế mới phát sinh. Sinh vật không thể tồn tại nếu chỉ vận hành bằng các phản xạ cứng nhắc; khả năng điều ứng linh hoạt trước biến động sinh thái chính là biểu hiện cao nhất của trí thông minh.

Dưới góc độ học máy và thị giác máy tính, một mô hình chỉ ghi nhớ máy móc các điểm dữ liệu trong tập huấn luyện (hiện tượng Overfitting) hoàn toàn không có tính thông minh thực sự. Thước đo thông minh của hệ thống AI được xác lập thông qua năng lực thích ứng với các biến số chưa từng xuất hiện:

- *Khả năng tổng quát hóa ngoài phân phối (Out-of-Distribution, OOD):* Mô hình duy trì độ chính xác cao khi phân phối ảnh đầu vào bị biến đổi do nhiễu thời tiết, góc chụp hay độ chiếu sáng bất thường.
- *Thích ứng miền (Domain Adaptation) và Học chuyển giao (Transfer Learning):* Tận dụng tri thức đã học từ miền nguồn (như dữ liệu ảnh mô phỏng 3D) để nhanh chóng thích ứng với miền đích thực tế chỉ với số lượng mẫu gán nhãn tối thiểu.
- *Học tăng cường thích ứng (Reinforcement Learning):* Tác tử tự động điều chỉnh chính sách hành động $\pi(a \mid s)$ khi ma trận chuyển trạng thái của môi trường biến động.
- *Học trong ngữ cảnh (In-Context Learning):* Khả năng của các mô hình thị giác ngôn ngữ lớn trong việc tiếp nhận nhiệm vụ mới thông qua một vài chỉ dẫn (Prompts) hoặc mẫu thị giác minh họa mà không cần tái huấn luyện trọng số mô hình.

2.5 Bản chất của Học máy: Phân tích tư tưởng Arthur Samuel (1959)

Năm 1959, Arthur Samuel đã thiết lập nền móng cho học máy hiện đại qua định nghĩa kinh điển:

> *"Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed."* (*Học máy: Phân ngành nghiên cứu mang lại cho máy tính khả năng tự học mà không cần phải được lập trình tường minh.*)

Tư tưởng mang tính bước ngoặt của Samuel đã phân định rõ ràng hai hệ hình tính toán căn bản trong khoa học máy tính:

Trong hệ hình lập trình truyền thống (Rule-based Systems), lập trình viên phải thấu suốt toàn bộ bản chất quy luật và tự tay viết tường minh từng câu lệnh điều kiện (`if-else`) cùng các công thức giải tích. Mô hình này vận hành theo sơ đồ tuyến tính:

$$\text{Dữ liệu đầu vào (Data)} + \text{Quy tắc lập trình (Rules)} \longrightarrow \text{Kết quả đầu ra (Output)} \tag{1}$$

Phương pháp tiếp cận dựa trên luật gặp phải giới hạn cố hữu khi đối mặt với các bài toán thị giác máy tính phi cấu trúc. Con người không thể lập trình thủ công hàng triệu nhánh điều kiện để bao quát toàn bộ sự biến thiên phức tạp của các ma trận điểm ảnh tự nhiên.

Ngược lại, hệ hình học máy (Learning-based Systems) đảo ngược hoàn toàn quy trình xây dựng phần mềm. Kỹ sư chỉ cần thiết lập một kiến trúc tham số hóa $f_\theta(x)$ cùng một hàm mục tiêu (Loss Function) $\mathcal{L}$. Thông qua giải thuật tối ưu hóa, máy tính sẽ tự động điều chỉnh tập tham số θ để mô hình hóa quy luật ẩn sâu trong dữ liệu:

$$\text{Dữ liệu đầu vào (Data)} + \text{Mục tiêu mong muốn (Output)} \longrightarrow \text{Mô hình tri thức } (\theta^*) \tag{2}$$

2.6 Định nghĩa Hình thức về Học máy theo Tom Mitchell (1997)

Nhằm chuẩn hóa học máy thành một ngành khoa học toán học có khả năng đo lường chính xác, Tom Mitchell (1997) đã đưa ra định nghĩa hình thức:

> *"A computer program is said to learn from experience $\mathcal{E}$ with respect to some class of tasks $\mathcal{T}$ and performance measure $\mathcal{P}$, if its performance at tasks in $\mathcal{T}$, as measured by $\mathcal{P}$, improves with experience $\mathcal{E}$."*

Định nghĩa này cấu thành từ bộ ba toán học $(\mathcal{T}, \mathcal{E}, \mathcal{P})$ chặt chẽ:

- **Nhiệm vụ $\mathcal{T}$ (Task):** Mục tiêu nghiệp vụ xác định mà thuật toán cần giải quyết. Ví dụ trong thị giác máy tính: Phát hiện và định vị các vết nứt trên bề mặt sản phẩm công nghiệp từ ảnh $I \in \mathbb{R}^{H \times W \times C}$.
- **Trải nghiệm $\mathcal{E}$ (Experience):** Tập hợp dữ liệu lịch sử hoặc chuỗi tương tác thực tế mà mô hình được tiếp cận. Ví dụ: Tập dữ liệu huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ gồm $100,000$ hình ảnh đã được chuyên gia gán nhãn hộp bao lỗi chuẩn xác.
- **Độ đo hiệu năng $\mathcal{P}$ (Performance Measure):** Thước đo định lượng dùng để đánh giá chất lượng thực thi tác vụ. Ví dụ: Chỉ số độ chính xác trung bình mAP@0.5 trên tập kiểm thử độc lập $\mathcal{D}_{\text{test}}$.

Về mặt giải tích, thuật toán được chứng minh là đang thực hiện tiến trình học tập khi chất lượng thực thi tăng đơn điệu theo lượng trải nghiệm tích lũy:

$$\frac{\partial \mathcal{P}(\mathcal{T})}{\partial |\mathcal{E}|} > 0 \quad \text{tương đương với} \quad \lim_{|\mathcal{E}| \to \infty} \mathcal{L}(\theta; \mathcal{E}) \to \min \tag{3}$$

2.7 Tiến trình Chuyển tiếp từ Học máy sang Học sâu: Giới hạn và Động lực Đột phá

Lịch sử phát triển của thị giác máy tính ghi nhận bước chuyển dịch mang tính nền tảng từ các thuật toán học máy cổ điển sang các kiến trúc học sâu:

Trong kỷ nguyên học máy truyền thống (trước năm 2012), quy trình xử lý hình ảnh bị phân tách thành hai công đoạn độc lập: (1) Trích xuất đặc trưng thủ công (Hand-crafted Feature Extraction) dựa trên các bộ mô tả hình học cục bộ do con người thiết kế như SIFT (Lowe, 2004), HOG (Dalal & Triggs, 2005) hay LBP; (2) Đưa các vector đặc trưng này vào các mô hình phân loại nông như Support Vector Machines (SVM (Cortes & Vapnik, 1995)) hoặc Random Forest (Breiman, 2001). Ngược lại, học sâu tích hợp toàn bộ tiến trình thành một mạng nơ-ron thống nhất theo cơ chế học biểu diễn đầu-cuối (End-to-End Representation Learning), ánh xạ trực tiếp từ cường độ pixel thô đến dự đoán ngữ nghĩa cuối cùng.

Sự chuyển dịch này xuất phát từ ba hạn chế căn bản của phương pháp học máy truyền thống:

- *Nút thắt cổ chai từ đặc trưng thủ công:* Các bộ trích xuất đặc trưng truyền thống chỉ nắm bắt được các biến đổi hình học cục bộ đơn giản (đường biên, góc, kết cấu xám), dễ bị phá vỡ tính bất biến trước các biến dạng phối cảnh phức tạp, hiện tượng che khuất hoặc thay đổi điều kiện ánh sáng mạnh.
- *Lời nguyền số chiều (Curse of Dimensionality):* Khi độ phân giải ảnh tăng cao, không gian đặc trưng bùng nổ theo cấp số nhân khiến các thuật toán dựa trên ranh giới phẳng hoặc khoảng cách hình học bị suy thoái hiệu năng nghiêm trọng.
- *Sự bão hòa dung lượng biểu diễn:* Khi quy mô dữ liệu mở rộng từ hàng vạn lên hàng triệu mẫu, hiệu năng của các mô hình nông nhanh chóng chạm ngưỡng bão hòa do dung lượng tham số bị hạn chế.

Ngược lại, học sâu tạo nên bước đột phá nhờ ba động lực cốt lõi: khả năng tự động học biểu diễn phân cấp (Hierarchical Feature Abstraction) từ đường nét cục bộ ở tầng thấp đến ngữ nghĩa hoàn chỉnh ở tầng cao; tuân theo định luật mở rộng quy mô (Scaling Laws) khi hiệu năng liên tục tăng trưởng tương ứng với dung lượng dữ liệu và kích thước mô hình dưới sự hỗ trợ của phần cứng GPU; và khả năng biểu diễn đa phương thức thống nhất giữa hình ảnh, video, âm thanh và ngôn ngữ tự nhiên.

2.8 Nền tảng của Học sâu: Vì sao Nguyên lý Đơn giản tạo nên Khả năng Biểu diễn Vượt trội?

Về mặt toán học, mọi kiến trúc mạng nơ-ron sâu phức tạp đều được cấu thành từ việc xếp chồng ba phép toán nền tảng:

1. Phép biến đổi affine tuyến tính: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$.
2. Phép kích hoạt phi tuyến cục bộ từng phần tử: $\mathbf{a} = \sigma(\mathbf{z})$ (với σ là các hàm ReLU, GELU, hoặc Sigmoid).
3. Quá trình tối ưu hóa tham số thông qua giải thuật lan truyền ngược (Backpropagation) dựa trên quy tắc đạo hàm chuỗi (Chain Rule) và phương pháp hạ dốc độ dốc ngẫu nhiên (SGD (Robbins & Monro, 1951)).

Sự kết hợp của những nguyên lý giải tích đơn giản này tạo nên năng lực biểu diễn vượt trội nhờ ba cơ chế toán học sâu sắc:

1. Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem): Mạng nơ-ron chứa các tầng ẩn phi tuyến có khả năng xấp xỉ bất kỳ hàm số liên tục nào với sai số nhỏ tùy ý. Khi mở rộng chiều sâu kiến trúc, cấu trúc phân cấp cho phép mô hình biểu diễn các hàm logic cực kỳ phức tạp với số lượng tham số tiết kiệm hơn theo hàm mũ so với một mạng nông tương đương.

2. Cấu trúc thiên lệch cảm ứng (Inductive Bias): Các kiến trúc hiện đại được thiết kế để khai thác tối đa cấu trúc vật lý của dữ liệu thị giác. Phép toán tích chập (Convolution) tận dụng tính bất biến tịnh tiến và tính tương quan không gian cục bộ của ảnh tự nhiên. Tương tự, cơ chế tự chú ý (Self-Attention) trong Vision Transformer mô hình hóa quan hệ phụ thuộc toàn cảnh linh hoạt giữa các vùng ảnh.

3. Hình học tối ưu trong không gian tham số cao chiều: Trong không gian tham số hàng triệu đến hàng tỷ chiều, các điểm cực tiểu cục bộ xấu (Spurious Local Minima) hiếm khi tồn tại; thay vào đó, các điểm dừng chủ yếu là các điểm yên ngựa (Saddle Points) mà giải thuật tối ưu hóa có thể vượt qua, giúp mô hình hội tụ về các thung lũng nghiệm phẳng có khả năng tổng quát hóa cao.

2.9 Mô hình 5 Cấp độ Nhận thức và Dữ liệu: Ứng dụng Giám sát An ninh Trường học

Quá trình chuyển hóa dữ liệu từ dạng tín hiệu vật lý thô sang hành động có giá trị nhân văn được hệ thống hóa qua **5 Cấp độ Nhận thức**:

1. **Level 1: Unfiltered Data (Dữ liệu thô chưa lọc):** Tín hiệu điểm ảnh quang học, sóng âm thanh, dữ liệu cảm biến vật lý thô.
2. **Level 2: Pattern Information (Thông tin mẫu):** Các mẫu hình học, đặc trưng thị giác, vector nhúng ngữ nghĩa được trích xuất có cấu trúc.
3. **Level 3: Predictability Knowledge (Tri thức có tính dự đoán):** Định danh đối tượng, xu hướng chuyển động, mối liên hệ nhân quả và quy luật hành vi.
4. **Level 4: Choice Intelligence (Trí thông minh ra quyết định):** Khả năng lựa chọn phương án can thiệp tối ưu theo quy tắc nghiệp vụ.
5. **Level 5: Compassion Wisdom (Thông tuệ thấu cảm):** Đánh giá hoàn cảnh nhân văn, thấu hiểu nguyên nhân gốc rễ và đưa ra giải pháp nhân bản.

Bảng 9 phân tích ma trận ứng dụng 5 cấp độ này vào bài toán **Hệ thống Giám sát An ninh & Điểm danh Khuôn mặt tại Trường học**.

Bảng 9: Ma trận phân tích 5 cấp độ nhận thức ứng dụng trong Hệ thống Giám sát An ninh Trường học

Task hệ thống	Cấp độ nhận thức	Dữ liệu đầu vào (Input)	Dữ liệu đầu ra (Output)	Nội dung chi tiết & Ví dụ minh họa trong ô
Task 1: Thu nhận luồng hình ảnh	**Level 1: Unfiltered Data**	Luồng video RTSP từ camera giám sát tại cổng trường	Mảng ma trận byte điểm ảnh thô $I_t \in \mathbb{R}^{H \times W \times 3}$	**Mức 1: Dữ liệu thô.** Thu nhận liên tục 30 khung hình mỗi giây chứa toàn bộ thông tin nền (bóng râm, cây cối, xe cộ, biển thiên ánh sáng) mà hệ thống chưa phân định được sự hiện diện của học sinh.
Task 2: Phát hiện & Trích xuất khuôn mặt	**Level 2: Pattern Information**	Khung ảnh thô I_t	Hộp bao khuôn mặt $\mathcal{B}$ và Vector nhúng đặc trưng $\mathbf{e} \in \mathbb{R}^{512}$	**Mức 2: Thông tin mẫu.** Thuật toán RetinaFace định vị vùng mặt và trích xuất vector 512 chiều biểu diễn cấu trúc khoảng cách hình học giữa mắt, mũi và miệng.
Task 3: Định danh & Đối chiếu lịch sử	**Level 3: Predictability Knowledge**	Vector nhúng $\mathbf{e}$ kèm cơ sở dữ liệu học sinh $\mathcal{D}_{\text{student}}$	Danh tính học sinh kèm độ tin cậy và lịch sử chuyên cần	**Mức 3: Tri thức dự đoán.** Xác định khuôn mặt thuộc về học sinh với độ tin cậy 99.4%; nhận diện học sinh đi trễ 3 ngày liên tiếp, dự báo xu hướng vi phạm nội quy.
Task 4: Điều khiển cổng & Cảnh báo tự động	**Level 4: Choice Intelligence**	Kết quả định danh và quy tắc nghiệp vụ an ninh	Tín hiệu relay mở chốt cổng, thông báo phụ huynh hoặc chuông báo	**Mức 4: Trí thông minh lựa chọn.** Nếu là học sinh hợp lệ: tự động kích hoạt relay mở cổng và gửi tin nhắn điểm danh cho phụ huynh; nếu phát hiện đối tượng lạ: tự động khóa cổng và báo bảo vệ.
Task 5: Đánh giá hoàn cảnh & Hỗ trợ nhân văn	**Level 5: Compassion Wisdom**	Nhật ký chuyên cần, biểu cảm mệt mỏi và dữ liệu hoàn cảnh gia đình	Đề xuất tư vấn tâm lý, hỗ trợ hoàn cảnh khó khăn	**Mức 5: Thông tuệ thấu cảm.** Không xử lý trừ điểm thi đua máy móc; hệ thống ghi nhận dấu hiệu kiệt sức kèm thông tin gia cảnh khó khăn để đề xuất giáo viên chủ nhiệm thăm hỏi và hỗ trợ tâm lý kịp thời.

2.10 Mô hình Giải pháp 3 Tầng trong Khoa học Dữ liệu Thị giác

Để hiện thực hóa một giải pháp thị giác máy tính hoàn chỉnh phục vụ thực tế, kiến trúc kỹ thuật được chuẩn hóa thành **Mô hình Giải pháp 3 Tầng** (Bảng 10):

1. **Tầng 1: Kỹ thuật Cốt lõi / Nền tảng Thuật toán (Foundational Technologies):** Các mô hình giải tích toán học, kiến trúc mạng nơ-ron nền tảng (CNN, Vision Transformer), hàm mất mát chuyên biệt, thuật toán tối ưu (AdamW (Loshchilov & Hutter, 2019), SGD (Robbins & Monro, 1951)) và hình học đa góc nhìn (Epipolar, Homography).

2. **Tầng 2: Tác vụ Thị giác Cơ sở (Core Vision Tasks):** Các khối chức năng thị giác cơ bản giải quyết bài toán kỹ thuật chuyên biệt như Phân loại ảnh, Phát hiện đối tượng, Nhận dạng khuôn mặt, Ước lượng chiều sâu hay Theo dõi vết chuyển động.

3. **Tầng 3: Hệ thống Ứng dụng Nghiệp vụ Hoàn chỉnh (End-to-End Business Applications):** Tích hợp các khối chức năng Tầng 2 cùng hệ thống cơ sở dữ liệu, phân quyền bảo mật, giao diện người dùng và cơ chế điều khiển phần cứng nhằm tạo thành sản phẩm hoàn chỉnh phục vụ nhu cầu xã hội.

Bảng 10: Cấu trúc Mô hình Giải pháp 3 Tầng trong Thị giác máy tính

Phân tầng	**Thành phần cốt lõi**	**Ví dụ minh họa cụ thể**
Tầng 1: Kỹ thuật cốt lõi	Kiến trúc mạng (ResNet (He et al., 2016), Swin-ViT (Liu et al., 2021), YOLO-backbone (Redmon et al., 2016)), Hàm mất mát (ArcFace, InfoNCE, GIoU (Rezatofighi et al., 2019)), Thư viện tối ưu (PyTorch, TensorRT).	Mạng ResNet-50 (He et al., 2016) kết hợp hàm mất mát ArcFace tối ưu hóa bằng thuật toán AdamW (Loshchilov & Hutter, 2019) trên tập dữ liệu chuẩn.
Tầng 2: Tác vụ thị giác	Các bài toán cơ sở: Face Detection, Facial Landmark Alignment, Re-Identification, Crowd Density Estimation.	Thuật toán phát hiện 68 điểm mốc khuôn mặt và trích xuất vector đặc trưng khuôn mặt 512 chiều.
Tầng 3: Ứng dụng nghiệp vụ	Hệ thống hoàn chỉnh tích hợp cơ sở dữ liệu, phân quyền bảo mật, cảnh báo thời gian thực và tự động hóa quy trình nghiệp vụ.	Hệ thống Kiosk tự động Check-in hành khách tại sân bay tích hợp cổng kiểm soát xuất nhập cảnh sinh trắc học.

2.11 Khái quát các Nhóm Ứng dụng Cơ bản của Thị giác Máy tính

Thị giác máy tính hiện đại đóng vai trò công nghệ nền tảng cho 5 nhóm lĩnh vực công nghiệp trọng yếu, được tổng hợp trong Bảng 11.

Bảng 11: Tổng hợp 5 nhóm lĩnh vực ứng dụng cơ bản của Thị giác máy tính

Lĩnh vực ứng dụng	**Các tác vụ Tầng 2 tích hợp**	**Hệ thống sản phẩm Tầng 3 tiêu biểu**
1. Giám sát an ninh & Đô thị thông minh	Detection, Tracking, License Plate Recognition, Crowd Counting.	Hệ thống camera giám sát và phạt nguội vi phạm giao thông; Trung tâm điều hành an ninh trật tự đô thị.
2. Y tế số & Chẩn đoán hình ảnh	Semantic Segmentation, 3D Voxel Reconstruction, Anomaly Detection.	Phần mềm tự động khoanh vùng tổn thương khối u trên ảnh chụp CT/MRI; Hệ thống phát hiện sớm polyp nội soi tiêu hóa.
3. Tự hành & Robot công nghiệp	3D Object Detection, Depth Estimation, Visual SLAM, Optical Flow.	Hệ thống xe tự lái; Cánh tay robot công nghiệp tự động định vị gắp thả linh kiện và kiểm tra lỗi hàn vi mạch.
4. Bán lẻ & Thương mại điện tử	Image Retrieval, Fine-grained Classification, Pose Estimation.	Công cụ tìm kiếm sản phẩm bằng hình ảnh trên sàn thương mại điện tử; Chuỗi cửa hàng bán lẻ tự động không người thu ngân.
5. Điện ảnh, CGI & Thực tế ảo (AR/VR)	3D Face Reconstruction, Motion Capture, Generative Video Inpainting.	Công nghệ kỹ xảo điện ảnh kỹ thuật số (VFX); Tạo nhân vật số Digital Avatar thời gian thực; Bộ lọc hiệu ứng khuôn mặt AR trên thiết bị di động.

2.12 Phân tích Chuyên sâu: Ứng dụng Trí tuệ Nhân tạo trong Ngành Điện ảnh

Ứng dụng AI trong phân tích trải nghiệm khán giả điện ảnh (AI for Cinema Audience Analysis) là minh chứng sinh động cho sự liên kết chặt chẽ giữa cả ba tầng kỹ thuật nhằm tối ưu hóa giá trị nghệ thuật và hiệu quả thương mại (Bảng 12).

Bảng 12: Quy trình phân tích trải nghiệm khán giả điện ảnh ứng dụng AI

Giai đoạn phân tích	**Công nghệ & Tác vụ thị giác (Tầng 1 & 2)**	**Giá trị ứng dụng nghiệp vụ điện ảnh (Tầng 3)**
1. Nhận diện khuôn mặt & Cảm xúc	Camera hồng ngoại trong rạp tối kết hợp thuật toán Facial Landmark Detection và mô hình phân loại biểu cảm theo hệ thống FACS.	Định lượng chính xác mức độ kích thích cảm xúc của khán giả tại từng phân cảnh; kiểm chứng thực nghiệm xem cảnh phim có đạt hiệu ứng nghệ thuật kỳ vọng.
2. Phân tích hành vi & Hướng nhìn	Mô hình ước lượng tư thế đầu (Head Pose), theo dõi ánh mắt (Eye Gaze Tracking) và nhận diện hành vi cơ thể (ngả người, che mắt, xem điện thoại).	Đánh giá mức độ tập trung và gắn kết (Engagement Level) của người xem; phát hiện chính xác các đoạn phim gây nhàm chán hoặc phân tâm.
3. Đồng bộ Timeline & Tối ưu kịch bản	Tích hợp chuỗi dữ liệu cảm xúc khán giả đồng bộ thời gian với biểu đồ dựng phim (Cut-by-cut synchronization).	Hỗ trợ đạo diễn và nhà sản xuất trong các buổi chiếu thử nghiệm (Test Screening): cắt gọt các phân đoạn dư thừa, điều chỉnh nhịp độ phim (Pacing) và tối ưu hóa chiến dịch quảng bá phòng vé.

2.13 Ý nghĩa Thực tiễn của Bài toán Đếm Đám đông ở Tầng Ứng dụng Nghiệp vụ

Ở cấp độ kỹ thuật Tầng 2, bài toán Đếm đám đông (Crowd Counting) thực hiện ánh xạ ảnh thô $I \in \mathbb{R}^{H \times W \times C}$ sang trường mật độ liên tục $D(x, y)$ để ước lượng tổng số thực thể $\hat{N} = \sum_{x,y} D(x, y)$. Khi được tích hợp lên **Tầng 3 (Hệ thống Ứng dụng Nghiệp vụ Thực tiễn)**, công nghệ này mang lại ba giá trị quản trị và an toàn xã hội thiết thực:

Trước hết, hệ thống đóng vai trò quyết định trong việc phòng ngừa thảm họa giẫm đạp (Stampede Prevention). Tại các sự kiện lễ hội, nhà ga tàu điện ngầm hoặc sân vận động đông người, hệ thống liên tục giám sát mật độ cục bộ. Khi mật độ vượt ngưỡng an toàn (> 4 người$/\text{m}^2$), hệ thống tự động phát tín hiệu cảnh báo nguy cơ tắc nghẽn để lực lượng an ninh kịp thời phân luồng di chuyển trước khi sự cố xảy ra.

Thứ hai, dữ liệu đếm đám đông hỗ trợ tối ưu hóa phân bổ nguồn lực công cộng. Các cơ quan quản lý đô thị có thể dựa vào biểu đồ mật độ thời gian thực để linh hoạt điều phối tần suất xe buýt, tàu điện, bố trí lực lượng cứu hộ và mở thêm các làn thoát hiểm cục bộ.

Cuối cùng, trong lĩnh vực bán lẻ và quy hoạch không gian thương mại, việc phân tích lưu lượng người đi bộ qua từng gian hàng giúp các nhà quản lý xác định vị trí thu hút khách hàng để tối ưu hóa bố cục mặt bằng kinh doanh và định giá vị trí đặt biển quảng cáo một cách khách quan.

2.14 Phân tích Hệ thống Tái tạo Khuôn mặt 3D theo Mô hình Giải pháp 3 Tầng

Hệ thống tái tạo khuôn mặt 3D (3D Face Reconstruction) từ ảnh 2D đơn góc nhìn là ví dụ mẫu mực cho một kiến trúc phân tầng toàn diện từ cơ sở toán học hình học đến phần mềm nghiệp vụ hoàn chỉnh (Bảng 13).

Bảng 13: Phân tích hệ thống Tái tạo Khuôn mặt 3D theo Mô hình Giải pháp 3 Tầng

Phân tầng	Nhiệm vụ & Thành phần kỹ thuật	Chi tiết giải pháp công nghệ
Tầng 1: Kỹ thuật cốt lõi	Cơ sở hình học 3D, kiến trúc mạng nơ-ron và hàm tối ưu khả vi (Differentiable Rendering).	Mô hình tham số hình học 3D Morphable Model (3DMM như BFM, FLAME); Mạng Transformer trích xuất hệ số hình học $\boldsymbol{\alpha}$, biểu cảm $\boldsymbol{\beta}$ và tư thế $\mathbf{p}$; Hàm mất mát Photometric Consistency Loss, Landmark Reprojection Loss và Perceptual Identity Loss.
Tầng 2: Tác vụ thị giác cơ sở	Các khối trích xuất thông tin trung gian từ ảnh 2D sang không gian 3D.	Phát hiện 68 hoặc 468 điểm mốc không gian 3D; Ước lượng trường độ sâu đơn sắc (Monocular Depth Estimation); Tách bản đồ phản xạ ánh sáng bề mặt (Albedo/Texture UV Map) và góc chiếu sáng hình cầu (Spherical Harmonics).
Tầng 3: Ứng dụng nghiệp vụ hoàn chỉnh	Tích hợp vào các phần mềm ứng dụng phục vụ người dùng cuối.	• **Y tế & Thẩm mỹ:** Mô phỏng trực quan kết quả phẫu thuật chỉnh hình hàm mặt trước khi can thiệp y khoa. • **Giải trí & Game:** Tự động khởi tạo Digital Avatar 3D cá nhân hóa có khả năng biểu cảm thời gian thực trong game và thế giới thực tế ảo. • **Bảo mật sinh trắc học:** Xác thực khuôn mặt 3D chống giả mạo (Face Anti-spoofing) ngăn chặn các hành vi vượt rào bằng ảnh in 2D hoặc video kỹ thuật số.

3 Tuần 4: Hệ tọa độ, Thuật toán Vẽ Đường thẳng, Đường tròn và Đường Elip

3.1 Phân biệt ba hệ tọa độ WCS, UCS và DCS

Câu hỏi: Phân biệt ba hệ trục tọa độ WCS, UCS và DCS.

Trong đồ họa máy tính, một đối tượng đi từ mô hình toán học đến điểm ảnh trên màn hình phải đi qua nhiều hệ tọa độ, mỗi hệ phục vụ một mục đích riêng:

Hệ tọa độ thế giới thực (World Coordinate System, WCS): Là hệ tọa độ toàn cục, duy nhất cho toàn bộ cảnh. Mọi đối tượng trong cảnh đều được đặt vào WCS để xác định vị trí tương đối giữa chúng. Tọa độ trong WCS là **số thực**, có đơn vị tùy ý do ứng dụng quy ước (mét, ki-lô-mét, micromet, năm ánh sáng...) và về mặt lý thuyết **không bị chặn** $(x, y, z) \in \mathbb{R}^3$.

Hệ tọa độ người dùng (User Coordinate System, UCS): Là hệ tọa độ cục bộ do người dùng (người thiết kế, lập trình viên) tự định nghĩa sao cho việc mô tả một đối tượng là thuận tiện nhất, ví dụ đặt gốc tọa độ tại tâm bánh xe, trục Ox dọc theo thân xe. UCS còn được gọi là hệ tọa độ mô hình (Modeling Coordinates) hay hệ tọa độ cục bộ (Local Coordinates). Tọa độ trong UCS cũng là số thực; một UCS liên hệ với WCS thông qua một phép biến đổi affine (tịnh tiến, quay, tỉ lệ):

$$\mathbf{P}_{\text{WCS}} = \mathbf{M}_{\text{UCS}\to\text{WCS}} \cdot \mathbf{P}_{\text{UCS}} \tag{4}$$

Nhờ đó một mô hình chỉ cần mô tả một lần trong UCS rồi có thể đặt vào WCS nhiều lần ở các vị trí, hướng, kích thước khác nhau (instancing).

Hệ tọa độ thiết bị (Device Coordinate System, DCS): Là hệ tọa độ của thiết bị hiển thị (màn hình, máy in, cửa sổ ứng dụng). Tọa độ là **số nguyên** (chỉ số hàng, cột của điểm ảnh), **bị chặn** bởi độ phân giải $0 \le x_d \le W-1$, $0 \le y_d \le H-1$; gốc tọa độ thường nằm ở góc trên bên trái và trục y hướng xuống dưới. DCS phụ thuộc vào từng thiết bị cụ thể. Để chương trình độc lập với thiết bị,

người ta thường dùng thêm hệ tọa độ thiết bị chuẩn hóa (Normalized Device Coordinates, NDCS) với $x, y \in [0, 1]$ làm trung gian.

Bảng 14 tổng hợp sự khác biệt giữa ba hệ tọa độ.

Bảng 14: So sánh ba hệ tọa độ WCS, UCS và DCS

Tiêu chí	**WCS (Thế giới thực)**	**UCS (Người dùng)**	**DCS (Thiết bị)**
1. Phạm vi	Toàn cục, một hệ duy nhất cho cả cảnh.	Cục bộ, mỗi đối tượng/người dùng có thể có hệ riêng.	Gắn với một thiết bị hiển thị cụ thể.
2. Kiểu giá trị	Số thực.	Số thực.	Số nguyên (chỉ số pixel).
3. Miền giá trị	Không bị chặn, $\mathbb{R}^2$ hoặc $\mathbb{R}^3$.	Không bị chặn.	Bị chặn bởi độ phân giải $W \times H$.
4. Đơn vị	Đơn vị vật lý do ứng dụng quy ước.	Đơn vị thuận tiện cho việc mô tả đối tượng.	Pixel (điểm ảnh, điểm in).
5. Vai trò	Sắp đặt các đối tượng trong cảnh, xác định quan hệ không gian giữa chúng.	Mô hình hóa từng đối tượng một cách đơn giản, tái sử dụng được.	Hiển thị kết quả cuối cùng lên thiết bị.
6. Chuyển đổi	UCS $\rightarrow$ WCS (biến đổi mô hình) $\rightarrow$ NDCS (cắt xén, chiếu, chuẩn hóa) $\rightarrow$ DCS (ánh xạ cửa sổ – khung nhìn) $\rightarrow$ pixel (rời rạc hóa: vẽ đường thẳng, đường tròn…)		

3.2 Vì sao mô phỏng được thế giới thực bằng các vector có kích thước nhỏ

Câu hỏi: Tại sao lại nói chúng ta mô phỏng được thế giới trong khi kích thước vector rất nhỏ so với đời thực rất lớn?

Đồ họa máy tính không sao chép thế giới theo đúng kích thước vật lý của nó, mà xây dựng một **mô hình toán học** của thế giới: tập các đối tượng hình học được mô tả bằng tọa độ trong các hệ quy chiếu và liên kết với nhau bằng các phép biến đổi (Hearn et al., 2011; Hughes et al., 2014). Điều cần giữ đúng là **hình dạng** và **quan hệ không gian** của các đối tượng, không phải độ lớn tuyệt đối của các con số. Có bốn lý do khiến điều này khả thi.

1. Tọa độ là số đo tương đối, đơn vị do người mô hình hóa quy ước. Trong WCS, người lập trình được tự do chọn đơn vị phù hợp với ứng dụng: ki-lô-mét cho bản đồ, mét cho kiến trúc, micromet cho vi mạch (Hearn et al., 2011). Một vector (x, y, z) chỉ là bộ ba số đo theo đơn vị đó; giá trị 1.0 có thể là 1 mm hay 1 năm ánh sáng. Vì vậy kích thước "nhỏ" của các con số không giới hạn quy mô của thứ được mô tả, giống như tấm bản đồ tỉ lệ $1 : 1\,000\,000$ vẫn mô tả đúng cả một quốc gia.

2. Hình dạng bất biến qua phép đồng dạng. Đổi đơn vị là phép tỉ lệ đều $S(\mathbf{P}) = s\,\mathbf{P}$, $s > 0$; trong tọa độ thuần nhất đó là ma trận $\mathrm{diag}(s, s, s, 1)$, cùng loại với các ma trận biến đổi khác trong đường ống đồ họa. Phép biến đổi này bảo toàn mọi thông tin hình học:

- **Tỉ số độ dài:** $\dfrac{\|S(P) - S(Q)\|}{\|S(R) - S(T)\|} = \dfrac{s\,\|P - Q\|}{s\,\|R - T\|} = \dfrac{\|P - Q\|}{\|R - T\|}$.
- **Góc:** $\cos\angle\big(S(\mathbf{u}), S(\mathbf{v})\big) = \dfrac{s^2\,\mathbf{u}\cdot\mathbf{v}}{s^2\,\|\mathbf{u}\|\,\|\mathbf{v}\|} = \cos\angle(\mathbf{u}, \mathbf{v})$.
- **Quan hệ liên thuộc:** đường thẳng biến thành đường thẳng, đường tròn bán kính r thành đường tròn bán kính sr, điểm nằm trên một đối tượng vẫn nằm trên ảnh của đối tượng đó.

Như vậy mô hình thu nhỏ theo tỉ lệ s chứa đầy đủ thông tin hình học của thế giới thật; muốn khôi phục kích thước thật chỉ cần nhân với $1/s$.

3. Số dấu chấm động có độ chính xác tương đối. Máy tính lưu tọa độ dưới dạng số dấu chấm động theo chuẩn IEEE 754 (IEEE Computer Society, 2019): $x = (-1)^{\sigma} \cdot 1.f \cdot 2^{e}$. Với số 32 bit (phần định trị 23 bit), sai số làm tròn tương đối bị chặn bởi $u = 2^{-24} \approx 6 \times 10^{-8}$, và khoảng cách giữa hai số

biểu diễn được liền kề là $\mathrm{ulp}(x) \approx |x| \cdot 2^{-23}$ (Goldberg, 1991). Nghĩa là độ chính xác **tỉ lệ thuận với độ lớn** của giá trị: cùng một cơ chế biểu diễn cho sai số tương đối như nhau ở thang nanomet hay thang thiên văn. Ví dụ, số 32 bit tại $x = 10^4$ m có $\mathrm{ulp} = 2^{13-23} = 2^{-10} \approx 1$ mm; số 64 bit (định trị 52 bit) tại khoảng cách Trái Đất – Mặt Trời 1.5×10^{11} m vẫn phân giải được khoảng 3×10^{-5} m.

4. Độ chính xác cần thiết bị chặn bởi độ phân giải hiển thị. Kết quả cuối cùng chỉ là W điểm ảnh. Giả sử cửa sổ quan sát rộng L_w đơn vị thế giới được ánh xạ vào W pixel (mục 3.3). Một sai số δ trong WCS trở thành $\delta \cdot W/L_w$ pixel trên màn hình, và không thể nhận thấy nếu nhỏ hơn nửa pixel:

$$\delta \cdot \frac{W}{L_w} < \frac{1}{2} \iff \delta < \frac{L_w}{2W}. \tag{5}$$

Khi xem toàn cảnh một thành phố rộng 20 km trên màn hình 1920 pixel, sai số dưới $20\,000/3840 \approx 5.2$ m là vô hình; khi phóng to vào vùng rộng 2 m, ngưỡng này giảm còn khoảng 0.52 mm. Độ chính xác cần có luôn **tương đối với vùng đang quan sát**, trùng khớp với tính chất độ chính xác tương đối của số dấu chấm động ở lý do 3.

Khi thế giới quá lớn. Vì $\mathrm{ulp}(x)$ tăng theo $|x|$, các đối tượng ở rất xa gốc tọa độ bị mất độ chính xác và có thể "rung" khi hiển thị. Đồ họa máy tính xử lý bằng hai kỹ thuật bổ trợ:

- **Hệ tọa độ phân cấp:** mỗi đối tượng được mô tả trong UCS riêng với gốc đặt gần chính nó (mục 3.1), rồi ghép các phép biến đổi khi hiển thị; trong thực tế các hệ thống thường biểu diễn cảnh tương đối với vị trí camera để các tọa độ cần xử lý luôn nhỏ.
- **Mô hình phân cấp và mức chi tiết (Level of Detail):** Clark (Clark, 1976) đề xuất tổ chức cảnh thành cây phân cấp, trong đó mỗi đối tượng có nhiều phiên bản với độ chi tiết khác nhau; đối tượng càng xa hoặc càng nhỏ trên màn hình thì càng dùng phiên bản thô. Nhờ đó khối lượng dữ liệu cần xử lý phụ thuộc vào những gì nhìn thấy được, không phụ thuộc vào kích thước của cả thế giới.

Tóm lại, mô phỏng được thế giới không đòi hỏi các con số lớn như thế giới thật, mà đòi hỏi (i) bảo toàn hình dạng qua các phép biến đổi, (ii) độ chính xác tương đối đủ so với vùng quan sát và (iii) cơ chế tổ chức dữ liệu theo phân cấp. Cả ba đều được đáp ứng bởi mô hình tọa độ thực, số dấu chấm động và đường ống biến đổi của đồ họa máy tính.

3.3 Hiển thị thế giới thực rất lớn trên màn hình 2D hữu hạn

Câu hỏi: Thế giới thực rất lớn mà màn hình 2D thì nhỏ, vậy làm thế nào để hiển thị?

Màn hình chỉ có $W \times H$ điểm ảnh nguyên, trong khi WCS là không gian thực ba chiều không bị chặn. Khoảng cách này được giải quyết bằng **đường ống quan sát** (Viewing Pipeline): một chuỗi phép biến đổi và lựa chọn đưa tọa độ đi qua các hệ quy chiếu (Hearn et al., 2011; Hughes et al., 2014):

$$\begin{aligned}\text{MC (UCS)} &\xrightarrow{\mathbf{M}_{\text{model}}} \text{WC} \xrightarrow{\mathbf{M}_{\text{view}}} \text{VC} \xrightarrow{\text{chiếu}} \text{PC} \\ &\xrightarrow{\text{chuẩn hóa, cắt xén}} \text{NC} \xrightarrow{\text{cửa sổ – khung nhìn}} \text{DC} \xrightarrow{\text{rời rạc hóa}} \text{pixel}\end{aligned}$$

Mỗi bước trả lời một phần của câu hỏi "lớn thì làm sao vừa".

1. Biến đổi mô hình (MC → WC). Mỗi đối tượng được mô tả trong UCS của nó và được đặt vào WCS bằng ma trận $\mathbf{M}_{\text{model}}$ (tịnh tiến, quay, tỉ lệ). Kết quả là một cảnh thống nhất, nhưng chưa có khái niệm "nhìn" (mục 3.1).

2. Biến đổi quan sát (WC → VC): chọn chỗ đứng và hướng nhìn. Người xem được mô hình hóa bằng một camera có vị trí P_0, hướng nhìn và vector hướng lên. Ma trận $\mathbf{M}_{\text{view}} = \mathbf{R} \cdot \mathbf{T}(-P_0)$ tịnh tiến P_0 về gốc rồi quay sao cho hướng nhìn trùng với một trục tọa độ (Hearn et al., 2011). Từ đây, mọi tọa độ được đo *tương đối với người xem*: phần thế giới ở xa camera có tọa độ lớn nhưng sẽ bị thu nhỏ hoặc loại bỏ ở các bước sau.

3. Phép chiếu (VC → PC): từ 3D xuống 2D. Với phép chiếu phối cảnh lên mặt phẳng cách camera một khoảng d, từ hai tam giác đồng dạng ta có

$$x_p = d \cdot \frac{x_v}{z_v}, \qquad y_p = d \cdot \frac{y_v}{z_v}. \tag{6}$$

Một vật có kích thước h ở độ sâu z có ảnh kích thước $d \cdot h/z$: kích thước hiển thị **tỉ lệ nghịch với khoảng cách**. Đây chính là cơ chế giúp mắt người nhìn được cả dãy núi lẫn bầu trời: vật càng lớn thì thường càng ở xa, và phép chiếu thu chúng vào một mặt phẳng ảnh hữu hạn. Phép chiếu song song ($x_p = x_v$, $y_p = y_v$) giữ nguyên kích thước, phù hợp với bản vẽ kỹ thuật; khi đó việc thu nhỏ do bước 5 đảm nhận.

4. Thể tích quan sát và cắt xén (Clipping). Chỉ một vùng hữu hạn của thế giới được hiển thị: cửa sổ (Window) $[x_{w\,\min}, x_{w\,\max}] \times [y_{w\,\min}, y_{w\,\max}]$ trong 2D, hay hình chóp cụt quan sát (View Frustum) giới hạn bởi mặt gần và mặt xa trong 3D. Mọi phần nằm ngoài bị loại bỏ, ví dụ thuật toán Cohen–Sutherland gán cho mỗi đầu mút đoạn thẳng một mã vùng 4 bit (trái, phải, dưới, trên) để chấp nhận hoặc loại bỏ nhanh cả đoạn bằng phép AND/OR trên mã (Hearn et al., 2011). Nhờ cắt xén, khối lượng tính toán chỉ phụ thuộc vào phần nằm trong vùng quan sát, không phụ thuộc vào kích thước toàn bộ thế giới. Sau bước này tọa độ được chuẩn hóa về hệ NDCS ($[0, 1]$ hoặc $[-1, 1]$), độc lập với thiết bị.

5. Ánh xạ cửa sổ – khung nhìn (Window-to-Viewport). Cửa sổ được ánh xạ vào khung nhìn (Viewport) $[x_{v\,\min}, x_{v\,\max}] \times [y_{v\,\min}, y_{v\,\max}]$ trên thiết bị sao cho **vị trí tương đối** của mỗi điểm được giữ nguyên:

$$\frac{x_v - x_{v\,\min}}{x_{v\,\max} - x_{v\,\min}} = \frac{x_w - x_{w\,\min}}{x_{w\,\max} - x_{w\,\min}}, \qquad \frac{y_v - y_{v\,\min}}{y_{v\,\max} - y_{v\,\min}} = \frac{y_w - y_{w\,\min}}{y_{w\,\max} - y_{w\,\min}}. \tag{7}$$

Giải ra $x_v = x_{v\,\min} + s_x\,(x_w - x_{w\,\min})$ và $y_v = y_{v\,\min} + s_y\,(y_w - y_{w\,\min})$ với

$$s_x = \frac{x_{v\,\max} - x_{v\,\min}}{x_{w\,\max} - x_{w\,\min}}, \qquad s_y = \frac{y_{v\,\max} - y_{v\,\min}}{y_{w\,\max} - y_{w\,\min}}. \tag{8}$$

Dưới dạng ma trận, đây là hợp của ba phép biến đổi cơ sở (Hearn et al., 2011):

$$\mathbf{M}_{\mathrm{WV}} = \mathbf{T}(x_{v\,\min}, y_{v\,\min}) \cdot \mathbf{S}(s_x, s_y) \cdot \mathbf{T}(-x_{w\,\min}, -y_{w\,\min}), \tag{9}$$

tức là đưa góc cửa sổ về gốc, co giãn theo tỉ lệ khung nhìn/cửa sổ, rồi đặt vào vị trí khung nhìn. Một cửa sổ rộng hàng nghìn ki-lô-mét có s_x rất nhỏ nên vẫn vừa khít vài trăm pixel. Một số lưu ý khi cài đặt:

- **Giữ tỉ lệ khung hình:** nếu $s_x \neq s_y$ hình bị méo (đường tròn thành elip). Để giữ hình dạng (mục 3.2) ta dùng chung $s = \min(s_x, s_y)$ và căn giữa phần dư.
- **Phóng to, thu nhỏ và di chuyển:** thu hẹp cửa sổ k lần làm s tăng k lần (zoom in); tịnh tiến cửa sổ là di chuyển góc nhìn (pan). Người dùng "đi lại" trong một thế giới rộng lớn chỉ bằng cách thay đổi bốn tham số của cửa sổ.
- **Đảo trục y:** DCS thường có gốc ở góc trên trái và trục y hướng xuống, nên cần thêm $y_d = (H - 1) - y_v$.

6. Rời rạc hóa (Scan Conversion). Tọa độ (x_v, y_v) vẫn là số thực, còn pixel có tọa độ nguyên. Các thuật toán vẽ đường thẳng (mục 3.4–3.8) và đường tròn (mục 3.9–3.13) xác định tập điểm nguyên xấp xỉ tốt nhất hình học liên tục, liên tục trong lân cận 8 và có độ phức tạp tính toán thấp (Lý Quốc Ngọc, 2026a, slide 3).

Tóm lại, thế giới lớn được hiển thị trên màn hình nhỏ nhờ ba cơ chế: **chọn lọc** (chỉ hiển thị phần nằm trong cửa sổ/thể tích quan sát), **thu nhỏ có kiểm soát** (phép chiếu phối cảnh và phép tỉ lệ cửa sổ – khung nhìn bảo toàn vị trí tương đối) và **rời rạc hóa** (chuyển hình học liên tục thành pixel).

3.4 Giải thuật vẽ đoạn thẳng theo phương pháp trên bảng (tổng quát cho mọi hệ số góc)

Câu hỏi: Viết giải thuật để vẽ đoạn thẳng theo phương pháp trên bảng (viết chương trình cho tổng quát).

Bài toán. Cho tọa độ hai đầu mút của đoạn thẳng, cần xác định tập điểm trên lưới nguyên sao cho: các điểm xấp xỉ tốt nhất đoạn thẳng, liên tục trong lân cận 8 và được tính với độ phức tạp thấp (Lý Quốc Ngọc, 2026a, slide 3).

Cách 1 – tính trực tiếp. Với mỗi giá trị nguyên của x, tính $y = mx + b$ rồi vẽ điểm $\big(x, \mathrm{Round}(y)\big)$. Mỗi điểm tốn một phép nhân số thực, một phép cộng số thực và một phép làm tròn.

Cách 2 – phương pháp gia số (phương pháp trên bảng). Xét hai điểm liên tiếp (x_k, y_k) và (x_{k+1}, y_{k+1}) của đường thẳng $y = mx + b$. Lấy hiệu hai phương trình:

$$y_{k+1} - y_k = m\,(x_{k+1} - x_k) \iff \Delta y = m\,\Delta x \implies |\Delta y| = |m|\,|\Delta x|. \tag{10}$$

Ta cho tọa độ trên trục biến thiên nhanh hơn (trục chính) tăng đúng một đơn vị sau mỗi bước, khi đó tọa độ còn lại thay đổi không quá một đơn vị:

- $|m| \le 1$**:** chọn $|\Delta x| = 1$ thì $|\Delta y| = |m| \le 1$. Với $x_a < x_b$:

$$x_{k+1} = x_k + 1, \qquad y_{k+1} = y_k + m. \tag{11}$$

- $|m| > 1$**:** chọn $|\Delta y| = 1$ thì $|\Delta x| = \dfrac{|\Delta y|}{|m|} = \dfrac{1}{|m|} < 1$. Với $y_a < y_b$:

$$y_{k+1} = y_k + 1, \qquad x_{k+1} = x_k + \frac{1}{m}. \tag{12}$$

Điểm được vẽ ở mỗi bước là $\big(\mathrm{Round}(x_k), \mathrm{Round}(y_k)\big)$. So với Cách 1, phép nhân trong vòng lặp đã được thay bằng phép cộng dồn; phương pháp này còn được gọi là DDA (Digital Differential Analyzer).

Gộp các trường hợp. Đặt $\Delta x = x_b - x_a$, $\Delta y = y_b - y_a$ và số bước $n = \max(|\Delta x|, |\Delta y|)$. Gia số sau mỗi bước là

$$\delta_x = \frac{\Delta x}{n}, \qquad \delta_y = \frac{\Delta y}{n}. \tag{13}$$

Khi $|\Delta x| \ge |\Delta y|$ ta có $\delta_x = \pm 1$ và $|\delta_y| = |m|$; khi $|\Delta x| < |\Delta y|$ ta có $\delta_y = \pm 1$ và $|\delta_x| = 1/|m|$. Như vậy một công thức duy nhất bao quát cả hai trường hợp trên. Dấu của Δx, Δy quyết định hướng vẽ nên không cần hoán đổi hai đầu mút, và đoạn thẳng đứng ($\Delta x = 0$, m không xác định) cũng được xử lý vì giải thuật không cần tính m.

Tính liên tục lân cận 8. Sau mỗi bước, tọa độ trên trục chính thay đổi đúng 1, còn tọa độ trên trục phụ thay đổi một lượng thực δ với $|\delta| \le 1$. Với $\mathrm{Round}(t) = \lfloor t + 0.5 \rfloor$ và $u = t + 0.5$:

- nếu $0 \le \delta \le 1$ thì $\lfloor u \rfloor \le \lfloor u + \delta \rfloor \le \lfloor u + 1 \rfloor = \lfloor u \rfloor + 1$;
- nếu $-1 \le \delta < 0$ thì $\lfloor u \rfloor - 1 \le \lfloor u + \delta \rfloor \le \lfloor u \rfloor$.

Do đó $\big|\mathrm{Round}(t + \delta) - \mathrm{Round}(t)\big| \le 1$: hai điểm vẽ liên tiếp lệch nhau đúng 1 trên trục chính và không quá 1 trên trục phụ, tức là kề nhau trong lân cận 8. Mỗi giá trị nguyên trên trục chính có đúng một điểm được vẽ, nên nét vẽ không bị đứt và cũng không bị dày.

Mã giả. Giải thuật 1 trình bày phương pháp gia số tổng quát cho mọi hệ số góc và mọi hướng vẽ.

Mỗi vòng lặp chỉ còn hai phép cộng và hai phép làm tròn số thực. Tuy vậy, số học số thực vẫn chậm hơn số học nguyên, và sai số dấu chấm động có thể tích lũy qua nhiều bước cộng. Thuật toán Bresenham (mục 3.6) loại bỏ hoàn toàn hai nhược điểm này.

3.5 Vì sao giới hạn bài toán trong giả thiết $0 < m < 1$, $x_a < x_b$ và cách suy diễn các trường hợp còn lại

Câu hỏi: Giải thích vì sao chúng ta giới hạn bài toán chỉ nằm trong giả thiết (assumption), các trường hợp còn lại được suy diễn từ giới hạn đó như thế nào?

Bài giảng giả thiết đường thẳng $y = mx + b$ đi qua $A(x_a, y_a)$, $B(x_b, y_b)$ với $0 < m < 1$ và $x_a < x_b$; các trường hợp còn lại được suy ra dễ dàng từ trường hợp này (Lý Quốc Ngọc, 2026a, slide 4).

Lý do giới hạn bài toán.

- $x_a < x_b$**:** cho phép duyệt x theo một chiều tăng dần, $x_{k+1} = x_k + 1$.

Giải thuật 1 Vẽ đoạn thẳng tổng quát theo phương pháp gia số (Cách 2 – DDA)

Đầu vào: Hai đầu mút nguyên $A(x_a, y_a)$, $B(x_b, y_b)$
Đầu ra: Các điểm ảnh xấp xỉ đoạn thẳng AB

$\Delta x \leftarrow x_b - x_a$; $\Delta y \leftarrow y_b - y_a$
$n \leftarrow \max(|\Delta x|, |\Delta y|)$ ▷ số bước, bằng độ dài theo trục chính
if $n = 0$ **then**
  SETPIXEL(x_a, y_a) ▷ hai đầu mút trùng nhau
  return
end if
$\delta_x \leftarrow \Delta x/n$; $\delta_y \leftarrow \Delta y/n$ ▷ một trong hai gia số bằng ± 1
$x \leftarrow x_a$; $y \leftarrow y_a$ ▷ x, y là biến số thực
for $k \leftarrow 0$ **to** n **do**
  SETPIXEL$(\mathrm{Round}(x), \mathrm{Round}(y))$
  $x \leftarrow x + \delta_x$; $y \leftarrow y + \delta_y$ ▷ chỉ còn phép cộng số thực
end for

- $|m| < 1$**:** x là trục biến thiên nhanh hơn, nên khi x tăng 1 thì y tăng ít hơn 1; mỗi cột chỉ cần đúng một điểm vẽ và điểm kế tiếp luôn thuộc lân cận 8 của điểm hiện tại.
- $m > 0$**:** y chỉ giữ nguyên hoặc tăng 1, nên mỗi bước chỉ còn chọn **một trong hai** ứng viên $(x_k + 1, y_k)$ và $(x_k + 1, y_k + 1)$ (Lý Quốc Ngọc, 2026a, slide 5), quyết định bằng dấu của một biến p_k duy nhất.

Giả thiết biến bài toán chọn một trong tám lân cận thành bài toán chọn một trong hai. Ta chỉ cần xây dựng thuật toán một lần, rồi đưa mọi trường hợp khác về trường hợp này bằng phép đối xứng hình học.

Suy diễn qua từng phép đối xứng.

1. $x_a > x_b$ **– hoán đổi hai đầu mút.** Đoạn BA trùng với đoạn AB nên chỉ cần vẽ từ B đến A.
2. $-1 < m < 0$ **– đối xứng qua trục** Ox, $(x, y) \mapsto (x, -y)$. Đường thẳng $y = mx + b$ biến thành $y = -mx - b$, có hệ số góc $-m \in (0, 1)$. Vẽ đường mới bằng thuật toán gốc rồi đổi dấu y của từng điểm; trên thực tế chỉ cần thay bước $y_k + 1$ bằng $y_k - 1$.
3. $m > 1$ **– đối xứng qua đường** $y = x$, $(x, y) \mapsto (y, x)$. Đường thẳng $y = mx + b$ biến thành $x = my + b$, tức $y = \frac{1}{m}x - \frac{b}{m}$, có hệ số góc $\frac{1}{m} \in (0, 1)$. Vẽ đường mới rồi đổi chỗ hai tọa độ; trên thực tế chỉ cần đổi vai trò: y là trục chính tăng 1 mỗi bước, x giữ nguyên hoặc tăng 1.
4. $m < -1$ **– kết hợp hai phép trên:** đối xứng qua Ox để có hệ số góc $-m > 1$, sau đó đối xứng qua $y = x$.
5. **Các trường hợp đặc biệt:** $m = 0$ (nằm ngang), $|m| = 1$ (đường chéo) và $\Delta x = 0$ (thẳng đứng) vẽ trực tiếp được. Công thức Bresenham vẫn đúng với $m = 0$ (luôn có $p_k = -\Delta x < 0$, y không đổi) và $m = 1$ (luôn có $p_k = \Delta x > 0$, luôn đi chéo).

Vì sao kết quả vẫn đúng. Các phép đối xứng trên chỉ đổi dấu hoặc đổi chỗ tọa độ, nên chúng biến điểm nguyên thành điểm nguyên và giữ nguyên khoảng cách. Vì vậy pixel gần đường thẳng nhất sau biến đổi vẫn là pixel gần đường thẳng nhất trước biến đổi, và hai pixel kề nhau trong lân cận 8 vẫn kề nhau. Ba yêu cầu xấp xỉ tốt nhất, liên tục lân cận 8 và độ phức tạp thấp đều được giữ nguyên.

Ghép các trường hợp lại ta có thuật toán Bresenham tổng quát (theo bài ghi), với $s_x = \mathrm{sign}(x_b - x_a)$, $s_y = \mathrm{sign}(y_b - y_a)$:

- **Trục chính là** x ($|\Delta x| \geq |\Delta y|$): $p_0 = 2|\Delta y| - |\Delta x|$; nếu $p_k < 0$ thì $p_{k+1} = p_k + 2|\Delta y|$, vẽ $(x_k + s_x, y_k)$; nếu $p_k \geq 0$ thì $p_{k+1} = p_k + 2|\Delta y| - 2|\Delta x|$, vẽ $(x_k + s_x, y_k + s_y)$.
- **Trục chính là** y ($|\Delta x| < |\Delta y|$): $p_0 = 2|\Delta x| - |\Delta y|$; nếu $p_k < 0$ thì $p_{k+1} = p_k + 2|\Delta x|$, vẽ $(x_k, y_k + s_y)$; nếu $p_k \geq 0$ thì $p_{k+1} = p_k + 2|\Delta x| - 2|\Delta y|$, vẽ $(x_k + s_x, y_k + s_y)$.

Lưu ý: khi trục chính là y, giá trị khởi tạo là $p_0 = 2|\Delta x| - |\Delta y|$ (đổi vai trò $x \leftrightarrow y$ trong công thức $p_0 = 2\Delta y - \Delta x$), không phải $2|\Delta y| - |\Delta x|$.

3.6 So sánh Bresenham với Cách 2: vì sao Bresenham khử được phép cộng số thực

Câu hỏi: Giải thích vì sao làm theo Bresenham khử được các số thực mà phương pháp 2 trước đó không khử được phép cộng với số thực.

Bản chất số thực của Cách 2. Trong Cách 2 (mục 3.4), đại lượng được lưu và cập nhật là **chính tung độ thực** của đường thẳng:

$$y_{k+1} = y_k + m, \qquad m = \frac{\Delta y}{\Delta x}. \tag{14}$$

Vì m nói chung là số thực (phân số), y_k là số thực và phải làm tròn ở mỗi bước. Cách 2 chỉ khử được **phép nhân** (thay mx_k bằng phép cộng dồn), còn **phép cộng số thực** và **phép làm tròn** thì vẫn còn, vì thông tin vị trí đường thẳng nằm ngay trong phần lẻ của y_k.

Ý tưởng của Bresenham (Bresenham, 1965). Để chọn pixel ta không cần biết giá trị y chính xác, mà chỉ cần biết đường thẳng gần ứng viên nào hơn, tức là **dấu** của $d_1 - d_2$. Với $d_1 = y - y_k$ và $d_2 = (y_k + 1) - y$ tại $x_k + 1$ (Lý Quốc Ngọc, 2026a, slide 6):

$$d_1 - d_2 = 2m(x_k + 1) + 2b - 1 - 2y_k. \tag{15}$$

Do $\Delta x > 0$, nhân hai vế với Δx **không làm đổi dấu**; đồng thời phân số $m = \Delta y/\Delta x$ bị triệt tiêu mẫu số (Lý Quốc Ngọc, 2026a, slide 7):

$$p_k = \Delta x\,(d_1 - d_2) = 2\Delta y\,x_k - 2\Delta x\,y_k + C, \qquad C = 2\Delta y + \Delta x(2b - 1). \tag{16}$$

Chứng minh p_k luôn là số nguyên. Hằng số C chứa b là số thực, nhưng vì (x_0, y_0) nằm trên đường thẳng nên $b = y_0 - \frac{\Delta y}{\Delta x}\,x_0$, suy ra $\Delta x \cdot b = \Delta x\,y_0 - \Delta y\,x_0 \in \mathbb{Z}$. Do đó $C = 2\Delta y + 2(\Delta x\,b) - \Delta x \in \mathbb{Z}$. Với $x_k, y_k, \Delta x, \Delta y$ đều nguyên, $p_k \in \mathbb{Z}$ với mọi k. Công thức truy hồi (Lý Quốc Ngọc, 2026a, slide 8, 10)

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x\,(y_{k+1} - y_k) = \begin{cases} p_k + 2\Delta y, & p_k < 0, \\ p_k + 2\Delta y - 2\Delta x, & p_k \geq 0, \end{cases} \tag{17}$$

chỉ cộng các hằng số nguyên nên mọi phép tính trong vòng lặp là phép cộng, trừ số nguyên.

Mối liên hệ giữa hai phương pháp: Bresenham chính là Cách 2 được "phóng to" để khử phần lẻ. Gọi $y^* = m(x_k + 1) + b$ là tung độ thực tại $x_k + 1$. Từ (15), $d_1 - d_2 = 2(y^* - y_k) - 1$, nên

$$p_k = 2\Delta x\left(y^* - y_k - \tfrac{1}{2}\right). \tag{18}$$

Cách 2 chọn $y_{k+1} = \mathrm{Round}(y^*) = y_k + 1$ khi và chỉ khi $y^* - y_k \geq \frac{1}{2}$, tức khi và chỉ khi $p_k \geq 0$. Vậy **hai phương pháp chọn cùng một tập pixel**; điểm khác biệt duy nhất là Cách 2 lưu sai lệch $y^* - y_k$ dưới dạng số thực trong khoảng $[0, 1)$, còn Bresenham lưu cùng đại lượng đó sau khi nhân với $2\Delta x$ và trừ Δx. Hệ số $2\Delta x$ đúng bằng mẫu số chung nhỏ nhất làm cho cả $m = \Delta y/\Delta x$ lẫn ngưỡng $\frac{1}{2}$ trở thành số nguyên.

Bảng 15 so sánh chi phí tính toán của ba cách.

3.7 Tính giá trị khởi tạo p_0

Câu hỏi: Tính p_0 xem bằng bao nhiêu.

Để công thức truy hồi được trọn vẹn, cần tính p_0 (Lý Quốc Ngọc, 2026a, slide 9). Thay $k = 0$ vào (16):

$$p_0 = 2\Delta y\,x_0 - 2\Delta x\,y_0 + 2\Delta y + \Delta x\,(2b - 1). \tag{19}$$

Bảng 15: So sánh chi phí mỗi bước lặp giữa Cách 1, Cách 2 và Bresenham

Tiêu chí	**Cách 1:** $y = mx + b$	**Cách 2: gia số (DDA)**	**Bresenham**
Đại lượng lưu	Không lưu, tính lại từ đầu.	Tung độ thực y_k.	Biến quyết định nguyên p_k.
Phép nhân	1 phép nhân số thực.	Không.	Không.
Phép cộng	1 phép cộng số thực.	1 phép cộng số thực.	1 phép cộng số nguyên.
Làm tròn	Có.	Có.	Không.
Sai số tích lũy	Không tích lũy.	Có (sai số dấu chấm động cộng dồn).	Không (số học nguyên chính xác).

Vì điểm đầu (x_0, y_0) nằm trên đường thẳng, $y_0 = mx_0 + b \iff b = y_0 - \dfrac{\Delta y}{\Delta x} x_0$, nên $2\Delta x\, b = 2\Delta x\, y_0 - 2\Delta y\, x_0$. Thay vào:

$$\begin{aligned} p_0 &= 2\Delta y\, x_0 - 2\Delta x\, y_0 + 2\Delta y + 2\Delta x\, y_0 - 2\Delta y\, x_0 - \Delta x \\ &= 2\Delta y - \Delta x. \end{aligned} \tag{20}$$

Vì $\Delta x, \Delta y$ nguyên nên p_0 nguyên (Lý Quốc Ngọc, 2026a, slide 10).

3.8 Vì sao phương pháp Bresenham loại được phép nhân và số thực

Câu hỏi: Giải thích vì sao phương pháp Bresenham loại được phép nhân và số thực.

Loại phép nhân: nhờ công thức truy hồi. Biểu thức trực tiếp (16) chứa các phép nhân $2\Delta y \cdot x_k$ và $2\Delta x \cdot y_k$. Bresenham không tính trực tiếp mà tính hiệu hai bước liên tiếp (Lý Quốc Ngọc, 2026a, slide 8):

$$p_{k+1} - p_k = 2\Delta y\,(x_{k+1} - x_k) - 2\Delta x\,(y_{k+1} - y_k). \tag{21}$$

Theo giả thiết, $x_{k+1} - x_k = 1$ và $y_{k+1} - y_k \in \{0, 1\}$. Do đó hiệu này chỉ nhận đúng **hai giá trị hằng số**: $2\Delta y$ hoặc $2\Delta y - 2\Delta x$. Hai hằng số này được tính **một lần duy nhất** trước vòng lặp (bước B3 (Lý Quốc Ngọc, 2026a, slide 11)); phép nhân với 2 cũng chỉ là phép dịch bit. Trong vòng lặp chỉ còn một phép so sánh dấu và một phép cộng.

Loại số thực: nhờ chỉ quan tâm đến dấu. Như đã chứng minh ở mục 3.6, quyết định chọn pixel chỉ phụ thuộc vào dấu của $d_1 - d_2$. Nhân với $\Delta x > 0$ không đổi dấu nhưng triệt tiêu mẫu số của m, đưa mọi đại lượng về số nguyên.

Chứng minh quy nạp: $p_k \in \mathbb{Z}$ với mọi k.

- *Cơ sở:* $p_0 = 2\Delta y - \Delta x \in \mathbb{Z}$ theo (20).
- *Bước quy nạp:* giả sử $p_k \in \mathbb{Z}$. Khi đó $p_{k+1} = p_k + 2\Delta y$ hoặc $p_{k+1} = p_k + 2\Delta y - 2\Delta x$, là tổng của các số nguyên, nên $p_{k+1} \in \mathbb{Z}$.

Đồng thời $x_{k+1} = x_k + 1$ và $y_{k+1} \in \{y_k, y_k + 1\}$ đều nguyên. Vậy toàn bộ thuật toán chỉ dùng phép cộng, trừ số nguyên với hai hằng số (Lý Quốc Ngọc, 2026a, slide 10).

3.9 Thuật toán vẽ đường tròn cho kết quả "nham nhở" thì có được áp dụng hay không

Câu hỏi: Thuật toán vẽ đường tròn cho ra hình nham nhở như vậy thì có được áp dụng hay không?

Có, vẫn được áp dụng, vì hiện tượng răng cưa (aliasing, staircase effect) không phải là lỗi của thuật toán mà là giới hạn tất yếu của việc rời rạc hóa: đường tròn là đường cong liên tục, còn màn hình chỉ có các điểm ảnh tại tọa độ nguyên. **Mọi** thuật toán vẽ trên lưới nguyên đều cho kết quả bậc thang. Câu hỏi đúng là thuật toán có đáp ứng ba yêu cầu của bài toán (Lý Quốc Ngọc, 2026b, slide 3) hay không:

1. **Xấp xỉ tốt nhất:** tại mỗi cột $x_k + 1$, đường tròn cắt cột tại $y^* = \sqrt{r^2 - (x_k+1)^2}$. Vì $y_k - \frac{1}{2} > 0$, ta có
$$p_k < 0 \iff (x_k+1)^2 + \left(y_k - \tfrac{1}{2}\right)^2 < r^2 \iff y_k - \tfrac{1}{2} < y^*, \tag{22}$$
tức là thuật toán chọn y_k khi và chỉ khi y^* gần y_k hơn $y_k - 1$. Vậy pixel được chọn luôn là ứng viên gần đường tròn thật nhất theo phương thẳng đứng, và sai lệch không quá nửa pixel (chứng minh chi tiết ở mục 3.11). Không thuật toán nào trên cùng lưới làm tốt hơn.
2. **Liên tục lân cận 8:** được chứng minh ở mục 3.10.
3. **Độ phức tạp thấp:** chỉ dùng cộng, trừ số nguyên (mục 3.12); chỉ tính $1/8$ đường tròn, phần còn lại suy ra bằng đối xứng (mục 3.13).

Độ "nham nhở" giảm khi độ phân giải tăng: sai lệch tuyệt đối luôn ≤ 0.5 pixel, nên khi pixel càng nhỏ so với bán kính thì mắt người càng khó nhận ra bậc thang. Nếu cần mịn hơn nữa, người ta bổ sung các kỹ thuật khử răng cưa (anti-aliasing) như lấy mẫu dày (supersampling) hoặc tô cường độ pixel theo khoảng cách đến đường cong (thuật toán Xiaolin Wu). Các kỹ thuật này được đặt *bên trên* thuật toán midpoint chứ không thay thế nó.

3.10 Chọn pixel để đảm bảo liên tục trong lân cận 8 khi vẽ đường tròn

Câu hỏi: Chọn pixel như thế nào để liên tục trong lân cận 8 trong phép vẽ đường tròn?

Trong lân cận 8 của pixel hiện tại (x_k, y_k) có tám ứng viên. Bài giảng chỉ xét cung $1/8$ từ 90° đến 45°, tâm $O(0,0)$ (Lý Quốc Ngọc, 2026b, slide 4), và tại bước $k+1$ chỉ chọn giữa hai pixel (x_k+1, y_k) và (x_k+1, y_k-1) (Lý Quốc Ngọc, 2026b, slide 6). Ta chứng minh lựa chọn này đủ để đảm bảo liên tục.

Bước 1: trong bát phân đang xét, x là trục chính. Trên cung $90^\circ \to 45^\circ$ ta có $0 \leq x \leq y$. Đạo hàm từ phương trình $x^2 + y^2 = r^2$:
$$2x + 2y\frac{dy}{dx} = 0 \implies \frac{dy}{dx} = -\frac{x}{y} \in [-1, 0]. \tag{23}$$
Vậy khi đi theo chiều x tăng, y giảm (hoặc không đổi) và độ dốc có độ lớn không quá 1. Theo định lý giá trị trung bình, khi x tăng 1 đơn vị thì y giảm một lượng $|\Delta y| = |dy/dx|_\xi \leq 1$.

Bước 2: hai ứng viên là đủ. Vì x tăng đúng 1 và y giảm trong $[0,1]$, pixel kế tiếp chỉ có thể là (x_k+1, y_k) (hướng Đông) hoặc (x_k+1, y_k-1) (hướng Đông Nam). Cả hai đều thuộc lân cận 8 của (x_k, y_k), nên chọn pixel nào cũng đảm bảo liên tục. Sáu lân cận còn lại bị loại: đi lùi x hoặc tăng y trái với chiều của cung; đứng yên x sẽ vẽ hai pixel trên một cột, làm nét dày lên.

Bước 3: điều kiện dừng. Khi $x \geq y$ ta đã vượt qua đường $y = x$ (góc 45°), ở đó $|dy/dx| > 1$ và y trở thành trục chính. Vì vậy thuật toán dừng tại $x \geq y$ (Lý Quốc Ngọc, 2026b, slide 12).

Bước 4: tính liên tục được bảo toàn qua phép đối xứng. Bảy cung còn lại có được nhờ đối xứng qua $x = 0$, $y = 0$, $y = \pm x$ (mục 3.13). Các phép đối xứng này là đẳng cự biến $\mathbb{Z}^2$ vào chính nó nên hoán vị 8 lân cận (giống lập luận ở mục 3.5). Hai pixel kề nhau trong lân cận 8 vẫn kề nhau sau biến đổi, nên toàn bộ đường tròn liên tục.

3.11 Vị trí midpoint quyết định pixel nào gần đường tròn hơn

Câu hỏi: Chứng minh: nếu midpoint của đoạn đang xét nằm bên trong đường tròn thì một trong hai pixel ứng viên gần đường tròn hơn, và ngược lại khi midpoint nằm bên ngoài.

Thiết lập. Hàm đường tròn $f_{\text{circle}}(x,y) = x^2 + y^2 - r^2$ âm bên trong, bằng 0 trên biên và dương bên ngoài đường tròn (Lý Quốc Ngọc, 2026b, slide 8). Midpoint giữa hai ứng viên là $M = (x_k + 1, y_k - \frac{1}{2})$, và
$$p_k = f_{\text{circle}}\left(x_k+1, y_k - \tfrac{1}{2}\right) = (x_k+1)^2 + \left(y_k - \tfrac{1}{2}\right)^2 - r^2. \tag{24}$$
Gọi $y^* = \sqrt{r^2 - (x_k+1)^2}$ là tung độ giao điểm của đường tròn với cột $x = x_k + 1$, nằm giữa hai ứng viên: $y_k - 1 \leq y^* \leq y_k$ (theo mục 3.10). Trên cột này, $f(x_k+1, y) = y^2 - y^{*2}$ là hàm đồng biến theo y với $y \geq 0$.

Trường hợp 1: M nằm bên trong đường tròn ($p_k < 0$).

$$p_k < 0 \iff \left(y_k - \tfrac{1}{2}\right)^2 < y^{*2} \iff y_k - \tfrac{1}{2} < y^* \qquad (\text{vì } y_k - \tfrac{1}{2} > 0,\ y^* \geq 0). \tag{25}$$

Khi đó khoảng cách đến ứng viên trên là $d_{\text{trên}} = y_k - y^* < \frac{1}{2}$, còn khoảng cách đến ứng viên dưới là $d_{\text{dưới}} = y^* - (y_k - 1) > \frac{1}{2}$. Vậy $d_{\text{trên}} < d_{\text{dưới}}$: **pixel trên** $(x_k + 1, y_k)$ **gần đường tròn hơn** và được chọn (Lý Quốc Ngọc, 2026b, slide 9).

Trường hợp 2: M nằm bên ngoài hoặc trên đường tròn ($p_k \geq 0$). Tương tự, $p_k \geq 0 \iff y^* \leq y_k - \frac{1}{2}$, suy ra $d_{\text{dưới}} = y^* - y_k + 1 \leq \frac{1}{2} \leq d_{\text{trên}}$: **pixel dưới** $(x_k + 1, y_k - 1)$ **gần hơn hoặc cách đều**, và được chọn.

Giải thích hình học. Midpoint nằm trong đường tròn nghĩa là đường tròn đi qua cột $x_k + 1$ *phía trên* M, tức ở nửa trên của đoạn nối hai ứng viên, nên gần pixel trên hơn. Midpoint nằm ngoài nghĩa là đường tròn đi qua *phía dưới* M, nên gần pixel dưới hơn. Đây chính là lý do điểm giữa được dùng làm điểm thử (Lý Quốc Ngọc, 2026b, slide 8).

3.12 Khử phân số $5/4$ trong p_0 của thuật toán vẽ đường tròn

Câu hỏi: Với p_0 trong thuật toán vẽ đường tròn vướng phân số $5/4$, có cách nào đưa nó về hoàn toàn số nguyên không? Chứng minh điều này không ảnh hưởng đến kết quả (thay p_0 bằng p'_0).

Công thức gốc. Từ (1), (2) của bài giảng (Lý Quốc Ngọc, 2026b, slide 8, 9), với $x_{k+1} = x_k + 1$:

$$\begin{aligned} p_{k+1} - p_k &= \left[(x_k + 2)^2 + \left(y_{k+1} - \tfrac{1}{2}\right)^2\right] - \left[(x_k + 1)^2 + \left(y_k - \tfrac{1}{2}\right)^2\right] \\ &= (2x_k + 3) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) \\ \implies p_{k+1} &= p_k + 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1. \end{aligned} \tag{26}$$

Thay $y_{k+1} = y_k$ hoặc $y_{k+1} = y_k - 1$ (Lý Quốc Ngọc, 2026b, slide 10):

$$p_{k+1} = \begin{cases} p_k + 2x_{k+1} + 1, & p_k < 0, \\ p_k + 2x_{k+1} - 2y_{k+1} + 1, & p_k \geq 0, \end{cases} \tag{27}$$

và giá trị khởi tạo tại midpoint đầu tiên giữa $(1, r)$ và $(1, r - 1)$:

$$p_0 = f_{\text{circle}}\left(1, r - \tfrac{1}{2}\right) = 1 + \left(r - \tfrac{1}{2}\right)^2 - r^2 = \frac{5}{4} - r. \tag{28}$$

Bài giảng đề xuất làm tròn thành $p'_0 = 1 - r$ khi r nguyên (Lý Quốc Ngọc, 2026b, slide 11).

Định lý. Với r nguyên, dãy p_k (khởi tạo $\frac{5}{4} - r$) và dãy p'_k (khởi tạo $1 - r$), cùng cập nhật theo (27) với cùng điểm xuất phát $(0, r)$, sinh ra **cùng một dãy pixel**.

Chứng minh (quy nạp). Ta chứng minh với mọi k: (i) hai thuật toán đã chọn cùng các pixel $(x_0, y_0), \ldots, (x_k, y_k)$; (ii) $p'_k \in \mathbb{Z}$; (iii) $p_k = p'_k + \frac{1}{4}$.

- *Cơ sở $k = 0$:* cùng điểm $(0, r)$; $p'_0 = 1 - r \in \mathbb{Z}$; $p_0 - p'_0 = \frac{5}{4} - 1 = \frac{1}{4}$.
- *Hai thuật toán đưa ra cùng quyết định ở bước k.* Vì $p'_k \in \mathbb{Z}$:

  $$p_k < 0 \iff p'_k + \tfrac{1}{4} < 0 \iff p'_k < -\tfrac{1}{4} \iff p'_k \leq -1 \iff p'_k < 0. \tag{29}$$

  Không có số nguyên nào nằm trong khoảng $[-\frac{1}{4}, 0)$, nên p_k và p'_k luôn cùng dấu theo nghĩa của phép thử. Hai thuật toán chọn cùng pixel (x_{k+1}, y_{k+1}).
- *Bước quy nạp:* vì cùng quyết định và cùng (x_{k+1}, y_{k+1}), số gia ở (27) của hai dãy bằng nhau và là số nguyên ($2x_{k+1} + 1$ hoặc $2x_{k+1} - 2y_{k+1} + 1$). Do đó $p'_{k+1} = p'_k + (\text{số nguyên}) \in \mathbb{Z}$ và $p_{k+1} - p'_{k+1} = p_k - p'_k = \frac{1}{4}$.

Theo nguyên lý quy nạp, hai thuật toán cho cùng kết quả ở mọi bước. ■

Nhận xét. Chứng minh cho thấy p_k gốc luôn có dạng "số nguyên $+\frac{1}{4}$", nên không bao giờ bằng 0 và phần $\frac{1}{4}$ không bao giờ làm thay đổi kết quả phép thử dấu. Đó chính là lý do có thể bỏ đi (Lý Quốc Ngọc, 2026b, slide 11).

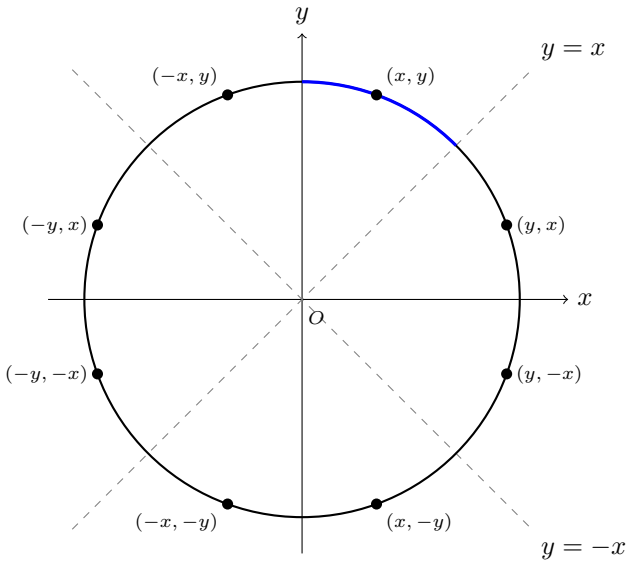
Cách khác (chính xác tuyệt đối): nhân toàn bộ biến quyết định với 4, đặt $P_k = 4p_k$. Khi đó $P_0 = 5 - 4r$, các số gia trở thành $8x_{k+1} + 4$ và $8x_{k+1} - 8y_{k+1} + 4$, tất cả đều nguyên. Vì nhân với $4 > 0$ không đổi dấu, kết quả không đổi. Cách này giống hệt ý tưởng nhân với Δx của Bresenham vẽ đoạn thẳng (mục 3.6), nhưng tốn thêm phép dịch bit nên cách $p'_0 = 1 - r$ gọn hơn.

3.13 Tọa độ 7 điểm đối xứng còn lại trong thuật toán vẽ đường tròn

Câu hỏi: Tọa độ 7 điểm còn lại trong thuật toán vẽ đường tròn được tính như thế nào?

Thuật toán chỉ tính các điểm (x, y) trên cung từ $90°$ đến $45°$ của đường tròn tâm $O(0, 0)$, nhờ tính đối xứng của đường tròn qua các trục $x = 0$, $y = 0$, $y = \pm x$ (Lý Quốc Ngọc, 2026b, slide 4, 5). Bước B8 xác định 7 điểm đối xứng, bước B9 tịnh tiến theo tâm (Lý Quốc Ngọc, 2026b, slide 12).

Chứng minh. Điểm (x, y) thuộc đường tròn khi và chỉ khi $x^2 + y^2 = r^2$. Vì $(-x)^2 = x^2$, $(-y)^2 = y^2$ và $x^2 + y^2 = y^2 + x^2$, nên khi đổi dấu một tọa độ hoặc đổi chỗ hai tọa độ, đẳng thức vẫn đúng. Do đó cả tám điểm $(\pm x, \pm y)$ và $(\pm y, \pm x)$ đều thuộc đường tròn. Về mặt hình học, đổi dấu x là đối xứng qua trục Oy, đổi dấu y là đối xứng qua trục Ox, đổi chỗ x, y là đối xứng qua đường $y = x$. Hình 1 và Bảng 16 cho vị trí của tám điểm.



Hình 1: Tám điểm đối xứng của đường tròn; cung tô đậm ($90° \to 45°$) là cung được tính bởi thuật toán

Bảng 16: Tọa độ 8 điểm đối xứng (tâm O) và sau khi tịnh tiến theo tâm $C(x_c, y_c)$

Cung	**Phép đối xứng từ** (x, y)	**Tâm** O	**Tâm** $C(x_c, y_c)$
$90° \to 45°$	Gốc (được tính)	(x, y)	$(x_c + x, y_c + y)$
$45° \to 0°$	Qua $y = x$	(y, x)	$(x_c + y, y_c + x)$
$0° \to -45°$	Qua $y = x$ rồi qua Ox	$(y, -x)$	$(x_c + y, y_c - x)$
$-45° \to -90°$	Qua Ox	$(x, -y)$	$(x_c + x, y_c - y)$
$-90° \to -135°$	Qua O	$(-x, -y)$	$(x_c - x, y_c - y)$
$-135° \to -180°$	Qua $y = -x$	$(-y, -x)$	$(x_c - y, y_c - x)$
$180° \to 135°$	Qua $y = x$ rồi qua Oy	$(-y, x)$	$(x_c - y, y_c + x)$
$135° \to 90°$	Qua Oy	$(-x, y)$	$(x_c - x, y_c + y)$

Tịnh tiến theo tâm. Đường tròn tâm $C(x_c, y_c)$ là ảnh của đường tròn tâm O qua phép tịnh tiến theo vector (x_c, y_c), nên mỗi điểm tính được chỉ cần cộng thêm (x_c, y_c). Đây là phép cộng số nguyên nên không làm tăng độ phức tạp.

Lưu ý: khi $x = 0$ (điểm đầu $(0, r)$) hoặc $x = y$ (điểm trên đường $45°$), một số điểm trong tám điểm trùng nhau và có thể bỏ bớt để không vẽ một pixel hai lần.

3.14 MÔ TẢ QUÁ TRÌNH TEXT-TO-VECTOR (THEO BÀI ĐĂNG TEXT2VECTOR)

Câu hỏi: Xem bài đăng text2vector rồi mô tả quá trình.

Bài toán. Text-to-Vector Graphics Generation (Text-to-SVG) là bài toán chuyển một câu mô tả bằng ngôn ngữ tự nhiên thành hình đồ họa vector, thường ở định dạng SVG (Scalable Vector Graphics) (W3C, 2026). Ví dụ, với đầu vào *"Một con chim màu đỏ đang đậu trên cành cây, nền trắng, phong cách hoạt hình đơn giản"*, đầu ra là một tệp SVG gồm các đối tượng hình học như thân, đầu, cánh, mắt, mỏ chim và cành cây; mỗi đối tượng được biểu diễn bằng đường cong Bézier, màu tô và các thuộc tính hình học. Khác với ảnh raster lưu giá trị từng pixel, SVG lưu các đối tượng và tham số hình học để dựng hình, nên phóng to tùy ý mà không vỡ hạt và chỉnh sửa được từng thành phần.

Cấu trúc của một tệp SVG. SVG là định dạng dựa trên XML, có cấu trúc cây tương tự DOM của HTML (W3C, 2026):

- Phần tử gốc `<svg>` khai báo không gian tọa độ và kích thước canvas (`width`, `height`, `viewBox`).
- `<defs>` chứa các định nghĩa dùng lại được: gradient, vùng cắt (`clipPath`), bộ lọc.
- `<g>` gom nhóm các đối tượng và có thể lồng nhau, biểu diễn quan hệ bộ phận – toàn thể (ví dụ nhóm `bird` chứa `body`, `head`, `wing`, `eye`).
- Các phần tử hình học cơ sở: `<rect>`, `<circle>`, `<ellipse>`, `<line>`, `<polyline>`, `<polygon>` và quan trọng nhất là `<path>`. Thuộc tính `d` của `<path>` là chuỗi lệnh: `M` (di chuyển), `L`/`H`/`V` (đoạn thẳng), `C`/`S` (Bézier bậc ba), `Q`/`T` (Bézier bậc hai), `A` (cung elip), `Z` (đóng đường); chữ hoa dùng tọa độ tuyệt đối, chữ thường dùng tọa độ tương đối.
- Mỗi phần tử mang thuộc tính kiểu dáng (`fill`, `stroke`, `opacity`) và phép biến đổi (`transform`: tịnh tiến, quay, co giãn).

Dưới góc nhìn mô hình học máy, một hình vector có thể được mô hình hóa thành tập đối tượng $V = \{O_1, \ldots, O_N\}$ với $O_i = (G_i, \theta_i, S_i, T_i, Z_i)$: loại primitive và cấu trúc path, tham số hình học (tọa độ điểm điều khiển), kiểu dáng, phép biến đổi và thứ tự chồng lớp.

Quy trình tổng quát gồm năm công đoạn.

1. **Hiểu câu mô tả (Text Understanding).** Câu mô tả T được đưa qua bộ mã hóa văn bản (CLIP (Radford et al., 2021), T5, BERT) hoặc một mô hình ngôn ngữ lớn để trích xuất các đối tượng (chim, cành cây), thuộc tính (chim màu đỏ), quan hệ không gian (chim đậu trên cành), nền và phong cách. Kết quả là một biểu diễn ngữ nghĩa hoặc vector nhúng $z_{\text{text}} = E_\theta(T)$.
2. **Lập bố cục (Scene & Layout Planning).** Xác định vị trí, kích thước của từng đối tượng trên canvas và thứ tự lớp trước – sau, có thể biểu diễn bằng đồ thị cảnh (Scene Graph) hoặc các hộp bao.
3. **Sinh hình học vector (Vector Generation).** Mỗi đối tượng được chuyển thành các primitive vector: đường cong Bézier, hình tròn, elip, hình chữ nhật, kèm màu tô, nét viền và phép biến đổi. Đường cong Bézier bậc ba có dạng
$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t\, P_1 + 3(1-t)t^2 P_2 + t^3 P_3, \qquad 0 \le t \le 1, \tag{30}$$
trong đó các điểm điều khiển $P_0, \ldots, P_3$ quyết định hình dạng đường cong. Hệ thống có thể sinh trực tiếp mã SVG hoặc dự đoán các tham số hình học rồi chuyển thành mã SVG.
4. **Kết xuất và tối ưu (Rendering & Optimization).** Tập vector V được kết xuất thành ảnh raster $I = R(V)$ để so sánh với câu mô tả bằng mô hình thị giác – ngôn ngữ. Các tham số vector được điều chỉnh để giảm hàm mất mát, ví dụ
$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{semantic}} + \lambda_2 \mathcal{L}_{\text{geometry}} + \lambda_3 \mathcal{L}_{\text{regularization}}. \tag{31}$$

Bước này đòi hỏi bộ kết xuất khả vi như DiffVG (Li et al., 2020) để lan truyền gradient từ ảnh về tọa độ điểm điều khiển. Bước tối ưu thường gặp trong VectorFusion (Jain et al., 2023) và SVGDreamer (Xing et al., 2024), nhưng không bắt buộc ở mọi kiến trúc.

5. **Kiểm tra và xuất SVG.** Kiểm tra mã SVG có hợp lệ, hình có đúng mô tả và các đối tượng có chỉnh sửa độc lập được hay không, rồi xuất tệp `.svg` để mở bằng trình duyệt hoặc chỉnh sửa bằng Inkscape, Adobe Illustrator.

Không phải mô hình nào cũng triển khai đủ năm công đoạn; chẳng hạn mô hình tự hồi quy có thể sinh thẳng mã SVG từ câu mô tả mà không cần đồ thị cảnh tường minh.

Hai kiến trúc triển khai chính.

- **Kiến trúc A – sinh trực tiếp mã SVG:** câu mô tả $\rightarrow$ mô hình ngôn ngữ (đa phương thức) $\rightarrow$ mã SVG. Mã SVG được coi là chuỗi token và sinh theo cơ chế dự đoán token kế tiếp:

$$P(V \mid T) = \prod_{k=1}^{m} P(v_k \mid v_{<k}, T), \tag{32}$$

  tiêu biểu là StarVector (Rodriguez et al., 2025).

- **Kiến trúc B – tối ưu tham số vector:** mô hình khuếch tán văn bản – ảnh cung cấp tín hiệu hướng dẫn, gradient được lan truyền qua bộ kết xuất khả vi để cập nhật trực tiếp các tham số vector, tiêu biểu là VectorFusion (Jain et al., 2023).

Liên kết giữa token của câu mô tả và hình vector. Để biết cụm từ nào điều khiển đối tượng, thuộc tính hay đường vector nào, có thể mã hóa câu thành $H_T \in \mathbb{R}^{N\times d}$, mã hóa các primitive SVG thành $H_V \in \mathbb{R}^{M\times d}$ rồi tính ma trận chú ý chéo

$$A = \mathrm{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{M\times N}, \qquad Q = H_V W_Q,\ K = H_T W_K, \tag{33}$$

trong đó A_{ji} là mức liên quan giữa primitive j và token i. Liên kết cần xét ở nhiều cấp: đối tượng (*bird* $\leftrightarrow$ nhóm `<g id="bird">`), thuộc tính (*red* $\leftrightarrow$ `fill`), primitive (*bird* $\leftrightarrow$ các `<path>`, `<circle>` tạo thành con chim) và quan hệ (*sitting on* $\leftrightarrow$ vị trí tương đối giữa chim và cành). SVGDreamer (Xing et al., 2024) là ví dụ dùng bản đồ chú ý chéo của mô hình khuếch tán để phân bố các điểm điều khiển Bézier theo vùng ngữ nghĩa của từng token đối tượng.

Liên hệ với nội dung tuần này. Hình vector chỉ là mô tả toán học; để hiển thị lên màn hình, bộ kết xuất $R(V)$ phải rời rạc hóa từng `<line>`, `<circle>`, `<ellipse>` và đường cong thành pixel. Đó chính là bài toán mà các thuật toán vẽ đường thẳng và đường tròn ở tuần này giải quyết.

3.15 Ý nghĩa của góc α trong hệ tọa độ cực khi vẽ đường elip

Câu hỏi: Góc α trong hệ tọa độ cực đối với hình elip có ý nghĩa gì? (gợi ý: slide trang 5 bài vẽ elip)

Dựng elip bằng phương pháp hai đường tròn đồng tâm (Concentric Circles Method). Xét elip tâm $O(0,0)$ với bán trục lớn a (theo Ox) và bán trục nhỏ b (theo Oy):

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1. \tag{34}$$

Cách dựng như sau (Hình 2):

1. Vẽ hai đường tròn đồng tâm O: đường tròn lớn bán kính a và đường tròn nhỏ bán kính b.
2. Quét một tia từ tâm O hợp với trục Ox góc θ (góc quét của đường tròn lớn). Tia cắt đường tròn lớn tại $A(a\cos\theta,\ a\sin\theta)$ và đường tròn nhỏ tại $B(b\cos\theta,\ b\sin\theta)$.
3. Từ A kẻ đường thẳng song song với Oy, từ B kẻ đường thẳng song song với Ox. Giao điểm P của hai đường này lấy hoành độ của A và tung độ của B:

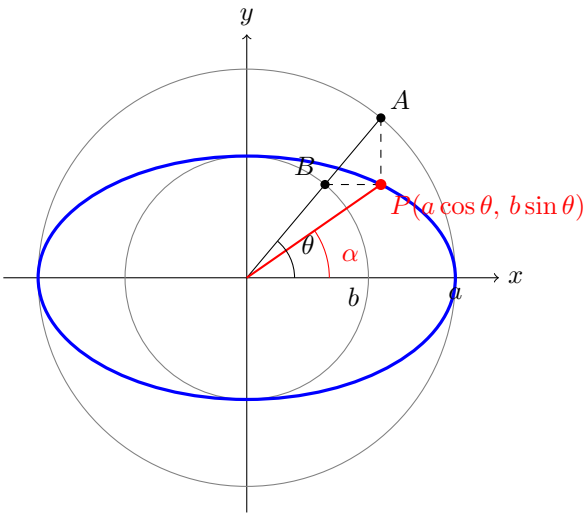
$$P = (a\cos\theta,\ b\sin\theta). \tag{35}$$

4. Cho θ chạy từ 0 đến 2π, điểm P vạch nên toàn bộ elip.

Chứng minh P nằm trên elip.

$$\frac{(a\cos\theta)^2}{a^2} + \frac{(b\sin\theta)^2}{b^2} = \cos^2\theta + \sin^2\theta = 1. \tag{36}$$

Ngược lại, với mọi điểm (x, y) trên elip, $\left(\frac{x}{a}\right)^2 + \left(\frac{y}{b}\right)^2 = 1$ nên tồn tại duy nhất $\theta \in [0, 2\pi)$ với $\cos\theta = x/a$, $\sin\theta = y/b$. Vậy phép dựng cho đúng và đủ các điểm của elip.



Hình 2: Dựng elip bằng hai đường tròn đồng tâm bán kính a và b: θ là góc quét của đường tròn lớn (tia OA), α là góc quét của elip (tia OP)

Ý nghĩa của góc α.

- **α là góc quét của elip trong hệ tọa độ cực:** α là góc hợp bởi tia OP (nối tâm với điểm P đang vẽ trên elip) và trục Ox. Trong hệ tọa độ cực tâm O, mỗi điểm của elip được xác định bởi cặp (r, α) với $x = r\cos\alpha$, $y = r\sin\alpha$. Thay vào phương trình elip:

$$r^2\left(\frac{\cos^2\alpha}{a^2} + \frac{\sin^2\alpha}{b^2}\right) = 1 \implies r(\alpha) = \frac{ab}{\sqrt{b^2\cos^2\alpha + a^2\sin^2\alpha}}. \tag{37}$$

  Như vậy khi quét α từ 0 đến 2π, khoảng cách từ tâm đến điểm vẽ thay đổi theo $r(\alpha)$: lớn nhất bằng a tại $\alpha = 0, \pi$ (hai đỉnh trục lớn) và nhỏ nhất bằng b tại $\alpha = \pm\frac{\pi}{2}$ (hai đỉnh trục nhỏ). Đây là điểm khác với đường tròn, nơi r không đổi.

- **Quan hệ giữa góc quét của elip α và góc quét của đường tròn lớn θ:** vì $\tan\alpha = \dfrac{y_P}{x_P} = \dfrac{b\sin\theta}{a\cos\theta}$ nên

$$\tan\alpha = \frac{b}{a}\tan\theta. \tag{38}$$

  Do đó $\alpha = \theta$ chỉ khi θ là bội của 90° (tại các đỉnh của elip) hoặc khi $a = b$. Với $a > b$, trong góc phần tư thứ nhất luôn có $\alpha \le \theta$: điểm P trên elip "chậm" hơn điểm A trên đường tròn lớn và lệch về phía trục lớn.

- **Elip là ảnh của đường tròn lớn qua phép co:** phép co theo trục Oy với hệ số b/a, $(x, y) \mapsto \left(x, \frac{b}{a}y\right)$, biến điểm A trên đường tròn lớn thành điểm P trên elip. Phép co giữ nguyên hoành độ nhưng ép tung độ, nên góc quét θ của đường tròn biến thành góc quét α nhỏ hơn của elip. Khi $a = b$, elip trở thành đường tròn và hai góc trùng nhau.

- **Ý nghĩa khi vẽ:** khi quét đều theo θ với bước $\Delta\theta$, độ dài cung giữa hai điểm liên tiếp xấp xỉ

$$\Delta s \approx \sqrt{a^2\sin^2\theta + b^2\cos^2\theta}\,\Delta\theta. \tag{39}$$

Gần đỉnh trục lớn ($\theta \approx 0$) có $\Delta s \approx b\,\Delta\theta$, gần đỉnh trục nhỏ ($\theta \approx 90^\circ$) có $\Delta s \approx a\,\Delta\theta$: các điểm dày ở hai đầu trục lớn và thưa ở hai đầu trục nhỏ. Để các điểm liên tiếp không cách nhau quá một pixel (liên tục lân cận 8), cần chọn $\Delta\theta \le 1/\max(a,b)$. Ngoài ra elip đối xứng qua Ox và Oy, nên chỉ cần quét trong góc phần tư thứ nhất ($0 \le \alpha, \theta \le 90^\circ$) rồi suy ra ba điểm $(\pm x, \pm y)$ còn lại.

4 TUẦN 5: VẼ CUNG TRÒN – ELLIPSE, ĐƯỜNG CONG BÉZIER VÀ TÔ MÀU ĐA GIÁC THEO DÒNG QUÉT

4.1 PHƯƠNG PHÁP VẼ CUNG TRÒN VÀ CUNG ELLIPSE TRÊN LƯỚI TỌA ĐỘ NGUYÊN

Câu hỏi: Trình bày phương pháp vẽ cung tròn và cung ellipse.

Phát biểu bài toán (Lý Quốc Ngọc, 2026c). Đầu vào: tâm $C(x_c, y_c)$, bán trục r_x, r_y (đường tròn là trường hợp $r_x = r_y = r$) và hai góc giới hạn cung θ_s, θ_e. Đầu ra: tập điểm ảnh nguyên xấp xỉ tốt nhất cung $\{(x_c + r_x\cos\theta,\ y_c + r_y\sin\theta) : \theta \in [\theta_s, \theta_e]\}$, liên tục trong lân cận 8 và có độ phức tạp tính toán thấp. Ý tưởng chung gồm hai lớp: (i) một thuật toán vẽ *toàn bộ* đường cong bằng số nguyên (Midpoint), và (ii) một phép *lọc* chỉ giữ lại các điểm thuộc cung.

1. Vẽ đường ellipse bằng phương pháp điểm giữa (Midpoint Ellipse) (Lý Quốc Ngọc, 2026c; Hearn et al., 2011). Xét ellipse tâm O và hàm

$$f_{\text{ellipse}}(x, y) = r_y^2x^2 + r_x^2y^2 - r_x^2r_y^2, \tag{40}$$

âm khi điểm ở trong, bằng 0 khi ở trên biên, dương khi ở ngoài. Do ellipse đối xứng qua $x = 0$ và $y = 0$ nên chỉ khảo sát góc phần tư thứ nhất, bắt đầu từ $(0, r_y)$. Hệ số góc tiếp tuyến $\frac{dy}{dx} = -\frac{r_y^2x}{r_x^2y}$ chia góc phần tư thành hai miền, phân giới tại $2r_y^2x = 2r_x^2y$:

- **Miền 1** ($|dy/dx| \le 1$): x tăng đều, ứng viên là $(x_k + 1, y_k)$ và $(x_k + 1, y_k - 1)$. Xét giá trị tại điểm giữa $p1_k = f_{\text{ellipse}}(x_k + 1, y_k - \frac{1}{2})$. Nếu $p1_k < 0$ thì điểm giữa nằm trong ellipse nên chọn $(x_k + 1, y_k)$, ngược lại chọn $(x_k + 1, y_k - 1)$. Lấy hiệu công thức (2) và (1) trên slide và rút gọn hạng tử $r_x^2[(y_{k+1} - \frac{1}{2})^2 - (y_k - \frac{1}{2})^2]$ (bằng 0 nếu $y_{k+1} = y_k$, bằng $-2r_x^2y_{k+1}$ nếu $y_{k+1} = y_k - 1$), ta được

$$p1_{k+1} = \begin{cases} p1_k + 2r_y^2x_{k+1} + r_y^2, & p1_k < 0, \\ p1_k + 2r_y^2x_{k+1} - 2r_x^2y_{k+1} + r_y^2, & p1_k \ge 0, \end{cases} \qquad p1_0 = r_y^2 - r_x^2r_y + \tfrac{1}{4}r_x^2. \tag{41}$$

- **Miền 2** ($|dy/dx| > 1$): y giảm đều, ứng viên là $(x_k, y_k - 1)$ và $(x_k + 1, y_k - 1)$, với $p2_k = f_{\text{ellipse}}(x_k + \frac{1}{2}, y_k - 1)$:

$$p2_{k+1} = \begin{cases} p2_k - 2r_x^2y_{k+1} + r_x^2, & p2_k > 0 \text{ (chọn } (x_k, y_k - 1)), \\ p2_k + 2r_y^2x_{k+1} - 2r_x^2y_{k+1} + r_x^2, & p2_k \le 0 \text{ (chọn } (x_k + 1, y_k - 1)), \end{cases} \tag{42}$$

  với $p2_0 = r_y^2(x_{\text{last}} + \frac{1}{2})^2 + r_x^2(y_{\text{last}} - 1)^2 - r_x^2r_y^2$ tính tại điểm cuối của miền 1. Miền 2 dừng khi $y = 0$.

Các đại lượng $2r_y^2x$ và $2r_x^2y$ được cập nhật bằng phép cộng hằng số $\pm 2r_y^2$, $\pm 2r_x^2$, nên mỗi bước chỉ cần phép cộng và so sánh dấu. Mỗi điểm (x, y) tính được sinh ra 4 điểm $(\pm x, \pm y)$ rồi tịnh tiến theo (x_c, y_c). Đường tròn là trường hợp riêng $r_x = r_y$, khi đó còn có thêm đối xứng qua $y = \pm x$ (8 điểm).

Lưu ý về slide: ở bước B13 điều kiện in là "$p2_k \ge 0$", đúng ra phải là $p2_k \le 0$ (phần bù của B12). Định nghĩa f_{ellipse} trên slide 9 ghi "outside the circle boundary" và phần phát biểu bài toán ghi "radius of circle", cần hiểu là ellipse với hai bán trục r_x, r_y.

2. Lọc điểm thuộc cung. Có hai cách:

- **Lọc theo góc bằng tích có hướng (không cần** $\mathrm{atan2}$**):** Gọi $\mathbf{s} = (\cos\theta_s, \sin\theta_s)$, $\mathbf{e} = (\cos\theta_e, \sin\theta_e)$ và $\mathbf{v} = (x - x_c, y - y_c)$ là vector từ tâm đến điểm ứng viên. Với cung quét ngược chiều kim đồng hồ có độ mở không quá 180°, điểm thuộc cung khi và chỉ khi

$$\mathbf{s} \times \mathbf{v} \ge 0 \quad \text{và} \quad \mathbf{v} \times \mathbf{e} \ge 0, \qquad \mathbf{a} \times \mathbf{b} = a_xb_y - a_yb_x. \tag{43}$$

  Với cung lớn hơn 180°, dùng phần bù: điểm thuộc cung khi *không* thỏa điều kiện trên đối với cung ngược lại từ $\mathbf{e}$ về $\mathbf{s}$. Phép kiểm tra chỉ gồm 4 phép nhân và áp dụng cho cả 8 (hoặc

4) điểm đối xứng sinh ra ở mỗi bước. Với ellipse, phải dùng hướng thật của điểm biên (góc cực α), khác với tham số θ trong $(r_x \cos\theta, r_y \sin\theta)$; quan hệ $\tan\alpha = \frac{r_y}{r_x}\tan\theta$ đã được chứng minh ở mục 3.15. Do đó khi người dùng nhập góc theo tham số θ, cần đổi $\mathbf{s}, \mathbf{e}$ thành $(r_x\cos\theta_s, r_y\sin\theta_s)$ và $(r_x\cos\theta_e, r_y\sin\theta_e)$.

- **Vẽ theo điểm mút:** tính trước điểm nguyên gần nhất của hai đầu mút, xác định chúng thuộc góc phần tư (bát phân) nào, rồi chỉ chạy Midpoint trên các phần tư bị cung phủ, dừng khi gặp điểm mút. Cách này tránh tính các điểm bị loại, nhưng phải xử lý chiều duyệt riêng cho từng phần tư (ở các phần tư khác nhau, điểm sinh ra theo chiều kim đồng hồ hoặc ngược lại).

3. Phương pháp tham số (dự phòng). Có thể vẽ trực tiếp $x = x_c + r_x\cos\theta$, $y = y_c + r_y\sin\theta$ với θ chạy từ θ_s đến θ_e, bước $\Delta\theta \le 1/\max(r_x, r_y)$ để giữ liên tục lân cận 8. Cách này đơn giản, vẽ được cả ellipse xoay (nhân thêm ma trận quay, xem mục 5.5), nhưng tốn phép tính lượng giác và số thực ở mỗi bước. Có thể giảm chi phí bằng công thức quay tăng dần $\cos(\theta+\Delta\theta) = \cos\theta\cos\Delta\theta - \sin\theta\sin\Delta\theta$, chỉ còn 4 phép nhân mỗi bước.

4.2 Khó khăn khi biểu diễn đường cong bằng quan hệ tường minh $y = f(x)$

Câu hỏi: Nếu biểu diễn mối quan hệ giữa y và x một cách tường minh ($y = f(x)$) thì sẽ gặp những khó khăn gì?

1. **Không biểu diễn được đường cong đa trị.** Hàm số gán mỗi x đúng một y, trong khi đường tròn, ellipse, đường cong khép kín hay tự cắt có nhiều y ứng với một x. Đường tròn phải tách thành hai nhánh $y = \pm\sqrt{r^2 - x^2}$, ellipse cũng vậy; đường cong chữ S nằm ngang không thể viết thành một hàm.
2. **Tiếp tuyến thẳng đứng làm đứt nét.** Khi $|f'(x)| \to \infty$ (ví dụ tại $x = \pm r$ của đường tròn), lấy mẫu đều theo x làm Δy giữa hai mẫu liên tiếp rất lớn, các điểm vẽ không còn liên tục lân cận 8. Đây chính là lý do thuật toán đường tròn, ellipse phải chia miền và đổi biến chạy (x hay y) khi $|dy/dx|$ vượt quá 1, giống như đoạn thẳng có $|m| > 1$ ở tuần 4.
3. **Mật độ điểm không đều và chi phí tính.** Lấy mẫu đều theo x thì điểm dày ở chỗ thoải, thưa ở chỗ dốc. Ngoài ra $\sqrt{\cdot}$ và số thực ở mỗi bước làm tăng độ phức tạp, trái với yêu cầu "độ phức tạp thấp" của bài toán.
4. **Phụ thuộc hệ trục, không bất biến với phép quay.** Quay một đồ thị hàm số đi một góc có thể làm nó không còn là đồ thị hàm số nữa (ví dụ quay parabol $y = x^2$ đi 90° được $x = y^2$). Mọi thao tác biến đổi hình học đều phải tìm lại một hàm mới, trong khi biến đổi trên dạng tham số chỉ cần biến đổi các điểm.
5. **Khó điều khiển hình dạng.** Thay đổi hệ số của f làm thay đổi toàn bộ đường cong một cách khó lường; người thiết kế không thể "kéo" một điểm để uốn đường cong như với điểm điều khiển.

Hướng giải quyết. Đồ họa máy tính dùng hai dạng còn lại (Hearn et al., 2011; Farin, 2002): dạng *ẩn* $F(x, y) = 0$ (như f_{ellipse}, thích hợp cho phép kiểm tra trong/ngoài, chính là cơ sở của phương pháp Midpoint) và dạng *tham số* $\mathbf{P}(u) = (x(u), y(u))$, $u \in [0, 1]$, trong đó x và y đều là hàm đơn trị của cùng một biến u. Dạng tham số biểu diễn được đường cong khép kín, tự cắt, có tiếp tuyến đứng, bất biến dưới phép biến đổi affine và cho phép điều khiển hình dạng qua các điểm điều khiển. Đường cong Bézier ở các mục sau là một dạng tham số như vậy.

4.3 Vẽ đường cong Bézier trên lưới tọa độ nguyên

Câu hỏi: Trình bày cách vẽ đường cong Bézier trên lưới tọa độ nguyên.

Phát biểu bài toán (Lý Quốc Ngọc, 2026d). Đầu vào là $n+1$ điểm điều khiển $\mathbf{p}_0, \ldots, \mathbf{p}_n$. Đầu ra là tập điểm ảnh nguyên xấp xỉ tốt nhất đường cong

$$\mathbf{P}(u) = \sum_{k=0}^{n} \mathbf{p}_k \, BEZ_{k,n}(u), \qquad BEZ_{k,n}(u) = C(n,k)\, u^k (1-u)^{n-k}, \quad 0 \le u \le 1, \tag{44}$$

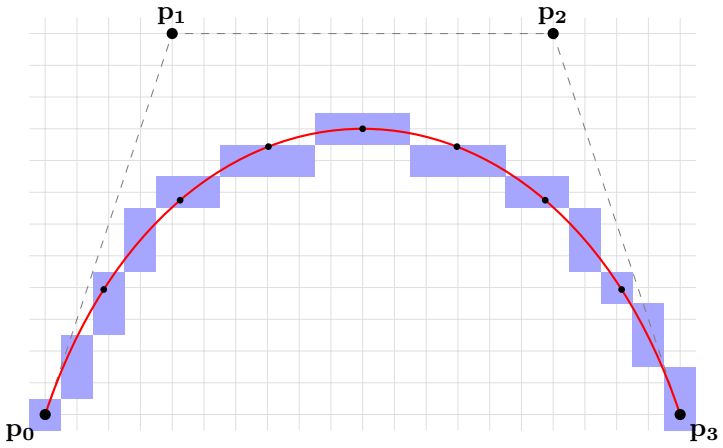
liên tục trong lân cận 8, với chi phí tính thấp. Khác với đoạn thẳng hay đường tròn, ta không có phương trình ẩn đơn giản để xét dấu tại điểm giữa, mà phải đi qua tham số u: lấy mẫu các giá trị $u_1 < u_2 < \cdots < u_N$, tính điểm thực $\mathbf{P}(u_i)$, rồi ánh xạ về lưới nguyên (hình vẽ trên bảng: các mốc $u_1, u_2, \ldots$ trên trục tham số ứng với các hoành độ $x_1, x_2, \ldots$ trên lưới). Slide nêu ba phương pháp (Lý Quốc Ngọc, 2026d):

1. **Xấp xỉ bằng các đoạn thẳng (6.3.1).** Chia đều $u_i = i/N$, tính $\mathbf{P}(u_i)$, làm tròn về điểm nguyên rồi nối các điểm liên tiếp bằng thuật toán Bresenham của tuần 4. Muốn các đoạn đủ ngắn để không thấy gấp khúc, có thể chọn N theo chiều dài đa giác điều khiển, ví dụ $N \approx \sum_k \|\mathbf{p}_{k+1} - \mathbf{p}_k\|$ (đường cong không dài hơn đa giác điều khiển do tính bao lồi).
2. **De Casteljau (6.3.2).** Tính $\mathbf{P}(u)$ bằng nội suy tuyến tính lặp (mục 4.13), ổn định số học hơn khai triển đa thức. Hơn nữa, tại $u = \frac{1}{2}$ các điểm trung gian chia đường cong thành hai đường Bézier con: nửa trái có điểm điều khiển $\mathbf{p}_0^0, \mathbf{p}_0^1, \ldots, \mathbf{p}_0^n$, nửa phải có $\mathbf{p}_0^n, \mathbf{p}_1^{n-1}, \ldots, \mathbf{p}_n^0$. Chia đệ quy đến khi đa giác điều khiển của mỗi phần "đủ phẳng" (mọi điểm điều khiển cách dây cung nối hai mút dưới 0.5 pixel) thì vẽ phần đó bằng một đoạn Bresenham. Số đoạn tự thích nghi: chỗ cong nhiều được chia nhiều, chỗ gần thẳng chia ít.
3. **Phương pháp kiểu Bresenham (6.3.3).** Slide chỉ nêu tên phương pháp. Theo tinh thần Bresenham (chỉ dùng phép cộng), ta tính các điểm liên tiếp bằng *sai phân tiến* (forward differencing) (Hughes et al., 2014). Với một tọa độ dạng đa thức bậc ba $x(u) = au^3 + bu^2 + cu + d$ và bước h, đặt

$$\Delta x_0 = ah^3 + bh^2 + ch, \qquad \Delta^2 x_0 = 6ah^3 + 2bh^2, \qquad \Delta^3 x = 6ah^3 \text{ (hằng số)}. \quad (45)$$

   Mỗi bước chỉ cần 3 phép cộng cho mỗi tọa độ: $x \mathrel{+}= \Delta x$, $\Delta x \mathrel{+}= \Delta^2 x$, $\Delta^2 x \mathrel{+}= \Delta^3 x$. Chọn h sao cho $\max(|\Delta x|, |\Delta y|) \le 1$ thì hai điểm làm tròn liên tiếp luôn kề nhau trong lân cận 8; nếu bước quá lớn thì giảm h đi một nửa, nếu quá nhỏ (hai điểm trùng pixel) thì gấp đôi h (sai phân tiến thích nghi).

Ví dụ trên lưới nguyên. Với $\mathbf{p}_0 = (0, 0)$, $\mathbf{p}_1 = (4, 12)$, $\mathbf{p}_2 = (16, 12)$, $\mathbf{p}_3 = (20, 0)$, lấy mẫu $N = 8$ ($u = 0, 0.125, \ldots, 1$) được các điểm thực $(0, 0)$, $(1.84, 3.94)$, $(4.25, 6.75)$, $(7.03, 8.44)$, $(10, 9)$, … Làm tròn và nối bằng Bresenham cho 27 điểm ảnh liên tục lân cận 8 như Hình 3. Mẫu đều theo u thì thưa ở hai đầu (nơi $|\mathbf{P}'(u)| = 3\|\mathbf{p}_1 - \mathbf{p}_0\|$ lớn) và dày ở giữa, nên việc nối bằng Bresenham là cần thiết để không đứt nét.



Hình 3: Vẽ Bézier bậc ba trên lưới nguyên: đường đỏ là đường cong lý tưởng, chấm đen là 9 mẫu $u = i/8$, ô xanh là 27 điểm ảnh sau khi làm tròn và nối bằng Bresenham

4.4 Đường cong Bézier bậc 1 nối hai đầu mút làm tiền đề cho điều gì

Câu hỏi: Đường cong Bézier bậc 1 nối hai đầu mút thì nó làm tiền đề gì cho các phần sau?

Với $n = 1$, $BEZ_{0,1}(u) = 1 - u$ và $BEZ_{1,1}(u) = u$, nên

$$\mathbf{P}(u) = (1 - u)\mathbf{p}_0 + u\,\mathbf{p}_1, \tag{46}$$

tức là đoạn thẳng nối hai đầu mút, cũng chính là phép nội suy tuyến tính (lerp). Phép toán đơn giản này là tiền đề cho bốn ý quan trọng:

1. **Viên gạch của thuật toán De Casteljau.** Mọi đường Bézier bậc n được dựng bằng cách lặp lại phép Bézier bậc 1 trên các đoạn nối điểm điều khiển (mục 4.13). Hiểu bậc 1 là hiểu cách tính mọi bậc.
2. **Tính nội suy hai đầu mút.** Bậc 1 đi qua cả $\mathbf{p}_0$ và $\mathbf{p}_1$. Tính chất này được giữ cho mọi bậc: $\mathbf{P}(0) = \mathbf{p}_0$, $\mathbf{P}(1) = \mathbf{p}_n$ (slide 5), cho phép ghép nhiều đoạn cong thành một đường liên tục.
3. **Tiêu chuẩn dừng khi vẽ.** Khi các điểm điều khiển thẳng hàng, đường Bézier suy biến thành đoạn thẳng (tính chất f, slide 8). Thuật toán chia nhỏ lợi dụng điều này: khi một đoạn con đủ phẳng thì thay bằng Bézier bậc 1 và vẽ bằng Bresenham.
4. **Tổ hợp lồi.** Hai hệ số $1 - u$ và u không âm và có tổng bằng 1. Mọi bậc cao hơn đều kế thừa tính chất này (tổng các đa thức Bernstein bằng 1), dẫn tới tính bao lồi (slide 6) và tính bất biến affine (slide 7).

4.5 Duy trì bậc thấp nhưng số điểm điều khiển cao

Câu hỏi: Làm sao để duy trì được bậc thấp của đường cong nhưng số điểm điều khiển vẫn cao?

Một đường Bézier đơn với $n + 1$ điểm điều khiển có bậc n. Khi số điểm lớn, bậc cao gây ba vấn đề: chi phí tính tăng ($O(n^2)$ phép lerp cho mỗi điểm với De Casteljau), hệ số $C(n, k)$ rất lớn gây sai số làm tròn, và mất *điều khiển cục bộ* (mỗi $BEZ_{k,n}(u) > 0$ trên $(0, 1)$ nên dời một điểm điều khiển làm thay đổi cả đường cong).

Giải pháp: ghép nhiều đoạn Bézier bậc thấp (thường là bậc 3) (Hearn et al., 2011; Farin, 2002). Với m đoạn bậc ba dùng chung điểm mút, tổng số điểm điều khiển là $3m + 1$ trong khi bậc vẫn là 3. Để đường ghép trơn, áp điều kiện tại mối nối giữa đoạn $\mathbf{P}^{(1)}$ (điểm $\mathbf{p}_0, \ldots, \mathbf{p}_3$) và $\mathbf{P}^{(2)}$ (điểm $\mathbf{q}_0, \ldots, \mathbf{q}_3$):

- **Liên tục vị trí (C^0):** $\mathbf{q}_0 = \mathbf{p}_3$, dựa trên tính chất $\mathbf{P}(1) = \mathbf{p}_n$, $\mathbf{P}(0) = \mathbf{p}_0$.
- **Liên tục tiếp tuyến:** từ $\mathbf{P}'(1) = 3(\mathbf{p}_3 - \mathbf{p}_2)$ và $\mathbf{P}'(0) = 3(\mathbf{q}_1 - \mathbf{q}_0)$, điều kiện C^1 là $\mathbf{q}_1 - \mathbf{q}_0 = \mathbf{p}_3 - \mathbf{p}_2$ (điểm nối là trung điểm của $\mathbf{p}_2\mathbf{q}_1$). Điều kiện yếu hơn G^1 chỉ yêu cầu $\mathbf{p}_2$, $\mathbf{p}_3 = \mathbf{q}_0$, $\mathbf{q}_1$ thẳng hàng và $\mathbf{p}_2$, $\mathbf{q}_1$ nằm về hai phía.
- **Liên tục độ cong (C^2):** thêm $\mathbf{P}''(1) = \mathbf{Q}''(0)$, tức $\mathbf{p}_1 - 2\mathbf{p}_2 + \mathbf{p}_3 = \mathbf{q}_0 - 2\mathbf{q}_1 + \mathbf{q}_2$ (công thức đạo hàm bậc hai ở mục 4.11).

Bậc 3 là lựa chọn phổ biến vì là bậc thấp nhất cho phép điều khiển độc lập vị trí và tiếp tuyến ở cả hai đầu, và có thể đạt C^2 khi ghép. Một cách tổng quát hơn là đường B-spline: bậc được chọn độc lập với số điểm điều khiển, mỗi điểm điều khiển chỉ ảnh hưởng tới một số đoạn lân cận và độ trơn C^{k-2} (với bậc $k - 1$) được bảo đảm tự động (Hearn et al., 2011). Định dạng SVG, PostScript và font chữ OpenType đều lưu đường cong dưới dạng chuỗi đoạn Bézier bậc 2 hoặc 3 (W3C, 2026). Các phương pháp AI sinh đồ họa vector cũng chọn biểu diễn này: DiffVG làm cho phép rời rạc hóa đường Bézier khả vi để tối ưu tọa độ điểm điều khiển bằng gradient (Li et al., 2020), và VectorFusion sinh ảnh SVG từ văn bản bằng cách tối ưu một tập đường Bézier bậc 3 ghép mảnh (Jain et al., 2023). Số điểm điều khiển lớn nhưng bậc thấp giúp quá trình tối ưu ổn định và mỗi tham số chỉ ảnh hưởng cục bộ.

4.6 Ý nghĩa các biểu thức trong slide trang 5 (Bézier)

Câu hỏi: Các biểu thức trong slide trang 5 của bài Bézier nói lên điều gì?

Slide trang 5 (Lý Quốc Ngọc, 2026d) nêu hai tính chất:

$$\text{(a) } \mathbf{P}(0) = \mathbf{p}_0, \quad \mathbf{P}(1) = \mathbf{p}_n; \qquad \text{(b) } \mathbf{P}'(0) = n(\mathbf{p}_1 - \mathbf{p}_0), \quad \mathbf{P}'(1) = n(\mathbf{p}_n - \mathbf{p}_{n-1}). \tag{47}$$

Chứng minh (a). Tại $u = 0$: $BEZ_{k,n}(0) = C(n,k)\,0^k\,1^{n-k}$ bằng 1 khi $k = 0$ và bằng 0 khi $k \geq 1$, nên $\mathbf{P}(0) = \mathbf{p}_0$. Tương tự tại $u = 1$ chỉ còn $k = n$.

Chứng minh (b). Đạo hàm đa thức Bernstein: $BEZ'_{k,n}(u) = n\,[BEZ_{k-1,n-1}(u) - BEZ_{k,n-1}(u)]$ (quy ước $BEZ_{-1,n-1} = BEZ_{n,n-1} = 0$). Thay vào và gom số hạng:

$$\mathbf{P}'(u) = n \sum_{k=0}^{n-1} (\mathbf{p}_{k+1} - \mathbf{p}_k)\, BEZ_{k,n-1}(u). \tag{48}$$

Đạo hàm của một đường Bézier bậc n là một đường Bézier bậc $n-1$ có "điểm điều khiển" là các vector cạnh $n(\mathbf{p}_{k+1} - \mathbf{p}_k)$. Áp dụng (a) cho đường này được ngay (b).

Ý nghĩa.

- (a) Đường cong *đi qua* điểm điều khiển đầu và cuối nhưng nói chung *không đi qua* các điểm điều khiển ở giữa. Hai điểm mút là nơi đường cong bị "neo", còn các điểm giữa chỉ "kéo" đường cong về phía chúng. Đây là cơ sở để ghép nhiều đoạn (mục 4.5).
- (b) Tiếp tuyến tại điểm đầu nằm dọc cạnh $\mathbf{p}_0\mathbf{p}_1$, tiếp tuyến tại điểm cuối nằm dọc cạnh $\mathbf{p}_{n-1}\mathbf{p}_n$ của đa giác điều khiển. Vì vậy người dùng điều khiển hướng xuất phát và hướng kết thúc của đường cong chỉ bằng cách dời điểm điều khiển thứ hai và kế cuối (các "tay nắm" trong phần mềm vẽ vector).
- Hai tính chất cùng nhau cho thấy hình dạng tại mỗi đầu mút chỉ phụ thuộc vào vài điểm điều khiển gần đầu mút đó: vị trí phụ thuộc 1 điểm, tiếp tuyến phụ thuộc 2 điểm, và đạo hàm bậc hai (slide 6) phụ thuộc 3 điểm.

4.7 Ý nghĩa của $\mathbf{P}'(0) = n(\mathbf{p}_1 - \mathbf{p}_0)$

Câu hỏi: Tại $\mathbf{p}_0$, đạo hàm của đường cong bằng vector $n(\mathbf{p}_1 - \mathbf{p}_0)$ thì có ý nghĩa gì?

Coi $\mathbf{P}(u)$ là quỹ đạo của một chất điểm khi "thời gian" u chạy từ 0 đến 1; $\mathbf{P}'(u)$ là vector vận tốc. Đẳng thức $\mathbf{P}'(0) = n(\mathbf{p}_1 - \mathbf{p}_0)$ mang hai thông tin:

- **Phương và chiều:** đường cong rời $\mathbf{p}_0$ theo đúng hướng từ $\mathbf{p}_0$ tới $\mathbf{p}_1$, tức tiếp xúc với cạnh đầu tiên của đa giác điều khiển.
- **Độ lớn:** tốc độ xuất phát là $n\|\mathbf{p}_1 - \mathbf{p}_0\|$. Kéo $\mathbf{p}_1$ ra xa $\mathbf{p}_0$ (giữ nguyên hướng) thì đường cong "bám" theo tiếp tuyến lâu hơn trước khi uốn về phía các điểm còn lại; kéo $\mathbf{p}_1$ sát $\mathbf{p}_0$ thì đường cong gần như uốn ngay. Hệ số n cho thấy cùng một cạnh điều khiển nhưng bậc càng cao thì vận tốc đầu càng lớn.

Ứng dụng. (1) Điều kiện ghép trơn C^1/G^1 ở mục 4.5 được suy ra trực tiếp từ đẳng thức này. (2) Đổi giữa dạng Hermite và Bézier: tiếp tuyến Hermite $\mathbf{D}_0 = 3(\mathbf{p}_1 - \mathbf{p}_0)$ (mục 4.15). (3) Trong phần mềm vẽ, tay nắm (handle) chính là đoạn $\mathbf{p}_0\mathbf{p}_1$; kéo đầu tay nắm là thay đổi trực tiếp vector vận tốc đầu. Lưu ý trường hợp suy biến $\mathbf{p}_1 = \mathbf{p}_0$: khi đó $\mathbf{P}'(0) = \mathbf{0}$ và hướng tiếp tuyến được quyết định bởi $\mathbf{p}_2$.

4.8 Thực hiện các phép biến đổi hình học trên đường cong Bézier đã vẽ

Câu hỏi: Vẽ được đường cong Bézier rồi, nếu muốn thực hiện các phép biến đổi hình học cơ bản (tịnh tiến, quay, tỉ lệ…) thì phải làm sao?

Không biến đổi từng điểm ảnh, mà biến đổi các điểm điều khiển. Nhờ tính bất biến affine (mục 4.9), ảnh của đường cong qua phép affine $\mathbf{x} \mapsto A\mathbf{x} + T$ chính là đường Bézier có điểm điều khiển $A\mathbf{p}_k + T$. Quy trình:

1. Lưu đường cong dưới dạng mảng điểm điều khiển số thực trong WCS (không lưu điểm ảnh).
2. Lập ma trận thuần nhất 3×3 của phép biến đổi (tịnh tiến, quay quanh tâm bất kỳ, tỉ lệ theo tâm, đối xứng…, xem tuần 6) hoặc tích của nhiều phép.

3. Nhân ma trận với $n+1$ điểm điều khiển.
4. Vẽ lại đường cong từ các điểm điều khiển mới bằng một trong các phương pháp ở mục 4.3.

Lý do không biến đổi điểm ảnh: (i) chi phí: chỉ $n+1$ phép nhân ma trận thay vì hàng trăm, hàng nghìn điểm ảnh; (ii) chất lượng: biến đổi điểm ảnh rồi làm tròn lại gây lỗ hổng (khi phóng to) hoặc chồng điểm (khi thu nhỏ, quay), còn vẽ lại từ điểm điều khiển luôn cho đường liên tục lân cận 8; (iii) sai số không tích lũy qua nhiều lần biến đổi nếu luôn biến đổi từ dữ liệu gốc. *Giới hạn:* tính bất biến chỉ đúng với phép affine. Với phép chiếu phối cảnh (tuần 6, mục 5.13), ảnh của một Bézier không còn là Bézier đa thức; khi đó phải dùng Bézier hữu tỉ (rational Bézier/NURBS) với trọng số, hoặc chiếu các điểm đã lấy mẫu (Farin, 2002).

4.9 Vế trái, vế phải trong slide trang 7 (tính bất biến affine) và kết luận

Câu hỏi: Giải thích vế trái và vế phải trong slide trang 7, từ đó rút ra kết luận gì?

Slide 7 (Lý Quốc Ngọc, 2026d) viết:

$$\underbrace{A(\mathbf{P}(u))+T}_{\text{vế trái}} = A\Big(\sum_{k=0}^{n}\mathbf{p}_k BEZ_{k,n}(u)\Big)+T = \sum_{k=0}^{n}A(\mathbf{p}_k)\,BEZ_{k,n}(u)+T\sum_{k=0}^{n}BEZ_{k,n}(u)$$
$$= \underbrace{\sum_{k=0}^{n}\big(A(\mathbf{p}_k)+T\big)BEZ_{k,n}(u)}_{\text{vế phải}}. \tag{49}$$

- **Vế trái:** lấy *từng điểm* $\mathbf{P}(u)$ của đường cong đã dựng xong rồi áp phép biến đổi affine (ma trận tuyến tính A cho quay/tỉ lệ/biến dạng, vector T cho tịnh tiến). Đây là "biến đổi đường cong".
- **Vế phải:** biến đổi trước $n+1$ *điểm điều khiển* thành $A(\mathbf{p}_k)+T$, rồi dựng đường Bézier mới từ các điểm này. Đây là "đường cong của các điểm đã biến đổi".
- **Hai bước biến đổi ở giữa:** bước thứ nhất dùng tính tuyến tính của A (đưa A vào trong tổng: $A(\sum \mathbf{p}_k B_k)=\sum A(\mathbf{p}_k)B_k$). Bước thứ hai dùng tính chất phân hoạch đơn vị ở slide 6, $\sum_k BEZ_{k,n}(u)=[u+(1-u)]^n=1$, để viết $T=T\sum_k BEZ_{k,n}(u)$ rồi gộp T vào từng số hạng. Nếu tổng các hàm cơ sở không bằng 1, phép tịnh tiến sẽ không "chui" vào được và đẳng thức sai.

Kết luận.

1. Đường cong Bézier *bất biến affine*: ảnh affine của một Bézier là Bézier của các điểm điều khiển đã biến đổi. Vì vậy chỉ cần biến đổi $n+1$ điểm rồi vẽ lại (mục 4.8), độc lập với số điểm ảnh của đường cong.
2. Hình dạng đường cong được xác định hoàn toàn bởi đa giác điều khiển, không phụ thuộc hệ tọa độ được chọn để mô tả. Đây là điều mà dạng tường minh $y=f(x)$ không có (mục 4.2).
3. Tính chất đòi hỏi đúng hai điều: phép biến đổi là affine và các hàm cơ sở có tổng bằng 1. Mọi đường cong xây dựng bằng tổ hợp lồi (B-spline, Hermite với điều kiện thích hợp…) đều có tính chất này; phép chiếu phối cảnh không phải affine nên không áp dụng được.

4.10 Phương trình đường cong Bézier bậc 3 và cách vẽ

Câu hỏi: Viết phương trình đường cong Bézier bậc 3 và vẽ minh họa.

Với 4 điểm điều khiển $\mathbf{p}_0,\mathbf{p}_1,\mathbf{p}_2,\mathbf{p}_3$ (slide 9 (Lý Quốc Ngọc, 2026d)):

$$\mathbf{P}(u)=(1-u)^3\mathbf{p}_0+3u(1-u)^2\mathbf{p}_1+3u^2(1-u)\mathbf{p}_2+u^3\mathbf{p}_3,\qquad 0\le u\le 1, \tag{50}$$

hay dạng ma trận

$$\mathbf{P}(u)=\begin{bmatrix}u^3 & u^2 & u & 1\end{bmatrix}\begin{bmatrix}-1 & 3 & -3 & 1\\ 3 & -6 & 3 & 0\\ -3 & 3 & 0 & 0\\ 1 & 0 & 0 & 0\end{bmatrix}\begin{bmatrix}\mathbf{p}_0\\ \mathbf{p}_1\\ \mathbf{p}_2\\ \mathbf{p}_3\end{bmatrix}. \tag{51}$$

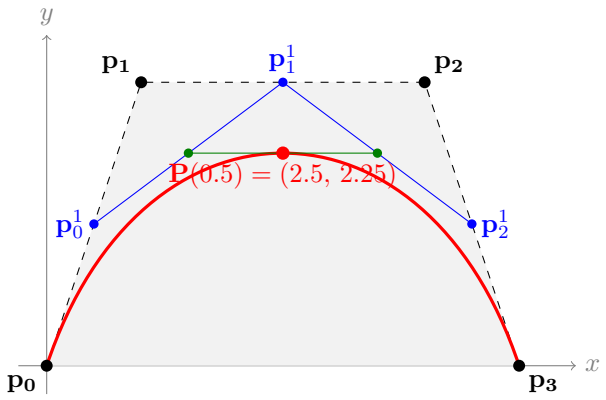
Ví dụ số. Cho $\mathbf{p}_0 = (0,0)$, $\mathbf{p}_1 = (1,3)$, $\mathbf{p}_2 = (4,3)$, $\mathbf{p}_3 = (5,0)$. Khai triển:

$$x(u) = 3u + 6u^2 - 4u^3, \qquad y(u) = 9u - 9u^2. \tag{52}$$

Bảng một số giá trị:

u	0	0.25	0.5	0.75	1
$x(u)$	0	1.0625	2.5	3.9375	5
$y(u)$	0	1.6875	2.25	1.6875	0

Kiểm tra tại $u = 0.5$ bằng De Casteljau: lần chia thứ nhất được $(0.5, 1.5)$, $(2.5, 3)$, $(4.5, 1.5)$; lần thứ hai được $(1.5, 2.25)$, $(3.5, 2.25)$; lần thứ ba được $(2.5, 2.25)$, trùng với kết quả khai triển. Hình 4 vẽ đường cong, đa giác điều khiển và các bước dựng De Casteljau tại $u = 0.5$. Quan sát hình thấy đủ các tính chất ở slide 5–6: đường cong đi qua $\mathbf{p}_0$, $\mathbf{p}_3$, tiếp xúc với $\mathbf{p}_0\mathbf{p}_1$ và $\mathbf{p}_2\mathbf{p}_3$ ở hai đầu, nằm trong bao lồi của 4 điểm và không đi qua $\mathbf{p}_1$, $\mathbf{p}_2$ (đỉnh cao nhất chỉ đạt $y = 2.25 < 3$).



Hình 4: Bézier bậc ba với $\mathbf{p}_0 = (0,0)$, $\mathbf{p}_1 = (1,3)$, $\mathbf{p}_2 = (4,3)$, $\mathbf{p}_3 = (5,0)$: đa giác điều khiển (nét đứt), bao lồi (vùng xám) và ba lần chia De Casteljau tại $u = 0.5$

Vẽ trên màn hình: chuyển các điểm thực sang tọa độ thiết bị rồi dùng một trong ba phương pháp ở mục 4.3. Ví dụ trên lưới nguyên đã được minh họa ở Hình 3 (cùng hình dạng, các điểm điều khiển nhân 4 theo x và theo y).

4.11 Kiểm tra đạo hàm bậc 2 của đường cong Bézier bậc 3

Câu hỏi: Kiểm tra công thức đạo hàm bậc 2 tại hai đầu mút của đường cong Bézier bậc 3 (slide 10).

Đạo hàm bậc nhất. Đạo hàm từng hàm cơ sở:

$$\begin{aligned}\mathbf{P}'(u) &= -3(1-u)^2\mathbf{p}_0 + [3(1-u)^2 - 6u(1-u)]\mathbf{p}_1 + [6u(1-u) - 3u^2]\mathbf{p}_2 + 3u^2\mathbf{p}_3 \\ &= 3(1-u)^2(\mathbf{p}_1 - \mathbf{p}_0) + 6u(1-u)(\mathbf{p}_2 - \mathbf{p}_1) + 3u^2(\mathbf{p}_3 - \mathbf{p}_2).\end{aligned} \tag{53}$$

Suy ra $\mathbf{P}'(0) = 3(\mathbf{p}_1 - \mathbf{p}_0)$, $\mathbf{P}'(1) = 3(\mathbf{p}_3 - \mathbf{p}_2)$, khớp với slide 10.

Đạo hàm bậc hai. Đạo hàm tiếp:

$$\begin{aligned}\mathbf{P}''(u) &= -6(1-u)(\mathbf{p}_1 - \mathbf{p}_0) + (6 - 12u)(\mathbf{p}_2 - \mathbf{p}_1) + 6u(\mathbf{p}_3 - \mathbf{p}_2) \\ &= 6(1-u)(\mathbf{p}_0 - 2\mathbf{p}_1 + \mathbf{p}_2) + 6u(\mathbf{p}_1 - 2\mathbf{p}_2 + \mathbf{p}_3).\end{aligned} \tag{54}$$

Do đó

$$\mathbf{P}''(0) = 6(\mathbf{p}_0 - 2\mathbf{p}_1 + \mathbf{p}_2), \qquad \mathbf{P}''(1) = 6(\mathbf{p}_1 - 2\mathbf{p}_2 + \mathbf{p}_3), \tag{55}$$

khớp với slide 10. Đối chiếu với công thức tổng quát ở slide 6 với $n = 3$: $n(n-1) = 6$ và $[(\mathbf{p}_2 - \mathbf{p}_1) - (\mathbf{p}_1 - \mathbf{p}_0)] = \mathbf{p}_0 - 2\mathbf{p}_1 + \mathbf{p}_2$; ở đầu cuối $[(\mathbf{p}_{n-2} - \mathbf{p}_{n-1}) - (\mathbf{p}_{n-1} - \mathbf{p}_n)] = \mathbf{p}_1 - 2\mathbf{p}_2 + \mathbf{p}_3$. Hai cách tính cho cùng kết quả.

Kiểm tra bằng số với ví dụ ở mục 4.10: $x''(u) = 12 - 24u$, $y''(u) = -18$, nên $\mathbf{P}''(0) = (12, -18)$, $\mathbf{P}''(1) = (-12, -18)$. Theo công thức: $6[(0,0) - 2(1,3) + (4,3)] = 6(2,-3) = (12,-18)$ và $6[(1,3) - 2(4,3) + (5,0)] = 6(-2,-3) = (-12,-18)$. Khớp.

Ý nghĩa. $\mathbf{P}''$ là gia tốc, quyết định độ cong $\kappa = \frac{|x'y''-y'x''|}{(x'^2+y'^2)^{3/2}}$. Độ cong ở đầu $u=0$ chỉ phụ thuộc ba điểm đầu, ở đầu $u=1$ chỉ phụ thuộc ba điểm cuối, nên khi ghép hai đoạn, điều kiện C^2 chỉ liên quan tới ba điểm mỗi bên mối nối. $\mathbf{P}''(u)$ là hàm bậc nhất theo u, nghĩa là "gia tốc" của Bézier bậc ba biến thiên tuyến tính giữa hai giá trị đầu mút.

4.12 Phương trình đoạn thẳng theo tham số

Câu hỏi: Viết phương trình đoạn thẳng theo tham số.

Đoạn thẳng nối $\mathbf{p}_0 = (x_0, y_0)$ và $\mathbf{p}_1 = (x_1, y_1)$:

$$\mathbf{P}(t) = (1-t)\mathbf{p}_0 + t\,\mathbf{p}_1 = \mathbf{p}_0 + t(\mathbf{p}_1 - \mathbf{p}_0), \qquad t \in [0,1], \tag{56}$$

tức là $x(t) = x_0 + t(x_1 - x_0)$, $y(t) = y_0 + t(y_1 - y_0)$. Tham số t là tỉ số chia: điểm $\mathbf{P}(t)$ chia đoạn $\mathbf{p}_0\mathbf{p}_1$ theo tỉ lệ $t : (1-t)$. Cho $t \in \mathbb{R}$ được cả đường thẳng; $t \geq 0$ được tia từ $\mathbf{p}_0$ (dùng trong phép thử tia ở mục 4.16). Đây là Bézier bậc 1 (mục 4.4), và cũng là thao tác "chia đoạn theo tỉ số t" mà De Casteljau lặp lại. So với dạng $y = mx + b$, dạng tham số không gặp vấn đề với đoạn thẳng đứng ($x_1 = x_0$), và giao của hai đoạn được tìm bằng cách giải hệ tuyến tính hai ẩn t, s rồi kiểm tra $t, s \in [0,1]$.

4.13 Chứng minh điểm De Casteljau $\mathbf{p}_0^2(t)$ trùng với đường cong Bézier

Câu hỏi: Hãy chứng minh $\mathbf{p}_0^2(t)$ (của De Casteljau) trùng khớp với đường cong Bézier bậc 2; chứng minh thêm cho trường hợp bậc 3.

Bậc 2 (3 điểm điều khiển). Theo slide 13–14 (Lý Quốc Ngọc, 2026d):

$$\mathbf{p}_0^1(t) = (1-t)\mathbf{p}_0 + t\,\mathbf{p}_1, \qquad \mathbf{p}_1^1(t) = (1-t)\mathbf{p}_1 + t\,\mathbf{p}_2, \tag{57}$$

$$\mathbf{p}_0^2(t) = (1-t)\mathbf{p}_0^1(t) + t\,\mathbf{p}_1^1(t) = (1-t)^2\mathbf{p}_0 + 2t(1-t)\mathbf{p}_1 + t^2\mathbf{p}_2 = \sum_{k=0}^{2} \mathbf{p}_k BEZ_{k,2}(t). \tag{58}$$

Bậc 3 (4 điểm điều khiển). Lần chia thứ hai cho (cùng phép tính như trên, dời chỉ số):

$$\mathbf{p}_0^2(t) = (1-t)^2\mathbf{p}_0 + 2t(1-t)\mathbf{p}_1 + t^2\mathbf{p}_2, \qquad \mathbf{p}_1^2(t) = (1-t)^2\mathbf{p}_1 + 2t(1-t)\mathbf{p}_2 + t^2\mathbf{p}_3. \tag{59}$$

Lần chia thứ ba:

$$\begin{aligned}\mathbf{p}_0^3(t) &= (1-t)\,\mathbf{p}_0^2(t) + t\,\mathbf{p}_1^2(t) \\ &= (1-t)^3\mathbf{p}_0 + [2t(1-t)^2 + t(1-t)^2]\mathbf{p}_1 + [t^2(1-t) + 2t^2(1-t)]\mathbf{p}_2 + t^3\mathbf{p}_3 \\ &= (1-t)^3\mathbf{p}_0 + 3t(1-t)^2\mathbf{p}_1 + 3t^2(1-t)\mathbf{p}_2 + t^3\mathbf{p}_3 = \sum_{k=0}^{3} \mathbf{p}_k BEZ_{k,3}(t).\end{aligned} \tag{60}$$

Lưu ý với 4 điểm, $\mathbf{p}_0^2(t)$ chỉ là Bézier bậc 2 của $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$; điểm nằm trên đường bậc 3 là $\mathbf{p}_0^3(t)$.

Tổng quát (quy nạp). Mệnh đề: với mọi r và i,

$$\mathbf{p}_i^r(t) = \sum_{j=0}^{r} \mathbf{p}_{i+j}\, BEZ_{j,r}(t). \tag{61}$$

Với $r = 0$ đúng vì $\mathbf{p}_i^0 = \mathbf{p}_i$. Giả sử đúng với $r-1$. Thay vào $\mathbf{p}_i^r = (1-t)\mathbf{p}_i^{r-1} + t\,\mathbf{p}_{i+1}^{r-1}$ (slide 18):

$$\begin{aligned}\mathbf{p}_i^r(t) &= \sum_{j=0}^{r-1} \mathbf{p}_{i+j}(1-t)BEZ_{j,r-1}(t) + \sum_{j=1}^{r} \mathbf{p}_{i+j}\, t\, BEZ_{j-1,r-1}(t) \\ &= \sum_{j=0}^{r} \mathbf{p}_{i+j}\big[(1-t)BEZ_{j,r-1}(t) + t\, BEZ_{j-1,r-1}(t)\big] \qquad (BEZ_{r,r-1} = BEZ_{-1,r-1} = 0).\end{aligned} \tag{62}$$

Biểu thức trong ngoặc chính là công thức truy hồi (4) ở slide 4, bằng $BEZ_{j,r}(t)$. Lấy $i = 0$, $r = n$ được $\mathbf{p}_0^n(t) = \mathbf{P}(t)$. Như vậy De Casteljau và công thức Bernstein là hai cách tính cùng một đường cong: De Casteljau chỉ dùng phép lerp (tổ hợp lồi) nên ổn định số học, còn công thức Bernstein thuận tiện cho chứng minh tính chất.

4.14 Thiết kế giao diện người dùng để vẽ đường cong Bézier bậc 3

Câu hỏi: Giải thích cách tạo giao diện người dùng (user interface) để vẽ đường cong Bézier bậc 3.

Một giao diện vẽ Bézier là một hệ thống đồ họa nhỏ gồm ba phần: **cấu trúc dữ liệu**, **xử lý sự kiện** và **vòng vẽ lại**.

1. Cấu trúc dữ liệu.

- `struct Point { double x, y; }` – điểm điều khiển lưu bằng số thực trong WCS, không lưu điểm ảnh.
- `struct CubicSegment { int idx[4]; }` – chỉ số 4 điểm điều khiển trong một mảng điểm chung; hai đoạn liền nhau dùng chung điểm mút, nên kéo điểm nối sẽ cập nhật cả hai đoạn (bảo đảm C^0).
- `struct Path { vector<Point> pts; vector<CubicSegment> segs; vector<int> jointType; }` – `jointType` ghi kiểu nối tại mỗi điểm neo (gấp góc, G^1 hay C^1).
- Trạng thái tương tác: chế độ hiện tại (thêm điểm/sửa/chọn), chỉ số điểm đang được kéo, cặp cửa sổ–khung nhìn (window–viewport) hiện tại.

2. Xử lý sự kiện chuột. Tọa độ chuột nằm trong DCS nên trước hết phải đổi sang WCS bằng phép biến đổi khung nhìn – cửa sổ (viewport-to-window, mục 5.9).

- *Thêm đường:* ở chế độ tạo, mỗi lần nhấp thêm một điểm; đủ 4 điểm thì tạo một `CubicSegment`. Các đoạn sau chỉ cần thêm 3 điểm (điểm đầu là điểm cuối của đoạn trước). Cách phổ biến hơn trong phần mềm vẽ (Pen tool): nhấp để đặt điểm neo, nhấn–kéo để kéo tay nắm $\mathbf{p}_1/\mathbf{p}_2$.
- *Chọn điểm (hit-test):* khi nhấn chuột tại $\mathbf{m}$, tìm điểm điều khiển gần nhất với $\|\mathbf{p}_k - \mathbf{m}\| \le \varepsilon$ (ε khoảng 5 pixel, đổi sang WCS). Để chọn chính đường cong, tìm u có $\|\mathbf{P}(u) - \mathbf{m}\|$ nhỏ nhất trên các mẫu đã tính.
- *Kéo điểm:* khi di chuột với điểm đang chọn, gán $\mathbf{p}_k \leftarrow \mathbf{m}$. Nếu điểm neo có kiểu G^1/C^1, cập nhật luôn tay nắm đối diện: với C^1 thì $\mathbf{q}_1 = 2\mathbf{p}_3 - \mathbf{p}_2$ (đối xứng tâm); với G^1 thì giữ độ dài cũ của $\mathbf{q}_1 - \mathbf{q}_0$ nhưng đổi hướng thành ngược hướng $\mathbf{p}_2 - \mathbf{p}_3$. Kéo điểm neo thì tịnh tiến cả hai tay nắm đi kèm.
- *Phóng to, cuộn:* thay đổi cửa sổ (zoom, pan ở tuần 6), không thay đổi điểm điều khiển.

3. Vòng vẽ lại. Sau mỗi sự kiện làm thay đổi dữ liệu: xóa vùng vẽ, với mỗi đoạn tính các điểm bằng De Casteljau hoặc sai phân tiến, đổi sang DCS bằng window-to-viewport, nối bằng Bresenham; sau đó vẽ đa giác điều khiển (nét đứt), tay nắm (đoạn $\mathbf{p}_0\mathbf{p}_1$, $\mathbf{p}_2\mathbf{p}_3$) và các ô vuông nhỏ đánh dấu điểm điều khiển, tô màu khác cho điểm đang chọn. Để mượt khi kéo, có thể chỉ vẽ lại các đoạn chứa điểm vừa thay đổi (nhờ tính cục bộ của đường ghép) và dùng bộ đệm kép (double buffering) để tránh nhấp nháy.

4.15 Nhận xét đường spline Hermite so với đường cong Bézier bậc 3 và so sánh tính chất của hai phương trình

Câu hỏi: Nhận xét đường spline Hermite so với đường cong Bézier bậc 3, và so sánh các tính chất của hai phương trình đường cong.

Đường Hermite bậc 3 (Hearn et al., 2011) được xác định bởi hai điểm mút $\mathbf{P}_0, \mathbf{P}_1$ và hai vector tiếp tuyến $\mathbf{D}_0, \mathbf{D}_1$:

$$\mathbf{P}_H(u) = \mathbf{P}_0 H_0(u) + \mathbf{P}_1 H_1(u) + \mathbf{D}_0 H_2(u) + \mathbf{D}_1 H_3(u), \tag{63}$$

$$H_0 = 2u^3 - 3u^2 + 1, \quad H_1 = -2u^3 + 3u^2, \quad H_2 = u^3 - 2u^2 + u, \quad H_3 = u^3 - u^2. \tag{64}$$

Quan hệ. Hai dạng biểu diễn cùng một tập đường cong bậc ba. Từ $\mathbf{P}'(0) = 3(\mathbf{p}_1 - \mathbf{p}_0)$, $\mathbf{P}'(1) = 3(\mathbf{p}_3 - \mathbf{p}_2)$ suy ra

$$\mathbf{P}_0 = \mathbf{p}_0, \quad \mathbf{P}_1 = \mathbf{p}_3, \quad \mathbf{D}_0 = 3(\mathbf{p}_1 - \mathbf{p}_0), \quad \mathbf{D}_1 = 3(\mathbf{p}_3 - \mathbf{p}_2), \tag{65}$$

Giải thuật 2 Xử lý kéo một điểm điều khiển trong giao diện Bézier bậc 3

Đầu vào: Sự kiện chuột tại (x_v, y_v) trong DCS, đường `path`, chỉ số `sel` của điểm đang chọn (hoặc -1)
Đầu ra: Đường cong được cập nhật và vẽ lại

$\mathbf{m} \leftarrow$ VIEWPORTTOWINDOW(x_v, y_v)
if nhấn chuột **then**
  `sel` $\leftarrow \arg\min_k \|$`path.pts`$[k] - \mathbf{m}\|$ nếu khoảng cách $\leq \varepsilon$, ngược lại -1
else if di chuột **and** `sel` $\neq -1$ **then**
  $\boldsymbol{\delta} \leftarrow \mathbf{m} -$ `path.pts[sel]`; `path.pts[sel]` $\leftarrow \mathbf{m}$
  if `sel` là điểm neo **then** tịnh tiến hai tay nắm kề nó thêm $\boldsymbol{\delta}$
  else if `sel` là tay nắm **and** điểm neo kề có kiểu C^1/G^1 **then** cập nhật tay nắm đối diện
  end if
  REDRAW(các đoạn chứa `sel` và đoạn kề)
else if nhả chuột **then**
  `sel` $\leftarrow -1$
end if

ngược lại $\mathbf{p}_1 = \mathbf{P}_0 + \mathbf{D}_0/3$, $\mathbf{p}_2 = \mathbf{P}_1 - \mathbf{D}_1/3$. Ở dạng ma trận, hai phương trình chỉ khác nhau ở ma trận hệ số và vector dữ liệu:

$$\mathbf{P}_H(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{D}_0 \\ \mathbf{D}_1 \end{bmatrix}, \tag{66}$$

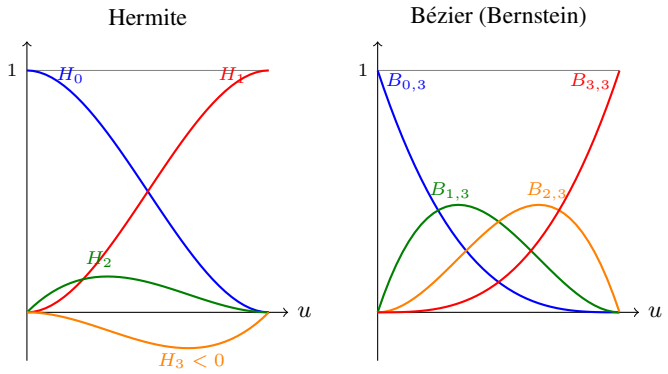
$$\mathbf{P}_B(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} M_B \begin{bmatrix} \mathbf{p}_0 \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \end{bmatrix}, \tag{67}$$

với M_B là ma trận Bézier ở mục 4.10.

So sánh tính chất của hai phương trình. Lấy các tính chất (a)–(f) của Bézier trên slide 5–8 (Lý Quốc Ngọc, 2026d) làm chuẩn và kiểm tra lần lượt trên phương trình Hermite:

Tính chất	**Hermite $\mathbf{P}_H(u)$**	**Bézier bậc 3 $\mathbf{P}_B(u)$**
Dữ liệu xác định đường cong	2 điểm $\mathbf{P}_0, \mathbf{P}_1$ và 2 vector $\mathbf{D}_0, \mathbf{D}_1$	4 điểm $\mathbf{p}_0, \ldots, \mathbf{p}_3$
(a) Đi qua hai đầu mút	Có: $\mathbf{P}_H(0) = \mathbf{P}_0$, $\mathbf{P}_H(1) = \mathbf{P}_1$	Có: $\mathbf{P}_B(0) = \mathbf{p}_0$, $\mathbf{P}_B(1) = \mathbf{p}_3$; không đi qua $\mathbf{p}_1, \mathbf{p}_2$
(b) Tiếp tuyến tại hai đầu	Cho trực tiếp: $\mathbf{P}'_H(0) = \mathbf{D}_0$, $\mathbf{P}'_H(1) = \mathbf{D}_1$	Suy ra từ điểm: $3(\mathbf{p}_1 - \mathbf{p}_0), 3(\mathbf{p}_3 - \mathbf{p}_2)$
(c) Đạo hàm bậc hai tại đầu mút	$\mathbf{P}''_H(0) = 6(\mathbf{P}_1 - \mathbf{P}_0) - 4\mathbf{D}_0 - 2\mathbf{D}_1$, $\mathbf{P}''_H(1) = -6(\mathbf{P}_1 - \mathbf{P}_0) + 2\mathbf{D}_0 + 4\mathbf{D}_1$: phụ thuộc cả 4 dữ liệu	$\mathbf{P}''_B(0) = 6(\mathbf{p}_0 - 2\mathbf{p}_1 + \mathbf{p}_2)$: chỉ phụ thuộc 3 điểm đầu (tương tự ở đầu cuối)
(d) Bao lồi	Không có: $\mathbf{D}_0, \mathbf{D}_1$ là vector, không có "đa giác" bao; $H_3 < 0$ nên hàm cơ sở không phải trọng số lồi	Có: đường cong nằm trong bao lồi của 4 điểm (các $B_{k,3} \geq 0$, tổng bằng 1)
(e) Bất biến affine	Có, nhưng phải biến đổi đúng loại: điểm $\mathbf{P}_i \mapsto A\mathbf{P}_i + T$, vector $\mathbf{D}_i \mapsto A\mathbf{D}_i$ (không tịnh tiến), vì $H_0 + H_1 = 1$	Có: chỉ cần biến đổi 4 điểm $\mathbf{p}_k \mapsto A\mathbf{p}_k + T$
(f) Suy biến thành đoạn thẳng	Khi $\mathbf{D}_0, \mathbf{D}_1$ cùng phương với $\mathbf{P}_1 - \mathbf{P}_0$	Khi 4 điểm thẳng hàng
Nội suy hay xấp xỉ	Nội suy: đi qua mọi điểm dữ liệu	Xấp xỉ: chỉ đi qua 2 trong 4 điểm
Mở rộng bậc	Cố định bậc 3 (bậc cao hơn cần thêm đạo hàm bậc cao)	Tổng quát cho bậc n bất kỳ với $n + 1$ điểm
Thuật toán tính	Đa thức, sai phân tiến	Thêm De Casteljau và chia nhỏ (mục 4.13)

Kiểm tra (c): thay $\mathbf{D}_0 = 3(\mathbf{p}_1 - \mathbf{p}_0)$, $\mathbf{D}_1 = 3(\mathbf{p}_3 - \mathbf{p}_2)$, $\mathbf{P}_1 - \mathbf{P}_0 = \mathbf{p}_3 - \mathbf{p}_0$ vào công thức Hermite được $6(\mathbf{p}_3 - \mathbf{p}_0) - 12(\mathbf{p}_1 - \mathbf{p}_0) - 6(\mathbf{p}_3 - \mathbf{p}_2) = 6(\mathbf{p}_0 - 2\mathbf{p}_1 + \mathbf{p}_2)$, trùng với Bézier. Vậy hai phương trình cho cùng đường cong, chỉ khác cách "đặt tham số điều khiển".



Hình 5: Hàm trộn của Hermite (trái) và của Bézier bậc ba (phải) trên $[0, 1]$

So sánh các hàm cơ sở (hàm trộn) của hai phương trình (Hình 5), là nguồn gốc của các khác biệt ở bảng trên:

Tiêu chí	**Hermite** $H_0..H_3$	**Bernstein** $B_{0,3}..B_{3,3}$
Dấu	$H_3 \leq 0$ trên $[0,1]$; H_2, H_3 không phải trọng số mà nhân với vector	Cả bốn hàm ≥ 0
Tổng	$H_0 + H_1 = 1$; H_2, H_3 nằm ngoài tổng này	$\sum_k B_{k,3} = 1$ (phân hoạch đơn vị)
Giá trị đầu mút	$H_0(0) = 1$, $H_1(1) = 1$, còn lại bằng 0; $H_2'(0) = 1$, $H_3'(1) = 1$	$B_{0,3}(0) = 1$, $B_{3,3}(1) = 1$; $B_{1,3}, B_{2,3}$ đạt cực đại $4/9$ tại $u = 1/3, 2/3$
Hệ quả hình học	Không có tính bao lồi; đường cong có thể vượt xa $\mathbf{P}_0, \mathbf{P}_1$ khi $\|\mathbf{D}\|$ lớn	Có tính bao lồi, đường cong nằm trong đa giác điều khiển; bất biến affine
Đối xứng	$H_1(u) = H_0(1-u)$, $H_3(u) = -H_2(1-u)$	$B_{3-k,3}(u) = B_{k,3}(1-u)$

Nhận xét.

- *Dữ liệu đầu vào:* Hermite cần vector tiếp tuyến (hướng và độ lớn), là đại lượng trừu tượng, khó nhập bằng chuột; Bézier thay vector bằng điểm $\mathbf{p}_1, \mathbf{p}_2$ đặt được trực tiếp trên màn hình, thuận tiện cho giao diện (mục 4.14).
- *Khi nào dùng Hermite:* khi dữ liệu có sẵn đạo hàm, ví dụ nội suy quỹ đạo chuyển động có vận tốc tại các khung hình khóa trong hoạt hình, hoặc khi cần đường cong đi qua *mọi* điểm dữ liệu. Spline Cardinal và Catmull–Rom là Hermite với tiếp tuyến tự tính từ các điểm lân cận, $\mathbf{D}_k = \frac{1}{2}(\mathbf{P}_{k+1} - \mathbf{P}_{k-1})$ (Catmull–Rom).
- *Ghép nối:* Hermite ghép C^1 rất tự nhiên (dùng chung $\mathbf{P}$ và $\mathbf{D}$ tại mối nối); Bézier cần điều kiện thẳng hàng như mục 4.5.
- *Vẽ:* Hermite thường được đổi sang Bézier để dùng De Casteljau, chia nhỏ và kiểm tra bao lồi. Vì vậy các định dạng như SVG chỉ lưu Bézier.

4.16 Xác định một điểm nằm trong hay ngoài đa giác (góc độ máy tính)

Câu hỏi: Làm sao để máy tính xác định một điểm nằm bên trong hay bên ngoài một đa giác? (định lý Jordan)

Đa giác được lưu dưới dạng dãy đỉnh $V_0, V_1, \ldots, V_{m-1}$ (cạnh V_iV_{i+1}, chỉ số lấy theo mod m). Có ba cách tiêu chuẩn (Hearn et al., 2011; Hughes et al., 2014).

1. Phép thử tia – quy tắc chẵn lẻ (odd-even / ray casting) (Shimrat, 1962). Từ điểm $Q(x_q, y_q)$ phóng tia nằm ngang về phía $+x$, đếm số cạnh bị tia cắt. Số lẻ thì Q ở trong, số chẵn thì Q ở ngoài. Cơ sở là định lý Jordan (mục 4.17). Cài đặt cần xử lý trường hợp tia đi qua đỉnh. Cách gọn là quy ước *nửa mở*: cạnh V_iV_j chỉ được tính khi $(y_i > y_q) \neq (y_j > y_q)$, tức là một đầu ở trên tia, đầu kia ở dưới hoặc trên tia. Với quy ước này, đỉnh không cực trị được đếm đúng một lần và đỉnh cực trị được đếm 0 hoặc 2 lần, đúng như phân tích ở mục 4.20.

Giải thuật 3 Kiểm tra điểm trong đa giác bằng quy tắc chẵn lẻ

Đầu vào: Đỉnh $V_0, \ldots, V_{m-1}$, điểm $Q(x_q, y_q)$
Đầu ra: **true** nếu Q nằm trong đa giác

```
inside ← false;   j ← m − 1
for i ← 0 to m − 1 do
    if (y_i > y_q) ≠ (y_j > y_q) then                ▷ cạnh V_jV_i cắt đường y = y_q (nửa mở)
        x_int ← x_i + (y_q − y_i) (x_j − x_i)/(y_j − y_i)
        if x_q < x_int then inside ← ¬inside
        end if
    end if
    j ← i
end for
return inside
```

2. Số vòng quấn (winding number, non-zero rule). Đi dọc biên đa giác theo thứ tự đỉnh, mỗi cạnh cắt tia đi lên được cộng $+1$, đi xuống được cộng -1. Q ở trong khi tổng khác 0. Với đa giác đơn, kết quả trùng quy tắc chẵn lẻ; với đa giác tự cắt (ngôi sao 5 cánh vẽ một nét), vùng ngũ giác giữa có số vòng quấn 2: quy tắc khác-không coi là trong, quy tắc chẵn lẻ coi là ngoài. SVG cho chọn cả hai qua thuộc tính `fill-rule` (`nonzero` / `evenodd`) (W3C, 2026).

3. Đa giác lồi – dấu tích có hướng. Nếu đa giác lồi và các đỉnh theo chiều ngược kim đồng hồ, Q ở trong khi và chỉ khi Q nằm bên trái mọi cạnh: $(V_{i+1} - V_i) \times (Q - V_i) \geq 0$ với mọi i. Phép thử chỉ cần nhân và cộng, không chia, và là nền tảng của việc rời rạc hóa tam giác trên GPU (hàm cạnh – edge function).

Ứng dụng. Chọn đối tượng bằng chuột, cắt xén, tô màu, phát hiện va chạm. Trong học máy, mặt nạ phân đoạn (segmentation mask) thường được lưu dạng đa giác (như định dạng COCO) và được rời rạc hóa thành ảnh nhị phân bằng chính phép thử này hoặc thuật toán dòng quét ở các mục sau.

4.17 Định lý Jordan và cách chứng minh (trường hợp đa giác)

Câu hỏi: Phát biểu và nêu cách chứng minh định lý Jordan.

Phát biểu. Mọi đường cong đơn (không tự cắt), liên tục, khép kín C trong mặt phẳng chia $\mathbb{R}^2 \setminus C$ thành đúng hai miền liên thông: một miền bị chặn (miền trong) và một miền không bị chặn (miền ngoài), và C là biên của cả hai miền. Với đường cong liên tục tổng quát, chứng minh khó (cần tô-pô đại số; Hales đã kiểm chứng hình thức một chứng minh đầy đủ (Hales, 2007)). Trong đồ họa, biên luôn là *đa giác*, và với đa giác có chứng minh sơ cấp như sau.

Chứng minh cho đa giác đơn C. Chọn hướng tia cố định (nằm ngang về phía $+x$). Với $Q \notin C$, định nghĩa $\pi(Q)$ là tính chẵn lẻ của số lần tia từ Q cắt C, trong đó tia đi qua đỉnh được đếm theo quy ước của mục 4.20: đỉnh cực trị đếm 0 (hoặc 2), đỉnh không cực trị đếm 1; cạnh nằm ngang nằm trên tia được bỏ qua.

Bước 1 – π không đổi trên mỗi miền liên thông. Cho Q di chuyển liên tục mà không chạm C. Số giao điểm chỉ có thể thay đổi khi tia quét qua một đỉnh. Nếu đó là đỉnh cực trị, hai cạnh kề cùng xuất hiện hoặc cùng biến mất trên tia, số giao điểm thay đổi ± 2; nếu là đỉnh không cực trị, một cạnh thay chỗ cạnh kia, số giao điểm không đổi. Tính chẵn lẻ vì vậy bất biến, và π không đổi trên mọi đường liên tục không cắt C, tức là trên mỗi miền liên thông.

Bước 2 – có ít nhất hai miền. Lấy hai điểm Q_1, Q_2 rất gần một điểm trong của một cạnh e, ở hai phía của e và sao cho đoạn Q_1Q_2 cắt e. Tia từ hai điểm gần như trùng nhau, chỉ khác là đúng một trong hai tia cắt e. Do đó $\pi(Q_1) \neq \pi(Q_2)$ và theo bước 1 chúng thuộc hai miền khác nhau. Điểm ở

rất xa có tia không cắt C nên $\pi = 0$; miền chứa nó là miền ngoài không bị chặn; miền có $\pi = 1$ nằm trong hình chữ nhật bao của C nên bị chặn.

Bước 3 – có nhiều nhất hai miền. Lấy một dải hẹp bao quanh C (đủ hẹp để không chạm các cạnh không kề nhau). Dải này bị C chia thành hai "làn" chạy song song hai bên đa giác, mỗi làn liên thông vì có thể đi dọc theo cạnh rồi vòng qua các đỉnh. Từ một điểm Q bất kỳ không thuộc C, đi thẳng tới điểm gần nhất của C thì sẽ vào một trong hai làn trước khi chạm C. Vậy mọi điểm liên thông với một trong hai làn, nên có không quá hai miền.

Kết hợp ba bước: $\mathbb{R}^2 \setminus C$ có đúng hai miền, phân biệt bởi π. Đây cũng chính là lý do phép thử tia ở mục 4.16 đúng: Q ở trong khi và chỉ khi $\pi(Q) = 1$, bất kể chọn hướng tia nào.

4.18 Vì sao lấp đầy các cặp giao điểm lẻ–chẵn ở bước L4 thì lấp được miền trong

Câu hỏi: Tại sao ở bước L4 (tô các đoạn nằm giữa từng cặp giao điểm thứ lẻ và thứ chẵn) thì ta lấp đầy được miền trong của đa giác? Liên hệ với định lý Jordan.

Phương pháp tô màu theo dòng quét trên slide (mục 7.3.2) (Lý Quốc Ngọc, 2026e) giả thiết dòng quét nằm ngang, hai dòng cách nhau 1 điểm ảnh, và *miền trong được xác định theo định lý Jordan*:

L1. Với mỗi dòng quét;

L2. xác định giao điểm của dòng quét với các cạnh của S;

L3. sắp xếp các giao điểm theo x tăng dần;

L4. tô các điểm ảnh nằm giữa từng cặp giao điểm lẻ–chẵn.

Tại dòng quét y, gọi các giao điểm với biên (đã xử lý đỉnh như mục 4.20) sau khi sắp xếp là $x_1 \le x_2 \le \cdots \le x_{2k}$. Số giao điểm luôn chẵn, vì đi từ $x = -\infty$ (miền ngoài) tới $x = +\infty$ (miền ngoài) thì số lần vượt biên phải chẵn.

Xét một điểm $Q(x, y)$ với $x_j < x < x_{j+1}$. Tia nằm ngang từ Q về phía $+x$ cắt biên đúng tại $x_{j+1}, \ldots, x_{2k}$, tức là $2k - j$ lần. Theo phép thử tia (hệ quả của định lý Jordan, mục 4.17):

$$Q \text{ ở trong} \iff 2k - j \text{ lẻ} \iff j \text{ lẻ}. \tag{68}$$

Vậy các khoảng $(x_1, x_2), (x_3, x_4), \ldots$ (từ giao điểm thứ lẻ đến giao điểm thứ chẵn kế tiếp) nằm trong đa giác, còn các khoảng $(x_2, x_3), (x_4, x_5), \ldots$ và hai phía ngoài cùng nằm ngoài. Nói cách khác, khi đi dọc dòng quét từ trái sang phải, mỗi giao điểm là một lần "vượt biên" đổi trạng thái ngoài $\to$ trong $\to$ ngoài; bước L4 chỉ là áp dụng phép thử tia cho *tất cả* điểm ảnh của dòng cùng lúc, với chi phí $O(k)$ thay vì kiểm tra từng điểm. Lặp lại cho mọi dòng quét từ $y_{\min}$ đến $y_{\max}$ là lấp kín miền trong. Điều kiện để lập luận đúng là mỗi lần vượt biên chỉ được đếm đúng một giao điểm, và đó là lý do phải xử lý đỉnh (mục 4.20).

4.19 Trường hợp dòng quét $i = 7$ trong ví dụ của slide

Câu hỏi: Trong ví dụ tô màu đa giác theo dòng quét (slide 7.3.4), nếu dòng quét $i = 7$ thì gặp trường hợp gì?

Dữ liệu ví dụ (Lý Quốc Ngọc, 2026e). Đa giác S có 7 đỉnh $P_1(6, 8)$, $P_2(9, 5)$, $P_3(9, 1)$, $P_4(5, 5)$, $P_5(1, 2)$, $P_6(2, 7)$, $P_7(4, 8)$ và các cạnh $e_1 = P_1P_2$, $e_2 = P_2P_3$, …, $e_7 = P_7P_1$ (Hình 6). Trên slide, hai dòng quét được tô đỏ là $y = 5$ và $y = 7$: đó là hai dòng đi qua đỉnh và rơi vào các điểm yếu W1, W2 của phương pháp.

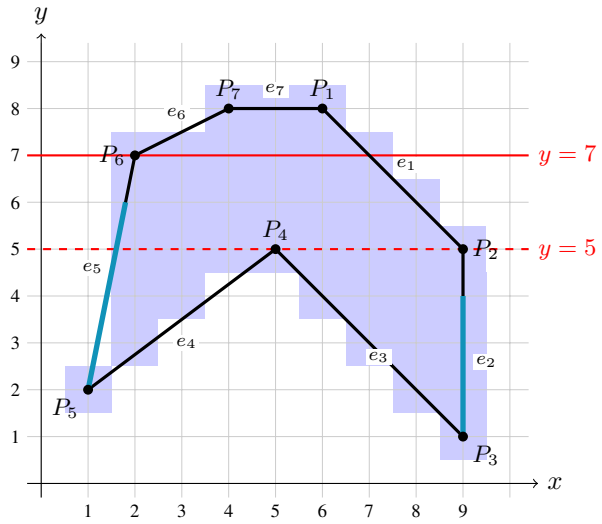
Dòng quét $y = 7$ đi qua đỉnh $P_6(2, 7)$, gặp điểm yếu W2. P_6 nằm giữa $P_5(1, 2)$ và $P_7(4, 8)$, có $\Delta y_6 = 7 - 2 = 5 > 0$ và $\Delta y_7 = 8 - 7 = 1 > 0$, nên $\Delta y_6 \cdot \Delta y_7 > 0$: P_6 là **đỉnh không cực trị**, thuộc tình huống *biên đi lên*. Dòng quét thực sự đi xuyên qua biên tại P_6 (cạnh e_5 ở dưới, cạnh e_6 ở trên).

- *Nếu không xử lý:* cả e_5 và e_6 đều cắt dòng 7 tại $x = 2$, cùng với e_1 cắt tại $x = 7$, được 3 giao điểm $\{2, 2, 7\}$. Ghép cặp lẻ–chẵn cho đoạn $[2, 2]$ và giao điểm 7 không có cặp: đoạn $(2, 7]$ nằm trong đa giác bị bỏ trống.

- *Sau khi tinh chỉnh dữ liệu (C2):* cạnh e_5 được rút ngắn từ $(1,2) \to (2,7)$ thành $(1,2) \to (2,6)$, tức $\texttt{y_upper} = 6$, nên e_5 đã bị loại khỏi AEL ở dòng 6. Tại dòng 7, ET[7] thêm e_6 vào AEL; AEL sau khi sắp xếp là e_6 ($\texttt{x_int} = 2$), e_1 ($\texttt{x_int} = 7$). Đúng 2 giao điểm, tô đoạn $[2, 7]$ (mũi tên đỏ trên slide). Sau đó cập nhật e_6: $x = 2 + 2 = 4$, e_1: $x = 7 - 1 = 6$, cho dòng 8 đoạn $[4, 6]$.

Cạnh nằm ngang ở dòng 8. Ngay phía trên, dòng 8 chứa cạnh nằm ngang $e_7 = P_7P_1$ (điểm yếu W1). Cạnh này đã bị loại khỏi dữ liệu; hai đầu P_7, P_1 cùng với e_6, e_1 tạo một đỉnh cực đại "phẳng" nên không rút ngắn, và dòng 8 được tô bằng cặp (e_6, e_1) là $[4, 6]$, phủ đúng cạnh e_7.

So sánh với dòng $y = 5$ (dòng đỏ còn lại trên slide): dòng này đi qua hai đỉnh cùng lúc. $P_2(9, 5)$ có $\Delta y = 5 - 8 < 0$ và $1 - 5 < 0$: không cực trị, biên đi xuống, nên e_2 được rút ngắn thành $(9, 4) \to (9, 1)$ và tại dòng 5 chỉ còn e_1 cho giao điểm $x = 9$. $P_4(5, 5)$ có $\Delta y = 5 - 1 > 0$ và $2 - 5 < 0$: là *đỉnh cực đại*, giữ nguyên hai giao điểm $x = 5$ của e_3 và e_4. AEL dòng 5 cho $\{1.6, 5, 5, 9\}$, tô $[2, 5]$ và $[5, 9]$, tức là liền một đoạn $[2, 9]$, đúng vì P_4 nằm bên trong miền tô ở dòng đó. Như vậy dòng 5 minh họa đủ hai loại đỉnh, còn dòng 7 minh họa đỉnh không cực trị đi lên.



Hình 6: Ví dụ trên slide 7.3.4: đa giác $P_1 \dots P_7$, các điểm ảnh được tô (ô xanh, với đoạn $[\lceil x_{2j-1} \rceil, \lfloor x_{2j} \rfloor]$), hai cạnh sau khi rút ngắn e_5, e_2 (nét xanh đậm) và hai dòng quét đặc biệt $y = 5$, $y = 7$

Toàn bộ quá trình tô theo giải thuật ET–AEL (mục 4.22) với bảng cạnh ET[1] $= \{e_2 : (4, 9, 0),\ e_3 : (5, 9, -1)\}$, ET[2] $= \{e_4 : (5, 1, \frac{4}{3}),\ e_5 : (6, 1, \frac{1}{5})\}$, ET[5] $= \{e_1 : (8, 9, -1)\}$, ET[7] $= \{e_6 : (8, 2, 2)\}$ (bộ ba $\texttt{y_upper}$, $\texttt{x_int}$, $\texttt{reci_slope}$, khớp với slide):

y	AEL đã sắp xếp (`x_int`)	Đoạn tô	Ghi chú
1	e_2: 9, e_3: 9	$[9, 9]$	P_3 cực tiểu: 2 giao điểm trùng
2	e_4: 1, e_5: 1, e_3: 8, e_2: 9	$[1, 1], [8, 9]$	P_5 cực tiểu
3	e_5: $\frac{6}{5}$, e_4: $\frac{7}{3}$, e_3: 7, e_2: 9	$[2, 2], [7, 9]$	
4	e_5: $\frac{7}{5}$, e_4: $\frac{11}{3}$, e_3: 6, e_2: 9	$[2, 3], [6, 9]$	xóa e_2 ($\texttt{y_upper} = 4$)
5	e_5: $\frac{8}{5}$, e_4: 5, e_3: 5, e_1: 9	$[2, 5], [5, 9]$	thêm e_1; xóa e_3, e_4
6	e_5: $\frac{9}{5}$, e_1: 8	$[2, 8]$	xóa e_5 (đã rút ngắn)
7	e_6: 2, e_1: 7	$[2, 7]$	thêm e_6; P_6 chỉ 1 giao điểm
8	e_6: 4, e_1: 6	$[4, 6]$	xóa e_6, e_1; AEL rỗng, dừng

4.20 Rút ngắn cạnh tại đỉnh không cực trị

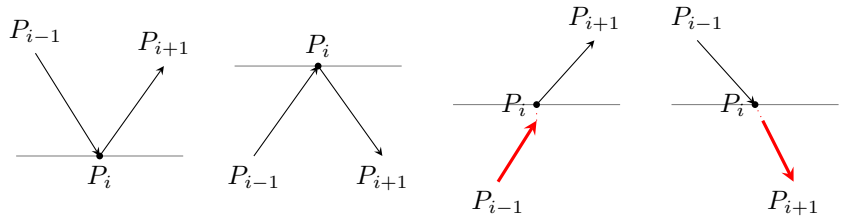
Câu hỏi: Nêu cách rút ngắn cạnh trong trường hợp đỉnh không cực trị (chia ra 2 tình huống). Rút ngắn để làm gì?

Theo hình vẽ trên bảng, với đỉnh P_i và hai đỉnh kề P_{i-1}, P_{i+1}, đặt $\Delta y_i = y_i - y_{i-1}$, $\Delta y_{i+1} = y_{i+1} - y_i$. Đỉnh P_i *không cực trị* khi $\Delta y_i \cdot \Delta y_{i+1} > 0$, gồm hai tình huống (Hình 7):

1. **Biên đi lên** ($\Delta y_i > 0$ và $\Delta y_{i+1} > 0$): P_{i-1} ở dưới, P_{i+1} ở trên. Cạnh nằm dưới đỉnh là $P_{i-1}P_i$ (cạnh đang xét). Rút ngắn đầu trên của nó một đơn vị: $y_{\text{upper}}(P_{i-1}P_i) = y_i - 1$.
2. **Biên đi xuống** ($\Delta y_i < 0$ và $\Delta y_{i+1} < 0$): P_{i-1} ở trên, P_{i+1} ở dưới. Cạnh nằm dưới đỉnh là P_iP_{i+1} (cạnh kế tiếp). Rút ngắn đầu trên của nó: $y_{\text{upper}}(P_iP_{i+1}) = y_i - 1$.

Trong cả hai tình huống ta rút ngắn *cạnh nằm dưới đỉnh* ở đầu trên của nó, đúng như quy ước của Hearn & Baker: duyệt cạnh theo thứ tự quanh đa giác, nếu tung độ tăng đơn điệu thì giảm đầu trên của cạnh hiện tại, nếu giảm đơn điệu thì giảm đầu trên của cạnh kế tiếp (Hearn et al., 2011). Cách tương đương là tăng đầu dưới của cạnh nằm trên đỉnh thêm 1 (khi đó phải cập nhật luôn x tại đầu dưới: $x \mathrel{+}= 1/m$).

Trong ví dụ của slide (Lý Quốc Ngọc, 2026e) (Hình 6), bước tinh chỉnh dữ liệu C2 cho đúng hai trường hợp: tại $P_2(9,5)$ biên đi xuống, cạnh dưới $e_2 : (9,5) \to (9,1)$ thành $(9,4) \to (9,1)$; tại $P_6(2,7)$ biên đi lên, cạnh dưới $e_5 : (1,2) \to (2,7)$ thành $(1,2) \to (2,6)$. Slide giữ nguyên hoành độ 2 ở đầu mút mới của e_5 (giá trị đúng trên cạnh là 1.8); điều này không ảnh hưởng kết quả vì AEL chỉ dùng `y_upper` của đầu trên, còn `x_int` xuất phát từ đầu dưới. Các đỉnh P_1, P_3, P_4, P_5, P_7 đều là cực trị (sau khi bỏ cạnh ngang e_7) nên giữ nguyên.



Hình 7: Phân loại đỉnh P_i theo dấu $\Delta y_i \cdot \Delta y_{i+1}$; đường xám là dòng quét đi qua đỉnh, cạnh đỏ là cạnh bị rút ngắn một đơn vị ở đầu trên

Rút ngắn để làm gì? Tại dòng quét $y = y_i$, nếu không rút ngắn thì cả hai cạnh kề đều cho giao điểm tại x_i, tức 2 giao điểm trùng nhau. Nhưng dòng quét chỉ vượt biên *một lần* tại đỉnh không cực trị (một cạnh ở trên, một cạnh ở dưới), nên đếm 2 sẽ làm sai tính chẵn lẻ: mọi cặp lẻ–chẵn phía sau trên dòng đó bị lệch, tô ra ngoài hoặc bỏ sót miền trong. Sau khi rút ngắn, cạnh dưới kết thúc ở $y_i - 1$ nên tại dòng y_i chỉ còn cạnh trên cho đúng một giao điểm. Chiều cao bị cắt chỉ là một dòng quét của cạnh dưới, và dòng đó vẫn được cạnh trên phủ, nên không mất điểm ảnh nào trên biên.

4.21 Vì sao chỉ xử lý đỉnh không cực trị mà không xử lý đỉnh cực trị

Câu hỏi: Tại sao chỉ xét (rút ngắn) những đỉnh không cực trị, còn đỉnh cực trị thì không xét?

Vì với đỉnh cực trị, việc đếm 2 giao điểm *đã đúng sẵn*:

- Tại đỉnh cực tiểu hoặc cực đại địa phương, hai cạnh kề nằm cùng một phía của dòng quét. Dòng quét chỉ chạm biên tại một điểm rồi quay lại miền cũ (từ ngoài chạm vào rồi ra ngoài, hoặc từ trong chạm biên rồi vẫn trong), tức là vượt biên 0 hoặc 2 lần. Đếm 2 giao điểm trùng nhau giữ nguyên tính chẵn lẻ, đúng với ý nghĩa hình học theo định lý Jordan (mục 4.17).
- Với đỉnh cực trị nằm ở ngoài (ví dụ đỉnh nhọn thấp nhất), cặp giao điểm (x_i, x_i) cho một đoạn tô dài 0, tức là chỉ tô đúng điểm đỉnh, là kết quả mong muốn. Với đỉnh cực trị lõm vào trong (đỉnh của chữ V ngược bên trong đa giác), hai giao điểm trùng nhau tách một đoạn tô thành hai đoạn liền kề, cũng không sai.
- Nếu rút ngắn ở đỉnh cực trị thì sẽ còn 1 giao điểm (khi rút một cạnh) hoặc 0 giao điểm (khi rút cả hai). Trường hợp 1 giao điểm làm sai tính chẵn lẻ, giống lỗi ta đang tránh. Trường hợp 0 giao điểm vẫn đúng tính chẵn lẻ nhưng làm mất điểm ảnh ở đỉnh.

Tóm lại, mục tiêu là *số giao điểm phản ánh đúng số lần vượt biên*: đỉnh cực trị có 2 cạnh cho 2 giao điểm khớp với 0/2 lần vượt biên nên giữ nguyên; đỉnh không cực trị có 2 cạnh nhưng chỉ 1 lần vượt biên nên phải bớt 1. Ngoài ra, việc kiểm tra cực trị chỉ cần dấu của tích $\Delta y_i \cdot \Delta y_{i+1}$ (hình vẽ trên bảng), thực hiện một lần khi lập bảng cạnh, không tốn chi phí trong vòng quét.

4.22 Ý NGHĨA TỪNG TRƯỜNG TRONG CẤU TRÚC AEL

Câu hỏi: Giải thích mỗi trường trong cấu trúc dữ liệu AEL (Active Edge List) của thuật toán tô màu theo dòng quét.

Theo slide 7.3.3 (Lý Quốc Ngọc, 2026e) (bản tiếng Việt chiếu trên lớp là mục 3.2.2, 3.1):

```
struct AEL {
    int   y_upper;     // tung do lon nhat cua canh (sau khi da rut ngan)
    float x_int;       // hoanh do giao diem voi dong quet hien tai
    float reci_slope;  // nghich dao he so goc: 1/m = dx/dy
    struct AEL *next;  // con tro toi canh ke tiep trong danh sach
};
```

- `y_upper`: dòng quét cuối cùng mà cạnh còn cắt. Khi dòng quét y vượt quá `y_upper`, cạnh bị loại khỏi danh sách. Đây là trường nhận kết quả của việc rút ngắn cạnh ($y_i - 1$) ở mục 4.20. Lưu dạng số nguyên vì dòng quét là số nguyên.
- `x_int`: hoành độ giao điểm (intersection) của cạnh với dòng quét đang xử lý. Khi cạnh được đưa vào AEL, giá trị khởi tạo là hoành độ đầu dưới của cạnh. Sau mỗi dòng quét được cập nhật tăng dần. Lưu số thực vì giao điểm thường không nguyên; khi tô thì làm tròn (đầu trái làm tròn lên, đầu phải làm tròn xuống) để không tô ra ngoài biên.
- `reci_slope`: $1/m = \Delta x/\Delta y$. Khi dòng quét tăng 1 thì x trên cạnh tăng đúng $1/m$, nên `x_int += reci_slope`: tính giao điểm mới chỉ bằng một phép cộng, không cần giải lại phương trình đường thẳng (cùng tư tưởng gia số như DDA ở tuần 4). Dùng nghịch đảo hệ số góc thay vì m vì cạnh thẳng đứng có $1/m = 0$ hợp lệ, còn cạnh nằm ngang ($m = 0$) đã bị loại khỏi danh sách.
- `next`: danh sách liên kết, vì số cạnh hoạt động thay đổi liên tục (thêm cạnh mới khi dòng quét chạm đầu dưới, xóa khi vượt đầu trên). Danh sách được giữ *sắp xếp theo* `x_int` tăng dần để bước L4 ghép cặp lẻ–chẵn trực tiếp. Do các cạnh trong đa giác đơn không cắt nhau, thứ tự này ít thay đổi giữa hai dòng liên tiếp, nên sắp xếp chèn (insertion sort) gần như $O(k)$.

AEL trong toàn bộ thuật toán. AEL đi kèm bảng cạnh ET (Edge Table), khai báo trên slide là `struct AEL *ET[MAX_LINES]`: phần tử $ET[y]$ trỏ tới danh sách các cạnh có *đỉnh thấp* (đầu dưới) nằm trên dòng quét y; mỗi phần tử dùng cùng cấu trúc trên, với `x_int` khởi tạo bằng hoành độ đầu dưới. ET được lập sau bước tinh chỉnh dữ liệu (bỏ cạnh song song Ox, rút ngắn cạnh tại đỉnh không cực trị). Giải thuật trên slide (Lý Quốc Ngọc, 2026e), con trỏ `Beglist` trỏ tới đầu danh sách AEL:

L1. Lặp với mỗi dòng quét y từ `MIN_Y` đến `MAX_Y`:
L2. nếu $ET[y] \neq$ `NIL` thì
L3. thêm các cạnh do $ET[y]$ trỏ tới vào AEL (`Beglist`);
L4. nếu `Beglist` $\neq$ `NIL` thì: sắp xếp AEL theo `x_int` tăng dần; tô các điểm ảnh giữa từng cặp giao điểm lẻ–chẵn trên dòng y; xóa khỏi AEL các cạnh có `y_upper` $= y$; cập nhật `x_int += reci_slope` cho các cạnh còn lại.

(Slide đánh số dòng kết thúc vòng lặp cũng là L4.) Bảng ở mục 4.19 cho thấy trạng thái AEL qua từng dòng của ví dụ trên slide. Thứ tự "xóa rồi mới cập nhật" quan trọng: cạnh có `y_upper` $= y$ vẫn tham gia tô ở dòng y, rồi mới bị loại, nên đầu trên của cạnh không bị mất.

Nhận xét (chuẩn G5). ET là cấu trúc sắp xếp theo thùng (bucket sort) theo y, AEL là danh sách động theo x. Hai cấu trúc này biến bài toán "tìm giao điểm của mọi cạnh với mọi dòng quét" (O(số cạnh $\times$ số dòng)) thành chỉ xử lý các cạnh đang hoạt động, mỗi bước một phép cộng. Đây

là ví dụ điển hình của việc khai thác *tính liên kết* (coherence) giữa các dòng quét liên tiếp trong hệ thống đồ họa.

5 TUẦN 6: CẢI TIẾN TÔ MÀU THEO DÒNG QUÉT VÀ CÁC PHÉP BIẾN ĐỔI HÌNH HỌC 2D, 3D

5.1 TÔ MÀU THEO DÒNG QUÉT NGHIÊNG MỘT GÓC α, KHOẢNG CÁCH GIỮA HAI DÒNG LÀ a ĐIỂM ẢNH

Câu hỏi: Cải tiến thuật toán tô màu theo dòng quét khi các dòng quét nghiêng một góc α so với trục Ox và khoảng cách giữa hai dòng quét liên tiếp là a điểm ảnh (tô gạch chéo – hatching).

Phương pháp gốc trên slide 7.3.1 (Lý Quốc Ngọc, 2026e) giả thiết *dòng quét nằm ngang* và *hai dòng quét cách nhau 1 điểm ảnh*; câu hỏi này bỏ cả hai giả thiết.

Ý tưởng: đổi hệ tọa độ để dòng quét nghiêng trở thành dòng quét ngang. Gọi $\mathbf{d} = (\cos\alpha, \sin\alpha)$ là vector chỉ phương của dòng quét và $\mathbf{n} = (-\sin\alpha, \cos\alpha)$ là pháp tuyến đơn vị. Với mỗi điểm $P(x, y)$ đặt

$$t = \mathbf{d} \cdot P = x\cos\alpha + y\sin\alpha, \qquad s = \mathbf{n} \cdot P = -x\sin\alpha + y\cos\alpha, \tag{69}$$

tức là (t, s) là ảnh của (x, y) qua phép quay $R(-\alpha)$ (mục 5.5). Trong hệ (t, s), các dòng quét nghiêng có phương trình $s =$ hằng, tức là *nằm ngang*, và hai dòng liền nhau cách nhau đúng a theo phương vuông góc vì phép quay bảo toàn khoảng cách.

Thuật toán.

1. Quay mọi đỉnh: $(t_i, s_i) = R(-\alpha)(x_i, y_i)$.
2. Chạy thuật toán dòng quét của tuần 5 trong hệ (t, s) với các thay đổi: dòng quét là $s_k = s_{\min} + k\,a$, $k = 0, 1, \ldots$ (bước a thay vì 1); bảng cạnh ET phân thùng theo chỉ số dòng $k = \lceil (s_{\text{lower}} - s_{\min})/a \rceil$; trong AEL, `y_upper` thay bằng s_{upper} (số thực) hoặc chỉ số dòng cuối cùng, `x_int` thay bằng t_{int}, và `reci_slope` thay bằng $a \cdot \frac{\Delta t}{\Delta s}$ (vì mỗi bước s tăng a). Quy tắc xử lý đỉnh và ghép cặp lẻ–chẵn giữ nguyên.
3. Mỗi đoạn tô $[t_{2j-1}, t_{2j}]$ trên dòng s_k được quay ngược về (x, y) bằng $R(\alpha)$:

$$(x, y) = (t\cos\alpha - s_k\sin\alpha,\ t\sin\alpha + s_k\cos\alpha), \tag{70}$$

   rồi vẽ đoạn thẳng nối hai đầu bằng Bresenham (tuần 4), vì sau khi quay ngược, đoạn tô là một đoạn thẳng nghiêng chứ không còn là dãy điểm ảnh liên tiếp trên một hàng.

Nhận xét. Với $\alpha = 0$, $a = 1$ thuật toán trở về dạng chuẩn. Chỉ có $2m$ phép quay cho m đỉnh và hai phép quay cho mỗi đoạn tô; phần còn lại vẫn là gia số cộng như cũ. Không nên quay ảnh điểm ảnh sau khi tô ngang, vì phép quay ảnh rời rạc gây lỗ hổng và răng cưa. Kết hợp hai họ đường với α và $\alpha + 90^\circ$ được mẫu tô ô lưới chéo (cross-hatching), thường gặp trong bản vẽ kỹ thuật để ký hiệu mặt cắt vật liệu.

5.2 TÔ MÀU THEO DÒNG QUÉT VỚI BIÊN LÀ ĐƯỜNG TRÒN, ELLIPSE

Câu hỏi: Áp dụng phương pháp tô màu theo dòng quét khi biên của vùng tô là đường tròn hoặc ellipse.

Đường tròn và ellipse là các miền lồi, nên mỗi dòng quét cắt biên đúng 2 điểm (hoặc 1 điểm tại đỉnh trên/dưới). Không cần ET hay AEL: đoạn tô trên mỗi dòng có thể suy trực tiếp từ thuật toán vẽ biên.

Cách 1 – tận dụng Midpoint và tính đối xứng. Chạy thuật toán Midpoint ellipse (mục 4.1) trong góc phần tư thứ nhất. Với mỗi điểm (x, y) tìm được, đoạn ngang nối hai điểm đối xứng $(-x, y)$ và (x, y) chính là phần trong của ellipse trên dòng đó. Vì vậy tô hai đoạn:

$$[x_c - x,\ x_c + x] \times \{y_c + y\} \qquad \text{và} \qquad [x_c - x,\ x_c + x] \times \{y_c - y\}. \tag{71}$$

Ở miền 1 nhiều điểm có cùng y; chỉ tô khi y sắp đổi (tức lấy x lớn nhất của dòng) để mỗi dòng tô đúng một lần. Ở miền 2 mỗi bước y giảm 1, tô ngay. Dòng $y = 0$ chỉ tô một lần. Với đường tròn, mỗi điểm (x, y) của bát phân cho bốn dòng $y_c \pm y$ (đoạn dài $2x$) và $y_c \pm x$ (đoạn dài $2y$). Toàn bộ chỉ dùng số nguyên và phép cộng; mỗi điểm ảnh bên trong được tô đúng một lần (quan trọng khi tô ở chế độ XOR hoặc tô trong suốt).

Cách 2 – giải trực tiếp trên mỗi dòng. Với dòng y, $|y - y_c| \le r_y$, đoạn tô là

$$x \in \left[x_c - r_x\sqrt{1 - \tfrac{(y-y_c)^2}{r_y^2}},\ x_c + r_x\sqrt{1 - \tfrac{(y-y_c)^2}{r_y^2}} \right]. \tag{72}$$

Đơn giản nhưng cần căn bậc hai và số thực ở mỗi dòng. Có thể bỏ căn bằng cách dò tăng/giảm x trên dòng tiếp theo xuất phát từ x của dòng trước, kiểm tra dấu của f_{ellipse} (tính liên kết giữa các dòng).

Cách 3 – kiểm tra ẩn từng điểm. Duyệt mọi điểm trong hình chữ nhật bao, tô điểm có $f_{\text{ellipse}}(x - x_c, y - y_c) \le 0$. Tốn $O(r_x r_y)$ phép kiểm tra, nhưng mọi điểm độc lập nên dễ song song hóa trên GPU, và tổng quát cho mọi biên ẩn $F(x, y) \le 0$. Với ellipse xoay, cách 3 chỉ cần biến đổi ngược điểm ảnh về hệ của ellipse trước khi kiểm tra.

5.3 Vai trò của phép biến đổi hình học trong đồ họa máy tính

Câu hỏi: Phép biến đổi hình học có vai trò gì trong đồ họa máy tính?

Phép biến đổi hình học là ánh xạ $\mathbf{P} \mapsto \mathbf{P}'$ thay đổi vị trí, hướng, kích thước, hình dạng của đối tượng. Trong tọa độ thuần nhất, mọi phép affine 2D được viết thành ma trận 3×3 (3D thành 4×4), nên một chuỗi phép biến đổi được gộp thành một ma trận duy nhất (Hearn et al., 2011). Slide 9.1 (Lý Quốc Ngọc, 2026f) định nghĩa phép affine 2D $w(X) = AX + T$ và nêu hai tính chất: biến đường thẳng song song thành đường thẳng song song, biến điểm hữu hạn thành điểm hữu hạn; mọi phép affine được sinh ra từ các phép cơ bản (quay, tỉ lệ, đối xứng, biến dạng – shear, tịnh tiến). Vai trò:

1. **Mô hình hóa (modeling):** mô tả mỗi đối tượng một lần trong UCS rồi đặt vào WCS ở nhiều vị trí, hướng, kích thước khác nhau (instancing; nối với mục 3.1 của tuần 4). Đối tượng phân cấp (cánh tay robot, hệ Mặt Trời) được xây bằng tích các ma trận theo cây phân cấp.
2. **Quan sát (viewing):** chuyển cảnh từ WCS sang hệ tọa độ camera, chiếu xuống mặt phẳng, rồi ánh xạ cửa sổ – khung nhìn sang DCS. Toàn bộ chuỗi chuyển đổi hệ tọa độ đã học đều là phép biến đổi.
3. **Hoạt hình và mô phỏng:** chuyển động của vật thể chính là phép biến đổi phụ thuộc thời gian $M(t)$; ví dụ bánh xe vừa quay vừa tịnh tiến.
4. **Tương tác:** kéo, xoay, phóng to đối tượng; zoom, pan khung nhìn (mục 5.10).
5. **Hiệu quả tính toán:** biến đổi các điểm điều khiển thay vì từng điểm ảnh (tính bất biến affine, mục 4.9); phần cứng GPU được thiết kế để nhân ma trận 4×4 với hàng triệu đỉnh mỗi khung hình.

Liên hệ AI (G4). Tăng cường dữ liệu (data augmentation) cho mạng thị giác máy tính dùng chính các phép tịnh tiến, quay, tỉ lệ, lật, biến dạng ngẫu nhiên. Spatial Transformer Network học trực tiếp một ma trận affine 2×3 từ ảnh đầu vào để "nắn thẳng" đối tượng trước khi phân loại (Jaderberg et al., 2015). NeRF dùng ma trận tư thế camera 4×4 để biến đổi tia từ hệ camera sang hệ thế giới khi tổng hợp ảnh từ góc nhìn mới (Mildenhall et al., 2020).

5.4 Phép tịnh tiến 2D: ý nghĩa và ứng dụng

Câu hỏi: Phép tịnh tiến hai chiều (two-dimensional translation) có tác dụng gì? Nêu ý nghĩa và ứng dụng thực tế.

Định nghĩa. Tịnh tiến theo vector $\mathbf{T} = (t_x, t_y)$: $x' = x + t_x$, $y' = y + t_y$. Trong tọa độ thuần nhất:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{73}$$

Ý nghĩa.

- Phép dời hình đơn giản nhất: giữ nguyên hình dạng, kích thước, hướng; chỉ đổi vị trí. Nghịch đảo là $T(-t_x, -t_y)$, hợp hai phép tịnh tiến là cộng vector.

- Tịnh tiến *không* phải phép biến đổi tuyến tính trên $\mathbb{R}^2$ (nó không giữ gốc O), nên không viết được thành ma trận 2×2. Đây là lý do chính để đưa vào **tọa độ thuần nhất**: thêm thành phần thứ ba bằng 1 để tịnh tiến cũng trở thành phép nhân ma trận, và mọi phép affine được hợp thành bằng phép nhân.
- Tịnh tiến là "công cụ đổi tâm": quay hay tỉ lệ quanh một điểm bất kỳ được làm bằng cách tịnh tiến điểm đó về gốc, thực hiện phép biến đổi quanh gốc, rồi tịnh tiến trả về (mục 5.5, 5.6).

Ứng dụng thực tế. Di chuyển đối tượng bằng kéo–thả trong phần mềm thiết kế; hoạt hình 2D (nhân vật, sprite di chuyển trên màn hình); cuộn trang, cuộn bản đồ (pan, mục 5.10); đặt các bản sao của một mô hình (cây, xe) ở nhiều vị trí trong cảnh; chuyển gốc tọa độ từ góc trên trái của DCS về tâm cửa sổ; trong ảnh số, dịch ảnh để căn chỉnh (registration). Trong học sâu, mạng tích chập có tính *đồng biến với phép tịnh tiến* (dịch ảnh vào thì bản đồ đặc trưng dịch theo), và tịnh tiến ngẫu nhiên là một phép tăng cường dữ liệu cơ bản.

5.5 Phép quay quanh một tâm bất kỳ

Câu hỏi: Khi có phép quay quanh tâm O thì dùng công thức ở slide 6; còn phép quay quanh một tâm bất kỳ thì phải làm sao?

Quay quanh gốc O góc θ (ngược chiều kim đồng hồ), slide 6 (Lý Quốc Ngọc, 2026f):

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{74}$$

Quay quanh tâm $C(x_c, y_c)$ bất kỳ thực hiện bằng ba bước (đúng như lời giải trên bảng):

1. Tịnh tiến C về gốc: $T(-x_c, -y_c)$.
2. Quay quanh gốc: $R(\theta)$.
3. Tịnh tiến trả về: $T(x_c, y_c)$.

Ma trận hợp (áp dụng từ phải sang trái):

$$\begin{aligned} R_C(\theta) = T(x_c, y_c)\, R(\theta)\, T(-x_c, -y_c) &= \begin{bmatrix} \cos\theta & -\sin\theta & x_c \\ \sin\theta & \cos\theta & y_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_c \\ 0 & 1 & -y_c \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos\theta & -\sin\theta & x_c - x_c\cos\theta + y_c\sin\theta \\ \sin\theta & \cos\theta & y_c - x_c\sin\theta - y_c\cos\theta \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned} \tag{75}$$

Kiểm tra: thay (x_c, y_c) vào được chính nó, nên tâm quay là điểm bất động. Phần 2×2 bên trái vẫn là ma trận quay $R(\theta)$: quay quanh tâm nào thì hướng của hình cũng xoay cùng một góc; tâm chỉ ảnh hưởng tới cột tịnh tiến. Thứ tự nhân là quan trọng vì phép nhân ma trận không giao hoán: $T(x_c, y_c)R(\theta)T(-x_c, -y_c) \neq R(\theta)$ khi $C \neq O$.

5.6 Ma trận co giãn (tỉ lệ) với tâm C

Câu hỏi: Hãy xác định ma trận của phép co giãn với tâm C.

Phép tỉ lệ quanh gốc $S(s_x, s_y) = \mathrm{diag}(s_x, s_y, 1)$ vừa co giãn vừa kéo đối tượng lại gần hoặc ra xa gốc. Muốn điểm $C(x_c, y_c)$ đứng yên (điểm cố định), làm như phép quay:

$$\begin{aligned} S_C(s_x, s_y) = T(x_c, y_c)\, S(s_x, s_y)\, T(-x_c, -y_c) &= \begin{bmatrix} s_x & 0 & x_c \\ 0 & s_y & y_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_c \\ 0 & 1 & -y_c \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} s_x & 0 & x_c - s_x x_c \\ 0 & s_y & y_c - s_y y_c \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & x_c(1-s_x) \\ 0 & s_y & y_c(1-s_y) \\ 0 & 0 & 1 \end{bmatrix}, \end{aligned} \tag{76}$$

khớp với kết quả trên bảng. Tức là $x' = x_c + s_x(x - x_c)$, $y' = y_c + s_y(y - y_c)$: khoảng cách tới C theo mỗi trục được nhân với hệ số tương ứng. $|s| < 1$ là co, $|s| > 1$ là giãn, $s_x = s_y$ là phép vị tự (giữ hình dạng), $s < 0$ thêm phép lật. Tâm C thường được chọn là tâm của đối tượng (để phóng to tại chỗ) hoặc là vị trí con trỏ chuột (zoom theo con trỏ, mục 5.10). Nếu muốn co giãn theo hai trục nghiêng góc φ, ghép thêm phép quay: $T(C)\,R(\varphi)\,S(s_x, s_y)\,R(-\varphi)\,T(-C)$.

5.7 Phép đối xứng qua một trục bất kỳ

Câu hỏi: Slide 8–10 cho phép đối xứng qua Ox, Oy và qua tâm O (slide 10). Hãy tìm phép đối xứng qua một trục bất kỳ.

Các phép đối xứng cơ bản trên slide (Lý Quốc Ngọc, 2026f): qua trục Ox: $MIR(Ox) = \mathrm{diag}(1, -1, 1)$ (slide 8); qua trục Oy: $MIR(Oy) = \mathrm{diag}(-1, 1, 1)$ (slide 9); qua tâm O: $MIR(O) = \mathrm{diag}(-1, -1, 1)$ (slide 10), cũng chính là phép quay 180° quanh O và bằng $MIR(Ox)\,MIR(Oy)$. Các trục và tâm này đều đi qua gốc. Muốn đối xứng qua trục hay tâm bất kỳ, ta dùng cùng kỹ thuật như quay quanh tâm bất kỳ: *đưa trục (tâm) về vị trí chuẩn, đối xứng, rồi đưa trả về.*

Đối xứng qua tâm $C(x_c, y_c)$ bất kỳ: $MIR(C) = T(x_c, y_c)\,MIR(O)\,T(-x_c, -y_c)$, tức $x' = 2x_c - x$, $y' = 2y_c - y$.

Đối xứng qua đường $L : y = mx + b$. Gọi θ là góc của L với Ox ($\tan\theta = m$). Đưa L về trùng Ox, đối xứng, rồi đưa trả về:

1. Tịnh tiến để L đi qua gốc: $T(0, -b)$.
2. Quay $-\theta$ để L trùng Ox: $R(-\theta)$.
3. Đối xứng qua Ox: $M_x = \mathrm{diag}(1, -1, 1)$.
4. Quay trả $R(\theta)$, rồi tịnh tiến trả $T(0, b)$.

$$M_L = T(0, b)\,R(\theta)\,M_x\,R(-\theta)\,T(0, -b). \tag{77}$$

Ba ma trận ở giữa cho $R(\theta)M_xR(-\theta) = \begin{bmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{bmatrix}$, và với $\cos 2\theta = \frac{1-m^2}{1+m^2}$, $\sin 2\theta = \frac{2m}{1+m^2}$:

$$M_L = \frac{1}{1+m^2}\begin{bmatrix} 1-m^2 & 2m & -2mb \\ 2m & m^2-1 & 2b \\ 0 & 0 & 1+m^2 \end{bmatrix}. \tag{78}$$

Kiểm tra: điểm $(0, b) \in L$ được giữ nguyên; điểm $(1, m + b) \in L$ cũng được giữ nguyên; $M_L^2 = I$ (đối xứng hai lần trở về như cũ); $\det M_L = -1$ (đảo chiều hướng).

Dạng tổng quát $ax + by + c = 0$ (bao gồm cả đường thẳng đứng mà dạng $y = mx + b$ không biểu diễn được): với pháp tuyến $\mathbf{n} = (a, b)$, ảnh của P là $P' = P - 2\frac{\mathbf{n}\cdot P + c}{\|\mathbf{n}\|^2}\mathbf{n}$, tức là

$$M = \frac{1}{a^2+b^2}\begin{bmatrix} b^2-a^2 & -2ab & -2ac \\ -2ab & a^2-b^2 & -2bc \\ 0 & 0 & a^2+b^2 \end{bmatrix}. \tag{79}$$

Công thức này chỉ cần tích vô hướng, không cần hàm lượng giác (một ví dụ cho mục 5.17). Thay $a = m$, $b = -1$, $c = b_L$ (hệ số tự do của L) được lại M_L ở trên.

5.8 Vai trò của phép biến đổi trong slide trang 15

Câu hỏi: Vai trò của phép biến đổi trong slide trang 15 (ma trận WTV – biến đổi tọa độ thế giới thực sang tọa độ hiển thị).

Phép biến đổi cửa sổ – khung nhìn (window-to-viewport, w2v). Cửa sổ $[x_{w\,\min}, x_{w\,\max}] \times [y_{w\,\min}, y_{w\,\max}]$ là vùng của WCS mà ta muốn xem; khung nhìn $[x_{v\,\min}, x_{v\,\max}] \times [y_{v\,\min}, y_{v\,\max}]$

là vùng trên thiết bị nơi hiển thị. Yêu cầu: điểm ở cùng vị trí tương đối trong cửa sổ thì nằm ở cùng vị trí tương đối trong khung nhìn:

$$\frac{x_v - x_{v\,\min}}{x_{v\,\max} - x_{v\,\min}} = \frac{x_w - x_{w\,\min}}{x_{w\,\max} - x_{w\,\min}}, \qquad \frac{y_v - y_{v\,\min}}{y_{v\,\max} - y_{v\,\min}} = \frac{y_w - y_{w\,\min}}{y_{w\,\max} - y_{w\,\min}}. \tag{80}$$

Slide 14–15 (Lý Quốc Ngọc, 2026f) xây dựng ma trận này bằng hai bước: (1) tịnh tiến $T(t_x, t_y)$ với $t_x = v_{\min}.x - w_{\min}.x$, $t_y = v_{\min}.y - w_{\min}.y$, đưa góc dưới trái của cửa sổ trùng góc dưới trái của khung nhìn; (2) co giãn $S(v_{\min}, s_x, s_y)$ *với tâm* $v_{\min}$ (công thức ở mục 5.6), với

$$s_x = \frac{v_{\max}.x - v_{\min}.x}{w_{\max}.x - w_{\min}.x}, \qquad s_y = \frac{v_{\max}.y - v_{\min}.y}{w_{\max}.y - w_{\min}.y}. \tag{81}$$

Nhân hai ma trận (slide 15):

$$\begin{aligned} WTV &= S(v_{\min}, s_x, s_y)\, T(t_x, t_y) \\ &= \begin{bmatrix} s_x & 0 & v_{\min}.x - s_x v_{\min}.x \\ 0 & s_y & v_{\min}.y - s_y v_{\min}.y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & v_{\min}.x - w_{\min}.x \\ 0 & 1 & v_{\min}.y - w_{\min}.y \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} s_x & 0 & v_{\min}.x - s_x w_{\min}.x \\ 0 & s_y & v_{\min}.y - s_y w_{\min}.y \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned} \tag{82}$$

Tức là $x_v = v_{\min}.x + s_x(x_w - w_{\min}.x)$: lấy vị trí của điểm so với góc cửa sổ, đổi đơn vị bằng s_x, rồi đặt lại so với góc khung nhìn. Cách phân tích khác cho cùng kết quả là $T(v_{\min})\, S(s_x, s_y)\, T(-w_{\min})$. Kiểm tra: $w_{\min} \mapsto v_{\min}$ và $w_{\max} \mapsto v_{\max}$. Nếu DCS có trục y hướng xuống (gốc ở góc trên trái), thay y_v bằng $y_{v\,\max} - s_y(y_w - y_{w\,\min})$, tức thêm một phép đối xứng.

Vai trò.

- Cầu nối giữa thế giới thực số thực, không bị chặn (WCS) và thiết bị số nguyên, bị chặn (DCS), giải đáp câu hỏi "làm sao hiển thị thế giới rất lớn trên màn hình hữu hạn" của tuần 4.
- Làm chương trình *độc lập thiết bị*: mô hình chỉ viết trong WCS, đổi kích thước cửa sổ ứng dụng hay đổi màn hình chỉ cần thay khung nhìn.
- Là nền tảng của mọi thao tác quan sát: thay đổi cửa sổ là zoom/pan, nhiều khung nhìn cho nhiều cửa sổ là bố cục chia màn hình (ví dụ bốn hình chiếu trong phần mềm CAD).
- Giữ tỉ lệ: nếu $s_x \neq s_y$ hình bị méo (đường tròn thành ellipse); muốn giữ hình dạng phải chọn cửa sổ và khung nhìn có cùng tỉ lệ khung hình.

5.9 Sự cần thiết của việc đồng bộ viewport-to-window và window-to-viewport

Câu hỏi: Giải thích sự cần thiết về việc đồng bộ giữa phép biến đổi view-to-window (v2w) và window-to-viewport (w2v) của hệ thống đồ họa.

Hệ thống đồ họa tương tác có hai luồng dữ liệu ngược chiều nhau:

- **Luồng xuất (w2v):** đối tượng trong WCS $\rightarrow$ điểm ảnh trên khung nhìn. Dùng khi vẽ.
- **Luồng nhập (v2w):** tọa độ chuột, cảm ứng trong DCS $\rightarrow$ điểm trong WCS. Dùng khi người dùng chọn, tạo, kéo đối tượng (ví dụ hit-test điểm điều khiển Bézier ở mục 4.14). Slide 16–17 (Lý Quốc Ngọc, 2026f) xây dựng $VTW = S(w_{\min}, s'_x, s'_y)\, T(w_{\min} - v_{\min})$ với $s'_x = \frac{w_{\max}.x - w_{\min}.x}{v_{\max}.x - v_{\min}.x} = 1/s_x$ (tương tự cho y), đối xứng hoàn toàn với WTV:

$$VTW = \begin{bmatrix} s'_x & 0 & w_{\min}.x - s'_x v_{\min}.x \\ 0 & s'_y & w_{\min}.y - s'_y v_{\min}.y \\ 0 & 0 & 1 \end{bmatrix} = WTV^{-1}. \tag{83}$$

Vì sao phải đồng bộ. Người dùng nhấp vào cái mình *nhìn thấy*; điều đó chỉ đúng nghĩa nếu v2w chính xác là nghịch đảo của w2v *đang được dùng để vẽ*. Nếu sau một thao tác zoom/pan, cửa sổ đã đổi nhưng chỉ một trong hai phép được cập nhật:

- nhấp vào một điểm điều khiển nhưng hệ thống lại tìm ở vị trí khác trong WCS: chọn nhầm hoặc không chọn được;
- điểm vừa tạo bằng chuột xuất hiện ở chỗ khác chỗ đã nhấp, kéo đối tượng thì đối tượng "trôi" lệch khỏi con trỏ;
- sai số tích lũy sau mỗi lần thao tác vì dữ liệu WCS bị ghi sai.

Ngoài ra v2w và w2v phải dùng cùng quy ước (trục y của DCS hướng xuống hay lên, làm tròn điểm ảnh, tỉ lệ khung hình). Trong cài đặt, cách an toàn là chỉ lưu *một* trạng thái duy nhất (cửa sổ, khung nhìn), tính M_{w2v} từ đó và luôn lấy M_{v2w} là nghịch đảo của nó mỗi khi trạng thái thay đổi; mọi thao tác zoom/pan chỉ thay đổi trạng thái này. Tính đồng bộ đó chính là điều kiện để vòng lặp tương tác *nhập* $\rightarrow$ *cập nhật mô hình* $\rightarrow$ *vẽ* của phần mềm đồ họa nhất quán.

5.10 Toán tử zoom (đầu vào, đầu ra) và toán tử pan

Câu hỏi: Viết toán tử zoom (nêu rõ đầu vào, đầu ra).

Zoom không thay đổi mô hình hay khung nhìn, mà chỉ *thay đổi cửa sổ* rồi tính lại w2v. Cửa sổ nhỏ đi thì vật hiện ra to hơn (phóng to) và ngược lại.

Giải thuật 4 Toán tử ZOOM

Đầu vào: Cửa sổ hiện tại $W = [x_{w\min}, x_{w\max}] \times [y_{w\min}, y_{w\max}]$; hệ số $k > 0$ ($k > 1$ phóng to, $k < 1$ thu nhỏ); điểm neo (x_a, y_a) trong DCS (vị trí con trỏ chuột hoặc tâm khung nhìn); khung nhìn V

Đầu ra: Cửa sổ mới W' và cặp ma trận M_{w2v}, M_{v2w} mới

1: $(x_A, y_A) \leftarrow M_{v2w}(x_a, y_a)$ ▷ đổi điểm neo sang WCS
2: $x'_{w\min} \leftarrow x_A + (x_{w\min} - x_A)/k$; $x'_{w\max} \leftarrow x_A + (x_{w\max} - x_A)/k$
3: $y'_{w\min} \leftarrow y_A + (y_{w\min} - y_A)/k$; $y'_{w\max} \leftarrow y_A + (y_{w\max} - y_A)/k$
4: Tính lại M_{w2v} từ (W', V) và $M_{v2w} \leftarrow M_{w2v}^{-1}$
5: Vẽ lại cảnh

Dòng 2–3 chính là phép co giãn cửa sổ với hệ số $1/k$ quanh tâm A (mục 5.6). Vì A là điểm cố định của phép co, điểm dưới con trỏ chuột đứng yên trên màn hình trong khi mọi thứ khác phóng ra xa nó, cho cảm giác zoom tự nhiên như trong bản đồ số. Tương đương, so với M_{w2v} cũ, phép zoom nhân thêm $S_a(k, k)$ (tỉ lệ k quanh điểm neo trong DCS). Có thể giới hạn k tích lũy để tránh cửa sổ quá nhỏ (vượt độ chính xác số thực) hoặc quá lớn.

Toán tử Pan. Đầu vào: cửa sổ W, độ dời chuột $(\Delta x_v, \Delta y_v)$ trong DCS. Đổi sang WCS: $\Delta x_w = \Delta x_v / s_x$, $\Delta y_w = \Delta y_v / s_y$ (đổi dấu Δy nếu trục y của DCS hướng xuống). Đầu ra: $W' = W - (\Delta x_w, \Delta y_w)$, tức tịnh tiến cửa sổ *ngược* hướng kéo để nội dung dịch theo hướng kéo; sau đó tính lại w2v, v2w và vẽ lại.

5.11 Vai trò của v2w, w2v, zoom, pan trong quy trình hoạt động của phần mềm đồ họa

Câu hỏi: Giải thích vai trò của v2w, w2v, zoom, pan trong quy trình hoạt động của phần mềm đồ họa.

Một phần mềm đồ họa tương tác (CAD, phần mềm vẽ vector, bản đồ số) lặp lại chu trình sau:

1. **Nhập (v2w):** sự kiện chuột/cảm ứng cho tọa độ DCS; v2w đổi sang WCS để biết người dùng trỏ vào đâu trong thế giới mô hình.
2. **Xử lý:**
   - thao tác *trên mô hình* (tạo điểm, kéo điểm điều khiển, biến đổi đối tượng) thay đổi dữ liệu WCS;
   - thao tác *trên góc nhìn* (zoom, pan) chỉ thay đổi cửa sổ, không đụng tới dữ liệu.
3. **Xuất (w2v):** cắt xén các đối tượng ngoài cửa sổ, ánh xạ phần còn lại sang khung nhìn, rời rạc hóa bằng các thuật toán vẽ đoạn thẳng, đường tròn, Bézier, tô màu đã học.

Thành phần	Vai trò	Thay đổi gì
w2v	Hiển thị: WCS → DCS; quyết định phần nào của thế giới, ở tỉ lệ nào, hiện ở đâu trên màn hình	Không thay đổi dữ liệu, chỉ đọc
v2w	Tương tác: DCS → WCS; cho phép chọn, tạo, sửa đối tượng bằng chuột	Không thay đổi dữ liệu; là nghịch đảo của w2v
Zoom	Thay đổi mức chi tiết quan sát (tỉ lệ cửa sổ)	Kích thước cửa sổ (co giãn quanh điểm neo)
Pan	Thay đổi vùng quan sát ở cùng mức chi tiết	Vị trí cửa sổ (tịnh tiến)

Nhờ tách *mô hình* (WCS) khỏi *góc nhìn* (cửa sổ, khung nhìn), dữ liệu được lưu chính xác bằng số thực và không bị suy giảm khi người dùng zoom/pan nhiều lần; còn zoom và pan chỉ là phép tỉ lệ và tịnh tiến trên bốn số của cửa sổ, rất rẻ. Zoom cũng gắn với mức chi tiết (Level of Detail): ở mức thu nhỏ có thể vẽ đường cong với ít đoạn thẳng hơn, ở mức phóng to thì chia mịn hơn (mục 4.3). Về cấu trúc dữ liệu (G5), toàn bộ trạng thái góc nhìn gói gọn trong một cấu trúc `View { Rect window; Rect viewport; Mat3 w2v, v2w; }`.

5.12 Vai trò của các phép biến đổi 3D

Câu hỏi: Vai trò của các phép biến đổi 3D.

Các phép biến đổi 3D (tịnh tiến, tỉ lệ, quay quanh Ox, Oy, Oz hoặc quanh trục bất kỳ, đối xứng qua mặt phẳng) được biểu diễn bằng ma trận 4×4 trong tọa độ thuần nhất $(x, y, z, 1)$ (Hearn et al., 2011). Chúng đảm nhận:

- **Biến đổi mô hình:** đặt các vật thể 3D vào cảnh với vị trí, hướng, kích thước mong muốn; xây dựng mô hình phân cấp (khung xương nhân vật, robot).
- **Biến đổi quan sát:** đặt camera ở vị trí, hướng bất kỳ; chuyển cả cảnh sang hệ tọa độ camera (camera ở gốc, nhìn theo $-z$). Ma trận này xây dựng từ phép tịnh tiến và phép quay đưa một vector (hướng nhìn) trùng trục Oz (mục 5.15).
- **Phép chiếu:** chuyển 3D thành 2D (mục 5.13); phép chiếu phối cảnh cũng là một ma trận 4×4 nhờ tọa độ thuần nhất.
- **Hoạt hình, mô phỏng vật lý, thực tế ảo:** chuyển động của vật rắn là tích của phép quay và tịnh tiến; theo dõi đầu người dùng trong VR là cập nhật ma trận quan sát mỗi khung hình.
- **Biến đổi vector pháp tuyến** cho chiếu sáng: pháp tuyến phải được biến đổi bằng $(M^{-1})^T$ để còn vuông góc với bề mặt khi M có tỉ lệ không đều.

Liên hệ AI (G4): tái tạo 3D từ ảnh (Structure from Motion, NeRF) chính là bài toán ước lượng ma trận biến đổi $[R \mid \mathbf{t}]$ của từng camera (Mildenhall et al., 2020); các mô hình sinh nội dung 3D cũng dùng các ma trận này để kết xuất vật thể từ nhiều góc nhìn khi huấn luyện.

5.13 Màn hình là 2 chiều thì biến đổi 3D như thế nào

Câu hỏi: Màn hình chỉ có 2 chiều thì phép biến đổi 3D được thực hiện (và hiển thị) như thế nào?

Mọi phép biến đổi 3D được thực hiện *trong không gian 3D* trên dữ liệu số thực (tọa độ (x, y, z) của các đỉnh); màn hình 2D chỉ xuất hiện ở bước cuối qua **phép chiếu**. Đường ống hiển thị 3D:

$$\text{UCS} \xrightarrow{M_{\text{model}}} \text{WCS} \xrightarrow{M_{\text{view}}} \text{tọa độ camera} \xrightarrow{M_{\text{proj}}} \text{mặt phẳng chiếu (2D + độ sâu)} \xrightarrow{M_{w2v}} \text{DCS}. \quad (84)$$

- **Chiếu song song (trực giao):** bỏ tọa độ z, $(x, y, z) \mapsto (x, y)$. Giữ đúng tỉ lệ, dùng trong bản vẽ kỹ thuật.
- **Chiếu phối cảnh:** camera ở gốc, mặt phẳng chiếu $z = d$; theo tam giác đồng dạng

$$x_p = \frac{d\,x}{z}, \qquad y_p = \frac{d\,y}{z}. \quad (85)$$

Vật càng xa (z lớn) càng nhỏ, cho cảm giác chiều sâu. Phép chia cho z không tuyến tính, nhưng trong tọa độ thuần nhất được viết thành ma trận 4×4 cho ra $(x, y, z, z/d)$ rồi chia cho thành phần thứ tư (phép chia phối cảnh).

Hiệu ứng "3D" trên màn hình 2D đến từ việc biến đổi được thực hiện trước, chiếu sau: quay vật thể quanh trục rồi chiếu thì hình chiếu thay đổi theo cách mắt người diễn giải là chuyển động 3D. Thêm vào đó là các tín hiệu chiều sâu: phối cảnh, khử mặt khuất bằng bộ đệm độ sâu (z-buffer, nên vẫn giữ z sau khi chiếu), chiếu sáng và đổ bóng. Lưu ý phép chiếu phối cảnh không phải phép affine nên phải biến đổi và chiếu các điểm, không thể chỉ chiếu điểm điều khiển của Bézier (mục 4.8).

5.14 Phép quay quanh một trục bất kỳ trong không gian 3D

Câu hỏi: Tìm phép biến đổi quay một góc θ quanh một trục bất kỳ trong hệ tọa độ 3D.

Trục quay đi qua $P_1(x_1, y_1, z_1)$ và $P_2(x_2, y_2, z_2)$. Vector đơn vị chỉ phương:

$$\mathbf{u} = \frac{P_2 - P_1}{\|P_2 - P_1\|} = (a, b, c), \qquad a^2 + b^2 + c^2 = 1, \qquad d = \sqrt{b^2 + c^2}. \tag{86}$$

Ý tưởng (slide 22–34 (Lý Quốc Ngọc, 2026f), theo Hearn & Baker (Hearn et al., 2011)): đưa trục về trùng Oz, quay quanh Oz, rồi đưa trả về. Các bước:

1. Tịnh tiến P_1 về gốc: $T(-x_1, -y_1, -z_1)$.
2. Quay quanh Ox góc α để $\mathbf{u}$ nằm trong mặt phẳng xz: $R_x(\alpha)$ với $\cos\alpha = c/d$, $\sin\alpha = b/d$ (mục 5.16).
3. Quay quanh Oy góc β để $\mathbf{u}$ trùng Oz: $R_y(\beta)$ với $\cos\beta = d$, $\sin\beta = -a$.
4. Quay quanh Oz góc θ: $R_z(\theta)$.
5. Thực hiện ngược các bước 3, 2, 1.

Theo ký hiệu slide 23:

$$R(\vec{V}, \theta) = T^{-1}(\vec{P_1O})\, A(\vec{V})^{-1}\, R(Oz, \theta)\, A(\vec{V})\, T(\vec{P_1O}), \tag{87}$$

trong đó $T(\vec{P_1O}) = T(-x_1, -y_1, -z_1)$ (slide 25) và $A(\vec{V})$ là phép quay đưa $\vec{V}$ về trùng Oz, gồm quay quanh Ox góc α rồi quay quanh Oy góc β. Với quy ước nhân ma trận vào vector cột (như toàn bộ slide, $P' = M \cdot P$), phép áp dụng trước đứng bên phải nên

$$A(\vec{V}) = R(Oy, \beta)\, R(Ox, \alpha), \qquad A(\vec{V})^{-1} = R(Ox, \alpha)^{-1}\, R(Oy, \beta)^{-1}. \tag{88}$$

Slide 23 viết $A(\vec{V}) = R(Ox, \alpha)R(Oy, \beta)$, tức là liệt kê theo thứ tự thực hiện; khi nhân ma trận cần đảo lại như trên (công thức A^{-1} trên slide đã đúng thứ tự). Các ma trận thành phần (slide 30, 33), với $R_x^{-1}(\alpha) = R_x^T(\alpha)$ vì là ma trận quay trực chuẩn:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{c}{d} & -\frac{b}{d} & 0 \\ 0 & \frac{b}{d} & \frac{c}{d} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad R_y(\beta) = \begin{bmatrix} d & 0 & -a & 0 \\ 0 & 1 & 0 & 0 \\ a & 0 & d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{89}$$

Giải thích bước 3: sau bước 2, $\mathbf{u}$ thành $\mathbf{u}'' = (a, 0, d)$. Góc giữa $\mathbf{u}''$ và $\mathbf{u}_z$ có $\cos\beta = \mathbf{u}'' \cdot \mathbf{u}_z = d$ và $\mathbf{u}'' \times \mathbf{u}_z = \mathbf{u}_y(-a)$, nên $\sin\beta = -a$ (dấu âm vì quay từ $\mathbf{u}''$ tới $\mathbf{u}_z$ là chiều âm quanh Oy). Kiểm tra: $R_y(\beta)(a, 0, d)^T = (ad - ad,\ 0,\ a^2 + d^2)^T = (0, 0, 1)^T$.

Trường hợp đặc biệt: nếu $d = 0$ thì trục trùng Ox, bỏ bước 2 ($R_x = I$) và quay trực tiếp bằng $R_x(\theta)$.

Dạng gọn (công thức Rodrigues) cho phần quay 3×3 quanh trục qua gốc:

$$R_{\mathbf{u}}(\theta) = \cos\theta\, I + (1 - \cos\theta)\, \mathbf{u}\mathbf{u}^T + \sin\theta \begin{bmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{bmatrix}, \tag{90}$$

dùng để kiểm tra kết quả nhân 7 ma trận. Trong thực hành (camera trackball, hoạt hình) người ta còn dùng quaternion để nội suy mượt giữa hai hướng quay.

5.15 Phép biến đổi quay vector $\mathbf{v}$ trùng với trục Oz

Câu hỏi: Tìm phép biến đổi để quay một vector $\mathbf{v}$ trùng với trục Oz.

Đây là bước 2–3 của mục 5.14. Với $\mathbf{u} = \mathbf{v}/\|\mathbf{v}\| = (a, b, c)$, $d = \sqrt{b^2 + c^2} \neq 0$:

$$M = R_y(\beta)\, R_x(\alpha) = \begin{bmatrix} d & 0 & -a \\ 0 & 1 & 0 \\ a & 0 & d \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{c}{d} & -\frac{b}{d} \\ 0 & \frac{b}{d} & \frac{c}{d} \end{bmatrix} = \begin{bmatrix} d & -\frac{ab}{d} & -\frac{ac}{d} \\ 0 & \frac{c}{d} & -\frac{b}{d} \\ a & b & c \end{bmatrix}. \tag{91}$$

Kiểm tra: $M\mathbf{u} = \left(ad - \frac{a(b^2+c^2)}{d},\ \frac{bc-cb}{d},\ a^2+b^2+c^2\right) = (0, 0, 1)$. Vậy $M\mathbf{v} = (0, 0, \|\mathbf{v}\|)$.

Nhận xét: hàng thứ ba của M chính là $\mathbf{u}$, và ba hàng của M là ba vector đơn vị đôi một vuông góc. Điều này cho một cách xây dựng nhanh: bất kỳ ma trận nào có ba hàng là một bộ trực chuẩn thuận $(\mathbf{r}_1, \mathbf{r}_2, \mathbf{u})$ đều quay $\mathbf{u}$ về Oz, với $\mathbf{r}_1, \mathbf{r}_2$ tìm bằng tích có hướng. Đây chính là cách xây ma trận quan sát "look-at" của camera: lấy $\mathbf{u}$ là hướng nhìn, $\mathbf{r}_1 = \frac{\mathbf{up}\times\mathbf{u}}{\|\mathbf{up}\times\mathbf{u}\|}$, $\mathbf{r}_2 = \mathbf{u} \times \mathbf{r}_1$.

5.16 Chứng minh hai góc α trong hình trang 28 bằng nhau

Câu hỏi: Chứng minh hai góc α trong hình ở slide trang 28 là như nhau.

Slide 28 (Lý Quốc Ngọc, 2026f) có hai hình. Hình trái: α là *góc giữa hình chiếu* $\mathbf{u}'$ của $\mathbf{u}$ lên mặt phẳng yz và vector $\mathbf{u}_z = (0, 0, 1)$. Hình phải (a): α là *góc quay quanh trục Ox* để đưa $\mathbf{u}$ vào mặt phẳng xz (bước 2 của phép quay quanh trục bất kỳ). Cần chứng minh hai góc này bằng nhau, để có thể tính góc quay từ hình chiếu.

Thiết lập. $\mathbf{u} = (a, b, c)$, hình chiếu lên mặt phẳng yz là $\mathbf{u}' = (0, b, c)$ với $\|\mathbf{u}'\| = d$.

Góc thứ nhất $\alpha_1 = \angle(\mathbf{u}', \mathbf{u}_z)$, tính bằng tích vô hướng và tích có hướng:

$$\cos\alpha_1 = \frac{\mathbf{u}' \cdot \mathbf{u}_z}{\|\mathbf{u}'\|\,\|\mathbf{u}_z\|} = \frac{c}{d}, \qquad \sin\alpha_1 = \frac{b}{d} \ \text{(mục 5.18)}. \tag{92}$$

Góc thứ hai α_2 là góc quay quanh Ox sao cho $R_x(\alpha_2)\,\mathbf{u}$ có thành phần y bằng 0 và thành phần z dương. Phép quay quanh Ox giữ nguyên x và tác động lên (y, z) như một phép quay phẳng:

$$y' = y\cos\alpha_2 - z\sin\alpha_2, \qquad z' = y\sin\alpha_2 + z\cos\alpha_2. \tag{93}$$

Áp dụng cho $\mathbf{u}$: điều kiện $y' = b\cos\alpha_2 - c\sin\alpha_2 = 0$ và $z' > 0$ cho $\tan\alpha_2 = b/c$ với $(\cos\alpha_2, \sin\alpha_2)$ cùng dấu với (c, b), tức $\cos\alpha_2 = c/d$, $\sin\alpha_2 = b/d$. Khi đó $z' = (b^2 + c^2)/d = d$.

Kết luận. α_1 và α_2 có cùng $\cos$ và cùng $\sin$ nên bằng nhau. Về trực giác: phép quay quanh Ox không thay đổi thành phần x, nên quay $\mathbf{u}$ cũng chính là quay hình chiếu $\mathbf{u}'$ của nó trong mặt phẳng yz; muốn $\mathbf{u}'$ trùng Oz thì phải quay đúng bằng góc giữa $\mathbf{u}'$ và Oz. Đây là lý do có thể tính góc quay α chỉ từ hình chiếu $\mathbf{u}'$ mà không cần biết a.

5.17 Vì sao tích vô hướng và tích có hướng là công cụ hiệu quả trong đồ họa máy tính

Câu hỏi: Giải thích vì sao tích vô hướng và tích có hướng là công cụ được sử dụng hiệu quả trong đồ họa máy tính, liệt kê một số ứng dụng.

Vì sao hiệu quả.

- **Mã hóa hình học bằng đại số đơn giản:** $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\varphi$ cho góc và hình chiếu; $\mathbf{a} \times \mathbf{b}$ cho vector vuông góc với cả hai, có độ lớn $\|\mathbf{a}\|\|\mathbf{b}\|\sin\varphi$ (diện tích hình bình hành) và chiều theo quy tắc bàn tay phải. Góc, vuông góc, song song, diện tích, hướng (trái/phải, trong/ngoài) đều được trả lời không cần hàm lượng giác.
- **Rẻ và ổn định:** tích vô hướng 3D chỉ cần 3 phép nhân, 2 phép cộng; tích có hướng 6 phép nhân, 3 phép trừ. Không có phép chia hay hàm siêu việt; nhiều phép thử chỉ cần *dấu* của kết quả nên không cần chuẩn hóa vector. GPU có lệnh phần cứng cho các phép này.

- **Độc lập hệ tọa độ:** kết quả bất biến dưới phép quay (tích vô hướng) hoặc biến đổi theo phép quay (tích có hướng), nên các công thức đúng trong mọi hệ tọa độ.

Một số ứng dụng.

- *Tích vô hướng:* góc giữa hai vector; hình chiếu điểm lên trục (khoảng cách điểm – đường thẳng, điểm – mặt phẳng, phép đối xứng ở mục 5.7); khử mặt sau (back-face culling: $\mathbf{N} \cdot \mathbf{V} > 0$ thì mặt quay lưng); chiếu sáng khuếch tán Lambert $I = k_d I_l \max(0, \mathbf{N} \cdot \mathbf{L})$ và phản xạ Phong $(\mathbf{R} \cdot \mathbf{V})^n$; tìm giao tia – mặt phẳng $t = \frac{(\mathbf{P}_0 - \mathbf{O}) \cdot \mathbf{N}}{\mathbf{D} \cdot \mathbf{N}}$; tính $\cos\alpha$ trong phép quay quanh trục.
- *Tích có hướng:* pháp tuyến mặt tam giác $\mathbf{N} = (\mathbf{V}_1 - \mathbf{V}_0) \times (\mathbf{V}_2 - \mathbf{V}_0)$; diện tích tam giác, đa giác; phép thử điểm bên trái/phải đường thẳng (tính lồi, điểm trong tam giác, điểm trong cung tròn ở mục 4.1, điểm trong đa giác lồi ở mục 4.16); chiều duyệt đỉnh (CW/CCW); dựng hệ trục trực chuẩn cho camera (look-at, mục 5.15); tính $\sin\alpha$, $\sin\beta$ trong phép quay quanh trục bất kỳ.
- *Liên hệ AI (G4):* độ tương đồng cosin giữa các vector nhúng (embedding) trong tìm kiếm ảnh, CLIP chính là tích vô hướng của hai vector đã chuẩn hóa; cơ chế attention trong Transformer tính điểm tương quan $QK^T/\sqrt{d_k}$, tức là một ma trận các tích vô hướng (Vaswani et al., 2017); ước lượng pháp tuyến cho đám mây điểm 3D dùng tích có hướng của các vector lân cận.

5.18 Nguồn gốc dòng 2 của tích có hướng trong slide trang 29

Câu hỏi: Tích có hướng ở dòng 2 của slide trang 29, $\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x\,|\mathbf{u}'|\,|\mathbf{u}_z|\sin\alpha$, ở đâu ra?

Slide 29 (Lý Quốc Ngọc, 2026f) có ba dòng: (1) $\cos\alpha = \frac{\mathbf{u}' \cdot \mathbf{u}_z}{|\mathbf{u}'|\,|\mathbf{u}_z|} = \frac{c}{d}$; (2) $\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x\,|\mathbf{u}'|\,|\mathbf{u}_z|\sin\alpha$; (3) $\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x \cdot b$, suy ra $\sin\alpha = \frac{b}{d}$. Dòng 2 và dòng 3 là cùng một tích có hướng $\mathbf{u}' \times \mathbf{u}_z$ (với $\mathbf{u}' = (0, b, c)$, $\mathbf{u}_z = (0, 0, 1)$) được tính theo hai cách.

Dòng 2 – định nghĩa hình học của tích có hướng: với hai vector $\mathbf{a}, \mathbf{b}$ hợp góc φ, $\mathbf{a} \times \mathbf{b} = \mathbf{n}\,|\mathbf{a}|\,|\mathbf{b}|\sin\varphi$, trong đó $\mathbf{n}$ là vector đơn vị vuông góc với mặt phẳng chứa $\mathbf{a}, \mathbf{b}$, chiều theo quy tắc bàn tay phải. Ở đây $\mathbf{u}'$ và $\mathbf{u}_z$ cùng nằm trong mặt phẳng yz, nên vector đơn vị vuông góc với mặt phẳng đó là $\mathbf{u}_x = (1, 0, 0)$ (bàn tay phải quay từ $\mathbf{u}'$ về $\mathbf{u}_z$, chiều này chính là chiều dương của góc quay quanh Ox). Thay $\mathbf{n} = \mathbf{u}_x$, $\varphi = \alpha$, $|\mathbf{u}'| = d$, $|\mathbf{u}_z| = 1$:

$$\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x\,|\mathbf{u}'|\,|\mathbf{u}_z|\sin\alpha = \mathbf{u}_x\,d\sin\alpha. \tag{94}$$

Dòng 3 – định thức theo tọa độ (đúng như lời giải trên bảng):

$$\mathbf{u}' \times \mathbf{u}_z = \begin{vmatrix} \mathbf{u}_x & \mathbf{u}_y & \mathbf{u}_z \\ 0 & b & c \\ 0 & 0 & 1 \end{vmatrix} = \mathbf{u}_x(b \cdot 1 - c \cdot 0) - \mathbf{u}_y(0 \cdot 1 - c \cdot 0) + \mathbf{u}_z(0 \cdot 0 - b \cdot 0) = \mathbf{u}_x\,b. \tag{95}$$

Trên bảng, đường chéo vẽ trong định thức chính là phép nhân chéo $b \cdot 1 - c \cdot 0$ của định thức con ứng với $\mathbf{u}_x$; hai định thức con còn lại bằng 0 vì cột đầu toàn số 0.

So sánh dòng 2 và dòng 3: $d\sin\alpha = b$, suy ra

$$\sin\alpha = \frac{b}{d}. \tag{96}$$

Kết hợp với dòng 1, $\cos\alpha = \frac{\mathbf{u}' \cdot \mathbf{u}_z}{d} = \frac{c}{d}$, ta có đủ hai phần tử để viết $R_x(\alpha)$ trong mục 5.14 mà không cần tính góc α bằng hàm lượng giác ngược. Tích vô hướng chỉ cho $\cos\alpha$, không phân biệt được α với $-\alpha$; tích có hướng cho thêm $\sin\alpha$ có dấu, xác định đúng chiều quay. Slide 32 làm tương tự cho góc β: $\mathbf{u}'' \times \mathbf{u}_z = \mathbf{u}_y\,|\mathbf{u}''|\,|\mathbf{u}_z|\sin\beta$ và $\mathbf{u}'' \times \mathbf{u}_z = \mathbf{u}_y(-a)$, cho $\sin\beta = -a$.

Tài liệu

Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.

Herbert Bay, Andreas Ess, Tinne Tuytelaars, and Luc Van Gool. Speeded-up robust features (SURF). *Computer Vision and Image Understanding*, 110(3):346–359, 2008. doi: 10.1016/j.cviu.2007.09.014.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

Jack E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965. doi: 10.1147/sj.41.0025.

Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, pp. 213–229. Springer, 2020.

James H. Clark. Hierarchical geometric models for visible surface algorithms. *Communications of the ACM*, 19(10):547–554, 1976. doi: 10.1145/360349.360354.

Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.

Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven C. H. Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 49250–49267, 2023.

Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pp. 886–893, 2005. doi: 10.1109/CVPR.2005.177.

Henghui Ding, Chang Liu, Shuting He, Kaining Ying, Xudong Jiang, Chen Change Loy, and Yu-Gang Jiang. Mevis: A multi-modal dataset for referring motion expression video segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(12):11400–11416, 2025. doi: 10.1109/TPAMI.2025.3600507.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Gerald Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Morgan Kaufmann, San Francisco, CA, 5th edition, 2002.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991. doi: 10.1145/103162.103163.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pp. 2672–2680, 2014.

Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. Hallusionbench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14375–14385, 2024.

Thomas C. Hales. The Jordan curve theorem, formally and informally. *The American Mathematical Monthly*, 114(10):882–894, 2007. doi: 10.1080/00029890.2007.11920481.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90.

Donald Hearn, M. Pauline Baker, and Warren R. Carithers. *Computer Graphics with OpenGL*. Pearson Prentice Hall, Upper Saddle River, NJ, 4th edition, 2011.

Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 6840–6851, 2020.

John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley. *Computer Graphics: Principles and Practice*. Addison-Wesley, Upper Saddle River, NJ, 3rd edition, 2014.

IEEE Computer Society. IEEE standard for floating-point arithmetic (IEEE std 754-2019). Institute of Electrical and Electronics Engineers, 2019.

Max Jaderberg, Karen Simonyan, Andrew Zisserman, and Koray Kavukcuoglu. Spatial transformer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pp. 2017–2025, 2015.

Ajay Jain, Amber Xie, and Pieter Abbeel. VectorFusion: Text-to-SVG by abstracting pixel-based diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1911–1920, 2023.

Hervé Jégou, Matthieu Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pp. 1097–1105, 2012.

Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955. doi: 10.1002/nav.3800020109.

Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*, pp. 12888–12900. PMLR, 2022.

Lin Li, Haoyan Guan, Jianing Qiu, and Michael Spratling. One prompt word is enough to boost adversarial robustness for pre-trained vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 24676–24685, 2024.

Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)*, 39(6), 2020. doi: 10.1145/3414685.3417871.

Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 292–305, 2023.

Yuhong Li, Xiaofan Zhang, and Deming Chen. CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1091–1100, 2018.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 34892–34916, 2023.

Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pp. 21–37. Springer, 2016.

Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022, 2021.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. doi: 10.1023/B:VISI.0000029664.99615.94.

Lý Quốc Ngọc. Computer graphics – lecture 3: Line-drawing method. Slide bài giảng môn Đồ họa Máy tính, Khoa Công nghệ Thông tin, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026a.

Lý Quốc Ngọc. Computer graphics – lecture 4: Circle-drawing method. Slide bài giảng môn Đồ họa Máy tính, Khoa Công nghệ Thông tin, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026b.

Lý Quốc Ngọc. Computer graphics – lecture 5: Ellipse-drawing method. Slide bài giảng môn Đồ họa Máy tính, Khoa Công nghệ Thông tin, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026c.

Lý Quốc Ngọc. Computer graphics – lecture 6: Bezier-drawing method. Slide bài giảng môn Đồ họa Máy tính, Khoa Công nghệ Thông tin, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026d.

Lý Quốc Ngọc. Computer graphics – lecture 7: Fill method. Slide bài giảng môn Đồ họa Máy tính, Khoa Công nghệ Thông tin, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026e.

Lý Quốc Ngọc. Computer graphics – lecture 9: Geometric transformations. Slide bài giảng môn Đồ họa Máy tính, Khoa Công nghệ Thông tin, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026f.

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 405–421, 2020.

OpenAI. GPT-4V(ision) system card. *OpenAI Technical Report*, 2023. URL `https://openai.com/research/gpt-4v-system-card`.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4195–4205, 2023.

Jean Piaget. *The Psychology of Intelligence*. Routledge & Kegan Paul, London, 1950. URL `http://www.jstor.org/stable/2088298`. Translated by Malcolm Piercy and D. E. Berlyne; Original French edition 1947.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pp. 8748–8763. PMLR, 2021.

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pp. 91–99, 2015.

Hamid Rezatofighi, Nathan Tsoi, Jianwei Geng, Rui Deng, Stefan Leutenegger, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 658–666, 2019.

Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.

Juan A. Rodriguez, Abhay Puri, Shubham Agarwal, Issam H. Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli. StarVector: Generating scalable vector graphics code from images and text. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16175–16186, 2025.

M. Shimrat. Algorithm 112: Position of point relative to polygon. *Communications of the ACM*, 5 (8):434, 1962. doi: 10.1145/368637.368653.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pp. 5998–6008, 2017.

W3C. Scalable vector graphics (SVG) 2. W3C Candidate Recommendation Snapshot, 06 October 2026, 2026. URL `https://www.w3.org/TR/SVG2/`.

Boyu Wang, Huidong Liu, Dimitris Samaras, and Minh Hoai. Distribution matching for crowd counting. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 1595–1607, 2020.

Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. SVGDreamer: Text guided SVG generation with diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4546–4555, 2024.

Chuanqi Zang, Hanqing Wang, Mingtao Pei, and Wei Liang. Discovering the real association: Multimodal causal reasoning in video question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 19027–19036, 2023.

Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, pp. 1–21. Springer, 2022.

Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-IoU loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 34, pp. 12993–12900, 2020.